Universitat Autònoma de Barcelona

Treball de Fi de Grau

Grau en Enginyeria Electrònica de Telecomunicació

Disseny i implementació d'un prototip d'antena rotatòria per al seguiment de satèl·lits LEO.

Jan Moran Ricardo

Director: José A. López Salcedo

Departament de Telecomunicació i Enginyeria de Sistemes

Universitat Autònoma de Barcelona

Bellaterra, Febrer 01, 2026

Abstract

This Final Degree Project describes the design, implementation, and validation of a low-cost rotary antenna prototype for tracking Low Earth Orbit (LEO) satellites. The project arises from the current increase in market demand for LEO satellite constellations, which are essential for global connectivity and Positioning, Navigation, and Timing (PNT) services. Since these constellations consist of numerous high-speed satellites with changing orbits, they require ground stations capable of precise real-time tracking, hence the necessity of this work. The proposed system integrates a mechanical structure driven by NEMA-23 stepper motors, a control layer based on Arduino, and a software environment developed in C. The tracking algorithm utilizes the SGP4 propagation model and TLE data to calculate the satellite's position (Azimuth and Elevation) relative to the observer. Experimental results, obtained by tracking the Globalstar constellation with a Software Defined Radio (SDR) receiver, demonstrate a significant improvement in signal quality and link stability compared to omnidirectional antennas. This project proves that it is possible to develop a functional and accurate tracking station using accessible components and open-source software.

Resum

Aquest Treball de Fi de Grau descriu el disseny, la implementació i la validació d'un prototip d'antena rotatòria de baix cost per al seguiment de satèl·lits en òrbita terrestre baixa (LEO). El projecte origina de l'actual increment de la demanda del mercat per les constel·lacions de satèl·lits LEO, útils per a la connectivitat global i el posicionament (PNT). Aquestes constel·lacions, en estar conformades per molts satèl·lits d'alta velocitat i d'òrbites canviants, requereixen estacions terrestres capaces de realitzar un seguiment precís en temps real, d'aquí la necessitat d'aquest treball. El sistema proposat integra una estructura mecànica moguda per motors pas a pas NEMA-23, una capa de control basada en Arduino i un entorn de programari desenvolupat en C. L'algorisme de seguiment utilitza el model de propagació SGP4 i dades TLE per calcular la posició del satèl·lit (azimut i elevació) respecte a l'observador. Els resultats experimentals, obtinguts mitjançant el seguiment de la constel·lació Globalstar amb un receptor de ràdio definida per programari (SDR), demostren una millora significativa en la qualitat del senyal i l'estabilitat de l'enllaç respecte a les antenes omnidireccionals. El projecte demostra que és possible desenvolupar una estació de seguiment funcional i precisa utilitzant components accessibles i programari de codi obert.

Índex

Abstract **i**

Resum **iii**

List of Figures **xii**

Índex de Taules **xiii**

1 Introducció **1**

1.1 Motivació 1

1.2 Objectius 1

1.3 Estructura del treball 2

1.4 Organització 4

1.5 Sigles i acrònims 4

2 Marc Teòric **7**

2.1 Satèl·lits d'òrbita baixa (LEO) 7

2.1.1 Constel·lacions 9

2.1.2 L'arquitectura de la constel·lació Globalstar 9

2.1.3 Càlcul de trajectòries 11

2.1.4 Antenes per a satèl·lits LEO 13

2.1.5 Les Software Defined Radios (SDR) 15

2.2 Fonaments de mecànica i estructures 16

2.2.1 Fonaments de la mecànica 16
2.2.2 Engranatges 19
2.2.3 Fabricació d'estructures i components 20
2.3 Motors i controladors 21
2.3.1 Motors 21
2.3.2 Controladors 23
2.4 Fonaments d'informàtica 23
2.4.1 Llenguatges de programació 23
2.4.2 Disseny web 28

3 Estat de l'art **37**
3.1 *An Agile, Portable Antenna System for LEO* 37

4 Desenvolupament del programari i interfícies **41**
4.1 Interfície web 41
4.1.1 La tecnologia 41
4.1.2 L'estructura i l'arquitectura 42
4.1.3 Estil 47
4.1.4 Hosting 47
4.2 Identificació de satèl·lits 48
4.2.1 Arquitectura 49
4.2.2 Validació 52
4.3 Seguiment de satèl·lits 53
4.3.1 Arquitectura 53
4.3.2 Validació 54
4.4 Moviment lliure 57
4.4.1 Arquitectura 57
4.4.2 Validació 60

5 Disseny i assemblatge de l'estructura 63

5.1 Requisits i selecció de components electrònics . . . 63

5.1.1 Antena . . . 63

5.1.2 Motors . . . 65

5.1.3 Microcontrolador . . . 70

5.2 Disseny de l'estructura i components mecànics . . . 71

5.2.1 Base del motor d'elevació . . . 73

5.2.2 El motor Azimut i la base rotatòria . . . 75

5.2.3 La base estàtica . . . 78

5.2.4 Procés de disseny . . . 79

5.3 Muntatge de l'estructura . . . 81

5.3.1 Muntatge dels models de la base rotatòria . . . 81

5.3.2 Muntatge dels models de la base estàtica . . . 81

5.3.3 Encaix de les parts: estàtica i rotatòria . . . 83

5.3.4 Col·locant components electrònics . . . 83

5.3.5 Connexions i esquemàtic dels components . . . 84

5.4 Programari de control . . . 85

5.4.1 Arquitectura . . . 86

5.4.2 Validació . . . 87

6 Captura i processament del senyal 91

6.1 Procés de captura del senyal . . . 91

6.1.1 *Set-up* per fer captures . . . 91

6.1.2 Programari de captura . . . 92

6.1.3 Dificultats i entrebancs . . . 94

6.2 Validant el seguiment satel·litari . . . 94

6.3 Validant la captura dels senyals . . . 95

6.3.1 Elevació de 10 a 20 graus . . . 96

6.3.2 Elevació de 30 a 40 graus . . . 97
6.3.3 Elevació de 50 a 60 graus . . . 98
6.3.4 Elevació de 70 a 80 graus . . . 99
6.3.5 Parada de la captura i conseqüències . . . 100

7 Conclusions **101**

7.1 Aspectes de millora per a futures versions del prototip . . . 102
7.2 Agraïments . . . 103

8 Annex **105**

8.1 Codis C del projecte . . . 105
8.1.1 Generació de la llista de SV . . . 105
8.1.2 Gestió de la captura d'un satèl·lit concret . . . 108
8.1.3 Gestió del moviment lliure de l'antena . . . 110
8.1.4 Codis útils per a dades satel·litàries . . . 111
8.1.5 Codis útils per a les comandes lliures . . . 118
8.1.6 Codis útils per a la gestió del *hardware* . . . 124
8.1.7 *Firmware*: codi de l'arduino . . . 127
8.2 Codis de la pàgina web . . . 129
8.2.1 Vistes PHP . . . 129
8.2.2 Api per gestionar peticions . . . 134
8.2.3 Estil . . . 135
8.3 Models 3D pròpis . . . 138

Bibliografia **i**

Índex de figures

1.1 Comparació entre antenes fixes amb una de rotatòria. Es pot veure com les fixes tenen un rang de visió/cobertura limitat (angle α), el qual es pot augmentar a canvi del guany. Mentre que una antena rotatòria permet conservar el guany i augmentar el rang de visió tant com es vulgui. 2

2.1 Comparació visual dels diferents tipus d'òrbites. 8

2.2 Comparació visual entre les geometries d'òrbita de Globalstar (esquerra) i Iridium (dreta). Figures extretes de [Yod24] 9

2.3 Generació d'una senyal CDMA [Vic20] 10

2.4 Arquitectura d'enllaç i bandes de la constel·lació Globalstar 11

2.5 Exemple dels possibles valors d'un fitxer TLE amb descripcions i unitats de cada camp. [Val06] 12

2.6 Esquema del sistema de coordenades AER. Extreta de [Bab18]. 13

2.7 Diagrama de raciació unidireccional d'una antena. [[eve24]] 14

2.8 Representació visual del parell motor (Γ). On: $\Gamma = F \cdot r$ 17

2.9 Representació de les diferents menes d'esforços 18

2.10 Representació visual de la inèrcia d'un cos [Koh22]. 18

2.11 Diferents menes d'engranatges [The03] 19

2.12 Diagrama d'un motor de corrent continu 21

2.13 Diagrama del funcionament d'un motor pas a pas 22

2.14 Diagrama d'un Servomotor 23

2.15 Representació de l'arquitectura client-servidor, amb els llenguatges més comuns de cadascún 28

2.16 Pàgina web resultant dels codis que hem anat veient 32

3.1 Estructura de l'antena rotatòria del paper de referència [Qin24] 38

4.1 Pàgina principal 43

4.2 Pàgina de planificació de captures 44

4.3 Procés de bloqueig de pantalla per tal de realitzar captures sense coincidències . 45

4.4 Pàgina del controlador lliure 46

51

4.6 Comparació visual entre les geometries d'òrbita de Globalstar extretes de l'eina online [Glo24] i el llistat de satèl·lits de més altitud en la següent hora generat pel codi d'aquest projecte. 52

4.7 Diagrama d'activitats del codi del seguiment de satèl·lits 1 55

4.8 Diagrama d'activitats del codi del seguiment de satèl·lits 2 56

4.9 QR al vídeo de l'antena rotant. Enllaç complet: https://www.youtube.com/shorts/BQt4AMmJ8hA 57

4.10 Diagrama d'activitats del codi del controlador lliure 60

5.1 Diagrama de radiació de l'antena Gatling 2.4 MK II 64

5.2 Imatge de l'antena Gatling 2.4 MK II 65

5.3 Imatge d'un motor pas a pas NEMA-23 66

5.4 Esquema del cas més desfavorable. La fletxa negra indica la direcció al zenit (elevació 90º), la d és la distància entre el centre de rotació i el centre de masses de l'antena i p el pes d'aquesta. 66

5.5 Controlador DM542 de *StepperOnline* 69

5.6 Estructura completa del projecte de la base rotatòria d'una càmera amb els motors identificats 72

5.7 Estructura completa del projecte de la base rotatòria d'una càmera. 72

5.8 Representació gràfica de la nova ubicació del motor d'elevació i dels nous models que cal fer 74

5.9 Model 3D de la base del motor 74

5.10 Model 3D de les parets del motor 75

5.11 Representació gràfica de la nova ubicació del motor d'elevació i dels nous models que cal fer 76

5.12 Representació d'un *Bearing* 76

5.13 Model 3D de la base rotatòria 78

5.14 Model 3D de la volandera 78

5.15 Model 3D de la base interior del *bearing* 79

5.16 Model 3D de la pilona 79

5.17 Model 3D de la base interior del *bearing* 80

5.18 Model 3D de la pilona 80

5.19 Esquemàtic dels diferents components electrònics. 85

5.20 Diagrama d'activitats del *firmware* 1 88

5.21 Diagrama d'activitats del *firmware* 2 89

5.22 Prototip de l'estructura completa de l'antena muntada 90

5.23 QR al vídeo de l'antena rotant. Enllaç complet: https://www.youtube.com/shorts/BQt4AMmJ8hA 90

6.1 Imatge del setup del procés de captura del senyal 92

6.2 Espectrograma de referència d'una captura d'un satèl·lit de globalstar. 96

6.3 Captura del senyal per a una elevació de 10 a 20 graus 97

6.4 Captura del senyal per a una elevació de 30 a 40 graus 97

6.5 Captura del senyal per a una elevació de 50 a 60 graus 98

6.6 Captura del senyal per a una elevació d'uns 70-80 graus 99

6.7 Captura del senyal amb l'antena desenfocada 100

8.1 Model 3D de la base del motor 138

8.2 Model 3D de les parets del motor 138

8.3 Model 3D de la base rotatòria 139

8.4 Model 3D de la volandera 139

8.5 Model 3D de la base interior del *bearing* 139

8.6 Model 3D de la pilona 140

8.7 Model 3D de la pilona 140

Índex de taules

2.1 Comparació de les característiques principals de les òrbites terrestres.[Aer23] . . 8

2.2 Tipus de dades fonamentals en llenguatge C. 26

5.1 Especificacions Tècniques de l'Antena Gatling 2.4 MK II 64

5.2 Especificacions tècniques importants del motor NEMA23 triat. 68

5.3 Combinacions de *microstepping* del controlador DM542 69

5.4 Taula de corrents del controlador (Peak = RMS x 1.4) 70

5.5 Configuració del controlador del motor 70

5.6 Especificacions tècniques de l'Arduino Uno Rev3. 71

5.7 Taula resum dels components electrònics 71

5.8 Llistat dels diferents models del projecte de la base d'una càmera rotatòria. Les columnes Çonservada", "Modificadaï "Eliminadaïndiquen el seu rol en el nostre projecte. 73

5.9 Especificacions del *bearing* que farem servir 77

5.10 Llistat complet del material necessari per muntar l'estructura 82

5.11 Configuració completa del controlador del motor 85

5.12 Pins de l'Arduino comparats amb els pins dels controladors i el seu estat inicial . 86

5.13 Llistat de comandes que pot rebre l'Arduino 86

5.14 Taula dels paràmetres diferents pels dos casos. 87

Capítol 1

Introducció

Aquest treball de recerca consisteix a dissenyar i implementar el prototip d'una antena rotatòria que faci el seguiment de satèl·lits d'òrbita baixa (LEO).

1.1 Motivació

Aquest treball de fi de grau (TFG) s'emmarca dins del món de la recerca de tecnologies i d'aplicacions relacionades amb els satèl·lits d'òrbita baixa (LEO), en especial amb el vessant que busca solucions per obtindre posicionament, navegació i "*timing*"(LEO PNT).

El projecte serà d'utilitat sobretot al grup de recerca promotor del TFG, els quals, fins ara, col·locaven les antenes mirant en una direcció fixa, sovint perpendicular a terra (elevació de $90^{\underline{o}}$) per tal de captar senyals de satèl·lits LEO.

Si bé aquesta metodologia no suposa un problema que impedeixi captar el senyal, sí que limita certs factors com el temps de captura o l'àrea de cobertura, sovint fent sacrificar el guany en forma d'antenes menys directives per tal d'augmentar-los.

És d'aquesta necessitat de trobar una forma d'augmentar l'àrea de cobertura sense comprometre ni el guany ni la directivitat que neix el projecte, ja que, una antena rotatòria solucionaria aquest problema (vegeu la figura 1.1).

1.2 Objectius

Es pot dividir aquest treball en tres objectius principals:

1. **Dissenyar i construir una estructura:** de forma que aquesta sigui capaç de mirar allà

on es desitgi i seguir l'òrbita d'un satèl·lit en temps real.

2. **Implementar una interfície visual i un programari que permetin realitzar totes les accions** descrites al punt anterior.

3. **Demostrar que l'aproximació de l'antena rotatòria és millor** que la de les antenes estàtiques.

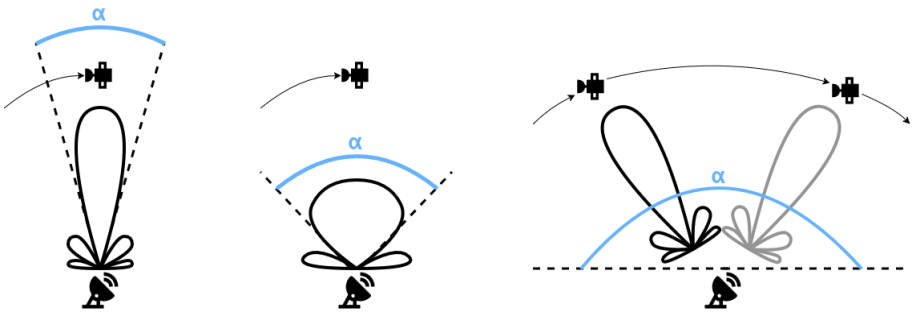

Figura 1.1: Comparació entre antenes fixes amb una de rotatòria. Es pot veure com les fixes tenen un rang de visió/cobertura limitat (angle α), el qual es pot augmentar a canvi del guany. Mentre que una antena rotatòria permet conservar el guany i augmentar el rang de visió tant com es vulgui.

1.3 Estructura del treball

Es pot dividir les tasques de desenvolupament del treball en tres blocs:

1. **Programari i interfícies:** Aquí es duu a terme tot el desenvolupament del *software* encarregat de donar instruccions a l'estructura que mourà l'antena.

2. **Estructura:** En aquest bloc es dedica a tot allò que mantingui una estreta relació amb l'estructura. Es considera la programació del microcontrolador dins d'aquest apartat.

3. **Captura:** Dins d'aquest apartat s'inclou tot el programari i *hardware* que tingui a veure amb la captura del senyal.

Cadascun d'aquests blocs disposarà dels següents apartats.

- Programari i interfícies
  - Moviment lliure
    - Scripts

        * Interfície web
    - Identificació de satèl·lits
        * Anàlisi de fitxers TLE
        * Càlcul i propagació d'òrbites.
        * Interfície web
    - Seguiment de SV
        * Generació de moviments a partir d'òrbites.
        * Interfície web
    - Interfície web (no relacionades amb les anteriors)
        * Hosting
        * Estil
- Estructura
    - Recerca de components HW
    - Disseny de l'estructura
        * Disseny
        * Impressió
        * Condicionament i redisseny
        * Muntatge de les peces
    - Muntatge de l'estructura amb els components HW
    - Programari de control
        * Crear llistat de comandes
        * Controlar i ajustar components HW
- Captures i validació
    - Integrar blocs 1 i 2
    - Programar software de captura del senyal
    - Proves de captura de senyals
    - Anàlisi de les captures.

En aquest document es tracten per separat i en ordre cadascuna d'aquestes parts.

1.4 Organització

Pel que fa a l'organització de la feina, aquesta té una estructura similar a la de la divisió de tasques:

Es comença desenvolupant el bloc 1 (Programari i Interfícies) doncs aquest permet fer avenços de forma més àgil, cosa que ajuda a veure errors de concepte i agafar idees de cara a blocs amb tasques no tan flexibles.

El bloc 2 (Estructura) no s'ha necessàriament executat després de començar el bloc 1, però sí que ha trigat més temps a començar a agafar forma, doncs la recerca prèvia de materials i components és important i poc resultant. A més, si bé és cert que s'ha esperat una mica al principi, de seguida ha fet falta posar-se a treballar en paral·lel amb els dos blocs. De fet, el bloc 2 ha acabat naturalment abans del primer.

Finalment, després d'acabar els dos primers blocs, s'ha dut a terme l'etapa 3 (Captura). Aquesta etapa no s'ha pogut començar abans, doncs existeix un coll d'ampolla ja que: si la plataforma no gira correctament (objectiu dels anteriors blocs) no es poden fer captures significatives, per tant, el bloc 3 exigeix, tant sí com no, haver finalitzat tota la feina anterior.

1.5 Sigles i acrònims

Al llarg d'aquest treball es troben (inevitablement) moltes paraules escrites en sigles o acrònims, és per això que s'anomenen a continuació en un llistat de totes les que apareixeran d'ara en avant.

- **TFG:** Treball de Fi de Grau.
- **LEO:** Low Earth Orbit (Òrbita Terrestre Baixa).
- **SV:** Satellite Vehicle (Vehicle Satel·litari).
- **PNT:** Positioning Navigation and Timing (Posicionament, Navegació i Temps).
- **MEO:** Medium Earth Orbit (Òrbita Terrestre Mitjana).
- **GEO:** Geostationary Orbit (Òrbita Geoestacionària).
- **HEO:** Highly Elliptical Orbit (Òrbita Molt El·líptica).
- **CDMA:** Code Division Multiple Access (Accés Múltiple per Divisió de Codi).
- **3GPP:** 3rd Generation Partnership Project.
- **LTE:** Long Term Evolution.

- **5G:** 5th Generation (Cinquena Generació de telefonia mòbil).
- **TLE:** Two-Line Element (Element de Dues Línies).
- **NORAD:** North American Aerospace Defense Command.
- **UTC:** Coordinated Universal Time (Temps Universal Coordinat).
- **SGP4:** Simplified General Perturbations 4.
- **SDP4:** Simplified Deep Space Perturbations 4.
- **TEME:** True Equator, Mean Equinox.
- **ECEF:** Earth-Centered, Earth-Fixed.
- **AER:** Azimut, Elevació i Rang.
- **ECI:** Earth-Centered Inertial.
- **FSPL:** Free Space Path Loss (Pèrdua de Propagació en Espai Lliure).
- **SDR:** Software Defined Radio (Ràdio Definida per Programari).
- **RF:** Radiofreqüència.
- **ADC:** Analog-to-Digital Converter (Convertidor Analògic-Digital).
- **USB:** Universal Serial Bus.
- **SI:** Sistema Internacional d'unitats.
- **FDM:** Fused Deposition Modeling (Modelatge per Deposició Fosa).
- **PLA:** Àcid Polilàctic (Polylactic Acid).
- **ABS:** Acrilonitril Butadiè Estirè.
- **PETG:** Polietilè Tereftalat Glicol.
- **CNC:** Control Numèric per Ordinador (Computer Numerical Control).
- **DC:** Direct Current (Corrent Continu).
- **CPU:** Central Processing Unit.
- **RAM:** Random Access Memory.
- **GCC:** GNU Compiler Collection.
- **POO:** Programació Orientada a Objectes.

- **PVM:** Python Virtual Machine.
- **HTTP:** HyperText Transfer Protocol.
- **HTTPS:** HyperText Transfer Protocol Secure.
- **HTML:** HyperText Markup Language.
- **CSS:** Cascading Style Sheets.
- **JS:** JavaScript.
- **DOM:** Document Object Model.
- **PHP:** Hypertext Preprocessor.
- **AWS:** Amazon Web Services.
- **IP:** Internet Protocol.
- **ROS:** Robot Operating System.
- **DTC:** Direct To Cell.
- **AOS:** Acquisition of Signal (Inici de visibilitat).
- **LOS:** Loss of Signal (Pèrdua de visibilitat).
- **JSON:** JavaScript Object Notation.
- **LED:** Light Emitting Diode.
- **IEEC:** Institut d'Estudis Espacials de Catalunya.
- **RHCP:** Right Hand Circular Polarization.
- **LHCP:** Left Hand Circular Polarization.
- **SWR:** Standing Wave Ratio (Relació d'Ona Estacionària).
- **PWM:** Pulse Width Modulation.
- **SNR:** Signal-to-Noise Ratio (Relació Senyal-Soroll).
- **IRT:** In Real Time (En temps real).

Capítol 2

Marc Teòric

En aquest apartat es dona una base científica, tècnica i conceptual necessària per entendre el projecte.

2.1 Satèl·lits d'òrbita baixa (LEO)

Quan es parla de satèl·lits, o **Satellite Vehicles (SV)**, es fa referència al conjunt d'objectes artificials posats en òrbita al voltant de la Terra amb finalitats científiques, militars o de telecomunicacions.

Hi ha moltes formes de dividir aquests satèl·lits, ja sigui per la seva massa, per la seva funció o, com es veurà en aquest apartat, per l'altura de les seves òrbites. D'aquesta darrera divisió n'hi ha quatre tipus:

- **LEO (Low Earth Orbit):** Satèl·lits d'òrbita baixa, amb un gran ventall d'utilitats com ara comunicacions de banda ampla, observació de la terra o estudis meteorològics; per posar alguns exemples.
- **MEO (Medium Earth Orbit):** Satèl·lits d'òrbita mitjana, típics de sistemes de navegació com el GPS o Galileo.
- **GEO (Geostationary Orbit):** Satèl·lits geoestacionaris que semblen fixos al cel des de la Terra.
- **HEO (Highly Elliptical Orbit):** Òrbites molt el·líptiques utilitzades per a comunicacions en latituds extremes.

A la figura 2.1 es pot veure una comparació visual de les diferents òrbites. I a la taula 2.1 una comparació genèrica sobre les seves característiques principals.

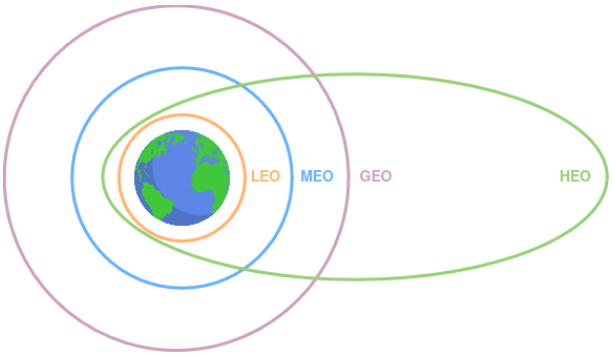

Figura 2.1: Comparació visual dels diferents tipus d'òrbites.

Òrbita	**Altitut**	**Període orbital**	**Cobertura**	**Millor per a...**
LEO	160–2,000 km	90–120 min	Petita	Observació de la terra, PNT
MEO	2,000–20,000 km	2–12 hrs	Regional	Navegació
GEO	35,786 km	24 hrs	Continua	TV, Meteorologia
HEO	El·líptica	Varies	Polar	Comunicacions Polars

Taula 2.1: Comparació de les característiques principals de les òrbites terrestres.[Aer23]

Cadascuna d'aquestes òrbites presenta unes propietats diferents que poden ser útils o no per una gran diversitat d'aplicacions. Aquest projecte, però, tracta amb els satèl·lits d'òrbita baixa (i.e. LEO).

Els satèl·lits LEO operen en una regió de l'espai situada habitualment entre els 500 i els 2.000 quilòmetres d'altitud. A causa de la seva proximitat a la Terra, aquests satèl·lits no es mantenen fixos en el cel des de la perspectiva d'un observador terrestre, sinó que es desplacen a velocitats orbitals extremadament altes, aproximadament 7,5 km/s, completant una òrbita cada 90-120 minuts.

El fet que les òrbites siguin tan baixes ofereix uns avantatges clau, com ara: una menor latència, una menor potència necessària per comunicar-se amb ells i un menor cost de posada en òrbita. Aquests mateixos avantatges han permès dissenyar SV més petits i barats. [Any23]

Tot i això, aquesta dinàmica imposa reptes significatius per a les comunicacions, ja que el temps de visibilitat d'un satèl·lit individual durant un "pas"per sobre d'una estació terrestre és d'aproximadament 10 a 20 minuts. Per garantir un servei continuat, és imprescindible organitzar aquests satèl·lits en xarxes coordinades anomenades constel·lacions.

2.1.1 Constel·lacions

Una constel·lació és una xarxa coordinada de satèl·lits del mateix tipus que operen sota un control compartit per tal de proporcionar cobertura contínua a escala global o regional.[Wik24]

L'organització d'aquestes xarxes es defineix habitualment mitjançant la **notació de Walker** [Wal70], la qual descriu paramètricament el nombre total de satèl·lits, la seva distribució en diferents plans orbitals i la inclinació respecte a l'equador terrestre.

Aquesta geometria està calculada per maximitzar factors com la redundància de senyal, la sincronització i, sobretot, la probabilitat que un usuari a terra tingui sempre almenys un satèl·lit per sobre de la seva màscara d'elevació. A la figura 4.6 es comparen dos dels dissenys més emblemàtics:

- **La constel·lació Iridium:** Proporciona cobertura total al planeta (incloent-hi els pols) fent servir una configuració "*Walker Star*".
- **La constel·lació Globalstar:** Fa ús d'una configuració "*Walker Delta*"amb plans inclinats a 52°, d'aquesta forma es pretén maximitzar la cobertura en zones poblades.

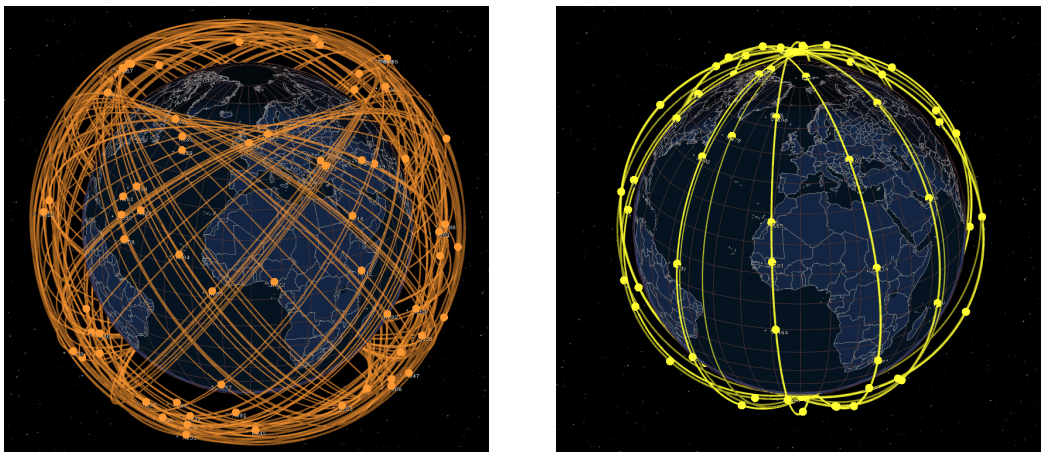

Figura 2.2: Comparació visual entre les geometries d'òrbita de Globalstar (esquerra) i Iridium (dreta). Figures extretes de [Yod24]

2.1.2 L'arquitectura de la constel·lació Globalstar

Globalstar és una empresa nord-americana nascuda a principis dels 2000 i que ofereix serveis com la telefonia mòbil per satèl·lit i la transmissió de dades de baixa velocitat a escala global [Glo24].

El projecte de Globalstar es basa en l'ús de satèl·lits de "mirall"o *bent-pipe*, el que significa

que el satèl·lit actua com un repetidor simple: rep el senyal, el canvia de freqüència i el torna a emetre cap a una estació terrestre o *gateway* sense processar les dades a bord.

El que a nosaltres ens interessa de Globalstar és la seva constel·lació de satèl·lits. Aquests estan distribuïts de forma que garanteixen que almenys un satèl·lit (i sovint més de dos) estigui sempre per sobre de l'horitzó per a qualsevol usuari situat entre les latituds 70ºS i 70ºN.

La configuració orbital de Globalstar ha evolucionat per adaptar-se a les demandes del mercat i a les capacitats de llançament. El sistema original va ser dissenyat com una constel·lació Walker 48/8/1, el que significa que estava format per 48 satèl·lits operatius distribuïts en 8 plans orbitals amb una inclinació de 52 graus. Actualment, el sistema s'està movent cap a una configuració Walker-32.

Pel que fa als senyals dels SV, Globalstar utilitza la tecnologia d'Accés Múltiple per Divisió de Codi (CDMA) basada en l'estàndard IS-95 de Qualcomm (veure figura 2.3). Aquesta elecció tecnològica permet la "diversitat de camins", on un terminal d'usuari pot establir connexions simultànies amb fins a tres satèl·lits diferents.

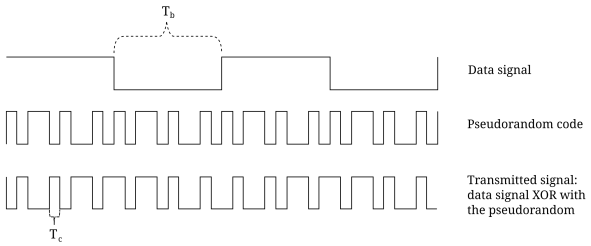

Figura 2.3: Generació d'una senyal CDMA [Vic20]

El sistema opera en diverses bandes de freqüència per gestionar els diferents enllaços de comunicació (veure imatge 2.4):

- **Enllaç d'usuari (Uplink):** Els terminals transmeten cap als satèl·lits en la banda L, concretament en el rang de 1610 a 1618.7 MHz.
- **Enllaç d'usuari (Downlink):** Els satèl·lits transmeten cap als usuaris en la banda S, en el rang de 2483.5 a 2500 MHz.
- **Enllaç de connexió (*Feeder Link*):** La comunicació entre el satèl·lit i la *gateway* utilitza la banda C (5 GHz per a l'uplink i 7 GHz per al downlink).

Aquesta asimetria en l'espectre ha permès a Globalstar llicenciar part de la seva banda S per a ús terrestre, coneguda com a Banda 53 o n53 en les especificacions 3GPP, ideal per a xarxes privades d'LTE i 5G a causa de la seva llicència global i absència d'interferències.

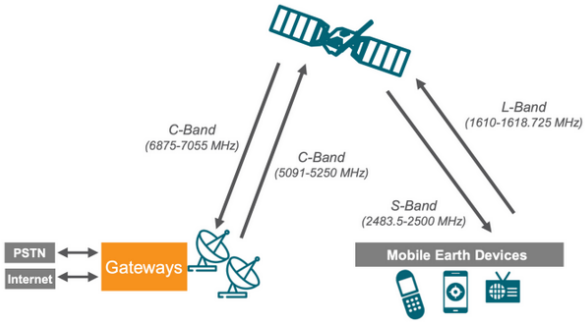

Figura 2.4: Arquitectura d'enllaç i bandes de la constel·lació Globalstar

2.1.3 Càlcul de trajectòries

S'ha comentat que des d'un principi se sap quina serà l'òrbita prevista que segueixi un satèl·lit al llarg de la seva vida útil, i això fa pensar que el seu càlcul és relativament senzill.

Malauradament, a causa de molts factors, com la rotació de la terra o l'arrossegament atmosfèric, aquest càlcul implica la resolució de models matemàtics complexos de mecànica orbital i transformacions de sistemes de coordenades.[Val06]

A part d'aquests models, és important que hi hagi algun sistema de monitoratge de satèl·lits, de forma que les dades orbitals s'actualitzin constantment per corregir les desviacions acumulades entre l'òrbita teòrica i la real.

És aquí on entra la comunitat de vigilància espacial de l'exèrcit dels Estats Units (a través de l'U.S. Space Command), que s'encarrega de publicar de forma periòdica uns fitxers anomenats TLE (*Two Line Elements*).

Els **fitxers TLE** (*Two Line Elements*) contenten la informació bàsica per al seguiment de satèl·lits. Es distribueix en el format de "Two-Line Element Set"(TLE), un estàndard desenvolupat pel NORAD i que conté les següents dades [Cha16]:

1. **Número de catàleg (NORAD ID):** Identificador únic de 5 dígits assignat per l'U.S. Space Command a cada objecte llançat a l'espai.

2. **Classificació:** Indica si l'objecte és de caràcter públic (U) o classificat.

3. **Designador internacional:** Indica l'any de llançament, el número de llançament d'aquell any i la peça del llançament.

4. **Època (Epoch):** El moment precís (any, dia i fracció de dia) en què es van prendre les mesures. Com més lluny estiguem de l'època, menys precís serà el càlcul del propagador a causa de les pertorbacions no modelades.

5. **Derivada primera del moviment mitjà (n˙/2):** Indica com canvia el moviment mitjà a causa de l'arrossegament atmosfèric.

6. **Coeficient d'arrossegament (B):** També conegut com a *BSTAR drag term*, és un paràmetre que modela com la resistència atmosfèrica afecta la velocitat del satèl·lit.

7. **Inclinació (i):** L'angle entre el pla de l'òrbita i el pla de l'equador terrestre.

8. **Ascensió Recta del Node Ascendent (RAAN, Ω):** L'angle des del punt Aries fins al punt on el satèl·lit creua l'equador de sud a nord.

9. **Excentricitat (e):** Descriu la forma de l'òrbita. Per a Globalstar és gairebé 0, indicant una òrbita gairebé circular.

10. **Argument del Perigeu (ω):** Defineix l'orientació de l'òrbita dins del seu pla respecte al node ascendent.

11. **Anomalia Mitjana (M):** Indica la posició del satèl·lit en l'òrbita en el moment de l'època.

12. **Moviment Mitjà (n):** Nombre de revolucions que el satèl·lit completa al voltant de la Terra per dia.

13. **Número de revolució:** El número total d'òrbites completades des del seu llançament.

Card #	Satellite Number	Class	International Designator (Year Lch# Piece)	Yr	Epoch Day of Year (plus fraction)	Mean motion derivative (rev/day /2)	Mean motion second derivative (rev/day2 /6)	Bstar (/ER)	Eph	Elem num	Chk Sum
1	16609	U	86017A	93	352.53502934	.00000789	00000-0	10529-3	0	342	
2	16609		51.6190	13.3404	0005770	102.5680	257.5950	15.59114070	44786	9	

Inclination (deg)	Right Ascension of the Node (deg)	Eccentricity	Arg of Perigee (deg)	Mean Anomaly (deg)	Mean Motion (rev/day)	Epoch Rev	Chk

Figura 2.5: Exemple dels possibles valors d'un fitxer TLE amb descripcions i unitats de cada camp. [Val06]

Aquests fitxers proporcionen tota la informació necessària perquè es pugui calcular l'òrbita del satèl·lit. Per a això es fa servir el model estàndard de propagació per a satèl·lits LEO: el **SGP4 (*Simplified General Perturbations 4*)** [Val06][Mon00]. Aquest algorisme té en compte les pertorbacions més significatives que afecten els satèl·lits en òrbita baixa, com ara:

- **Pertorbacions gravitatòries terrestres**, degut al fet que la terra no és una esfera perfecte, sinó un el·lipsoide aplanat als pols.

- **Arrossegament atmosfèric:** La resistència causada per la tènue atmosfera en altituds LEO, que provoca que el satèl·lit perdi energia i l'òrbita decaigui lentament.
- **Influència de la Lluna i el Sol:** Pertorbacions gravitatòries de cossos tercers, que són especialment rellevants en el model SDP4 per a satèl·lits de més altitud.

L'algorisme SGP4 rep un TLE i un instant de temps (normalment en format UTC) i calcula un vector d'estat (posició i velocitat) en el sistema de coordenades inercials conegut com a TEME (*True Equator, Mean Equinox*).

Aquest sistema de coordenades (TEME) és la convenció estàndard per a la propagació d'òrbites, ja que és un sistema inercial (no gira amb la Terra), cosa que simplifica les equacions del moviment dels cossos celestes. No obstant això, per a una estació de seguiment terrestre, aquestes coordenades no són directament utilitzables.

En el context d'aquest projecte, és necessari transformar les coordenades en dos passos: primer del sistema inercial al sistema ECEF (*Earth-Centered, Earth-Fixed*), que sí que gira amb el planeta, i finalment al sistema Topocèntric Local (AER: Azimut, Elevació i Rang), basat en coordenades esfèriques.

El sistema de coordenades esfèriques topocèntriques permet definir la posició del satèl·lit des de l'òptica de l'antena mitjançant tres paràmetres:

- **Azimut (α):** L'angle horitzontal mesurat des del Nord geogràfic (0 a 360).
- **Elevació (ϵ):** L'angle vertical des de l'horitzó de l'observador (0 a 90).
- **Distància o Rang (r):** La distància en línia recta entre l'antena i el satèl·lit.

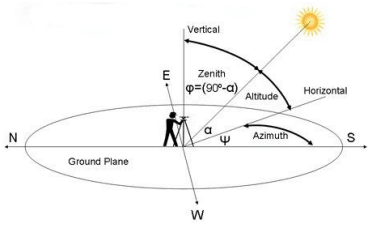

Figura 2.6: Esquema del sistema de coordenades AER. Extreta de [Bab18].

2.1.4 Antenes per a satèl·lits LEO

Una antena es defineix com el dispositiu dissenyat per radiar o rebre ones electromagnètiques. En el context de les comunicacions per satèl·lit, una antena es caracteritza principalment per les següents propietats:

- **Guany (G):** Mesura la capacitat de l'antena per dirigir/rebre l'energia en una direcció específica en comparació amb una antena isotròpica. Es mesura en dBi.

- **Ample de feix (*Beamwidth,* Θ):** L'angle d'obertura on l'antena radia/rep la major part de la seva potència (normalment mesurat a -3 dB del pic màxim).

- **Directivitat (D):** Indica la concentració de la radiació en una direcció determinada. A més directivitat, el feix de l'antena és més estret.

A la figura 2.7 es pot veure el diagrama de radiació d'una antena. En aquest queda clarament diferenciats diferents els diferents lòbuls. On un en destaca per sobre dels altres, aquest és el lòbul principal i el que rep i emet més potència. L'amplada d'aquest lòbul es defineix com l'amplada de feix (*Beamwidth*, Θ) i queda delimitada entre el màxim de potència i $-3dB$. El senyal emès/rebut més enllà de l'anlge Θ es considera suficientment atenuat.

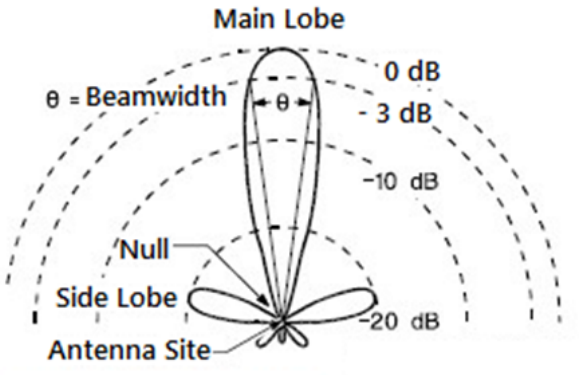

Figura 2.7: Diagrama de raciació unidireccional d'una antena. [[eve24]]

Aquestes característiques depenen unes de les altres i sovint cal arribar a compromisos (*trade-offs*) que milloren una qualitat, però n'empitjoren una d'altra perquè per la nostra aplicació és més pràctic.

Un bon exemple d'aquests compromisos és el seguiment de satèl·lits LEO, on el disseny d'una antena implica un equilibri entre el guany, l'ample de feix i la capacitat de seguiment, ja que, a trets generals, si s'utilitza una antena amb molt de guany, el feix serà tan estret que qualsevol petit error en el sistema rotatori provocarà la pèrdua del senyal. Per contra, les antenes per a satèl·lits GEO poden ser fixes i molt directives (amb un feix extremadament estret) perquè el satèl·lit és estacionari respecte a terra, les antenes per a LEO han de gestionar el moviment ràpid de l'objectiu.

Pel que fa al mínim de senyal que necessitem per processar senyals de SV LEO, cal assenyalar que la pèrdua de propagació en espai lliure (*Free Space Path Loss*, FSPL) per a un satèl·lit LEO

a 1.414 km d'altitud se situa al voltant dels 140-160 dB depenent de la freqüència i de l'angle d'elevació [Val06].

Algunes tipologies d'antenes LEO són:

- **Antena Yagi-Uda:** És una antena direccional formada per un element alimentat, un reflector i diversos directors. És molt popular en el seguiment de satèl·lits d'aficionats (com els satèl·lits OSCAR) a causa del seu baix cost, facilitat de construcció i pes lleuger, que permet l'ús de rotadors petits. Poden assolir guanys de 10 a 15 dBi.

- **Antena Parabòlica:** Utilitza un reflector en forma de paràbola per concentrar el senyal en un punt focal. Ofereix el guany més elevat (més de 20-30 dBi), necessari per a serveis de dades d'alta velocitat de Globalstar. El seu principal inconvenient és el seu gran pes i inèrcia, a més d'un feix molt estret que dificulta el seguiment si el sistema de motors no és molt precís.

- **Antena de Matriu de Fases (*Phased Array*):** Aquestes antenes realitzen el seguiment de manera electrònica, sense moviments mecànics, variant la fase del senyal en cadascun dels seus elements radiants. Són les més cares i complexes de dissenyar, però ofereixen la velocitat de seguiment més alta i la capacitat de seguir diversos satèl·lits simultàniament.

2.1.5 Les Software Defined Radios (SDR)

Una Radio Definida per Programari (*Software Defined Radio*, SDR) és un sistema de comunicacions on els components que tradicionalment s'implementaven mitjançant maquinari (com ara mescladors, filtres, amplificadors, moduladors i demuladors) s'implementen mitjançant programari en un ordinador personal o en un sistema encastat.

Aquest canvi de paradigma permet que un mateix dispositiu de radiofreqüència sigui capaç de sintonitzar, rebre i processar una enorme varietat de protocols i freqüències simplement canviant l'algorisme de processament, sense necessitat de modificar els circuits físics.

L'arquitectura bàsica d'una SDR consisteix en una etapa de radiofreqüència (*RF Front-end*) i un convertidor analògic-digital (ADC). L'etapa de RF rep el senyal de l'antena, l'amplifica i el trasllada a una freqüència intermèdia o directament a banda base. Immediatament després, l'ADC digitalitza el senyal, transformant l'ona electromagnètica en una seqüència de dades numèriques (mostres).

Aquestes mostres s'envien a l'ordinador a través de busos de dades com l'USB o Ethernet, on el programari s'encarrega de realitzar la desmodulació i l'extracció de la informació.

Les característiques que defineixen la qualitat i la capacitat d'una SDR són:

- **Rang de freqüències:** El ventall de l'espectre radioelèctric que el dispositiu pot sintonitzar. Per al seguiment de Globalstar, l'SDR ha de ser capaç de cobrir la banda S (2.4 GHz).
- **Ample de banda instantani (*Sample Rate*):** La quantitat d'espectre que el dispositiu pot "veureï digitalitzar simultàniament. Es mesura en mostres per segon (Sps).
- **Resolució (Bits):** Determina el rang dinàmic del dispositiu; a més bits, millor capacitat per distingir senyals febles de satèl·lits en presència de soroll o interferències.

En aquest projecte, l'SDR actua com el vincle final entre l'espai i l'usuari. Un cop l'antena rotatòria ha seguit el satèl·lit LEO i ha captat el senyal amb el guany suficient, l'SDR digitalitza aquesta energia i la lliura al programari de processament, tancant així el flux de dades que comença en l'òrbita del satèl·lit i acaba en la interfície web de monitoratge.

2.2 Fonaments de mecànica i estructures

Un cop definits els requisits de seguiment orbital i les característiques de l'antena cal començar a pensar en el món de la mecànica i les estructures. Aquí tractarem tota la mecànica i processos de fabricació necessaris per al nostre projecte.

2.2.1 Fonaments de la mecànica

La mecànica és la part de la física que estudia el moviment dels cossos físics i les causes d'aquests moviments, tals com les forces o les energies.

La mecànica pot ser dividida en: la mecànica clàssica, la mecànica relativista, la mecànica quàntica i la mecànica quàntica relativista. En aquest treball només es tracta la mecànica clàssica Newtoniana [Viq24]. Aquest tipus de mecànica es pot dividir en:

- **Estàtica:** Anàlisi de sistemes en equilibri.
- **Dinàmica:** Anàlisi de sistemes en moviment.
- **Cinemàtica:** Estudi del moviment sense tenir en compte les forces que el produeixen.

Abans de tractar més a fons cadascun d'aquests mons cal definir alguns conceptes fonamentals [1]:

[1] Les fórmules d'aquests conceptes es poden derivar fàcilment de les seves unitats.

- **Força (F):** Acció que provoca una pertorbació en la quantitat de moviment d'un cos. Es mesura en Newtons (N) o el seu equivalent en el SI: $kg \cdot m/s$.

- **Quantitat de moviment o moment lineal (p):** Magnitud vectorial que s'obté com el producte de la massa d'un objecte per la seva velocitat. Es mesura en: $kg \cdot m/s$ o en: Ns.

- **Parell motor o Moment d'una força (Γ o M):** Moment de torsió que s'aplica sobre un eix per provocar-ne la rotació o vèncer una resistència al gir. Es mesura en: Nm. Vegeu la imatge 2.8.

- **Tensió (σ):** Intensitat de les forces internes que es distribueixen dins d'un material com a resposta a càrregues externes. Es mesura en: N/m^2; o, el que és més habitual, en Pascals (Pa).

- **Acceleració angular (α):** Variació de la velocitat angular per unitat de temps. Es mesura en rad/s^2.

- **Deformació (ϵ):** Canvi relatiu en la geometria d'un cos a causa de l'aplicació de tensions. És una magnitud adimensional.

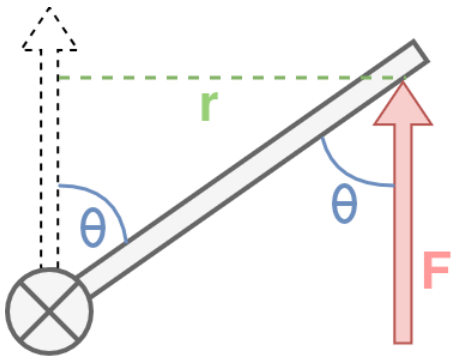

Figura 2.8: Representació visual del parell motor (Γ). On: $\Gamma = F \cdot r$

2.2.1.1 Estàtica

L'estàtica estudia el sistema de forces que actuen sobre un cos en repòs o en equilibri. Aquest equilibri es produeix quan la suma de forces i la suma de moments són nul·les en qualsevol punt del sistema, complint les equacions fonamentals:

$$\sum \vec{F} = 0 \text{ N} \qquad \sum \vec{\Gamma} = 0 \text{ Nm}$$

En el disseny estructural, l'estàtica permet determinar les reaccions en els suports i els esforços interns, com ara: tensions, compressions, flexions, cisallament i torsions (a la figura 2.9 els podeu veure representats).

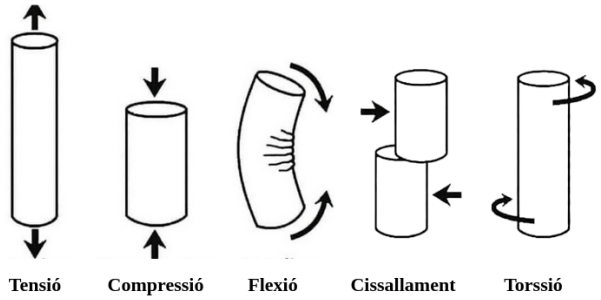

Figura 2.9: Representació de les diferents menes d'esforços

A més, en el disseny d'una estructura caldrà anar amb compte amb l'estabilitat d'aquesta, doncs si les forces externes (com el vent o el mateix pes de la càrrega en posicions extremes) generen un moment respecte a la base superior al que l'estructura pot contrarestar, es podria produir una bolcada. L'estabilitat depèn directament de la geometria de la base i de la posició del centre de gravetat del conjunt.

2.2.1.2 Dinàmica

La dinàmica analitza el comportament d'una estructura quan està sotmesa a acceleracions i forces variables en el temps, regida per la Segona Llei de Newton ($F = m \cdot a$). En sistemes rotatoris, l'equació equivalent relaciona el parell (Γ) amb l'acceleració angular (α):

$$\Gamma = I \cdot \alpha$$

On I és el moment d'inèrcia, una propietat geomètrica que indica la resistència d'un cos a canviar el seu estat de rotació (vegeu la figura 2.10).

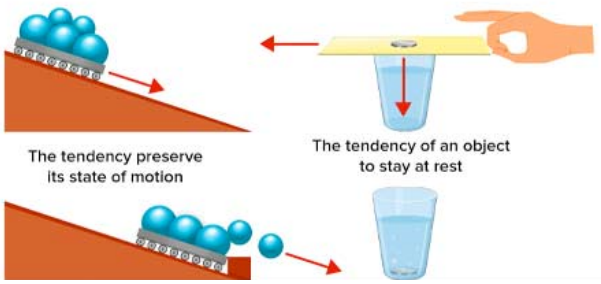

Figura 2.10: Representació visual de la inèrcia d'un cos [Koh22].

A més, cal considerar les vibracions estructurals. Tota estructura posseeix freqüències naturals; si les excitacions externes (com el moviment dels motors o el vent) coincideixen amb aquestes, es produeix el fenomen de la ressonància, que pot derivar en inestabilitats o fallades catastròfiques[2].

2.2.2 Engranatges

Els engranatges són mecanismes utilitzats per a transmetre potència mecànica variant la velocitat de rotació i el parell.

Perquè dos engranatges puguin acoblar-se, han de compartir el mateix mòdul (m), que és la relació entre el diàmetre primitiu i el nombre de dents. Aquesta unió d'engranatges es defineix per la relació de transmissió (i):

$$i = \frac{\omega_{entrada}}{\omega_{sortida}} = \frac{Z_{sortida}}{Z_{entrada}}$$

on Z és el nombre de dents de cada roda i ω

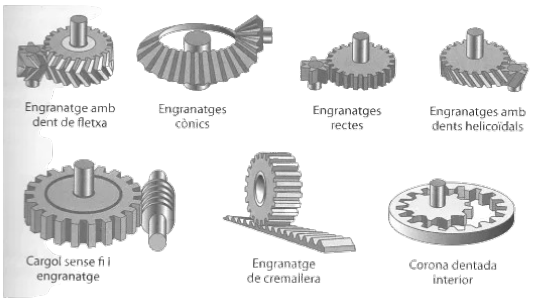

Figura 2.11: Diferents menes d'engranatges [The03]

Aquesta relació de transmissió ens permet:

- **Modificar el parell i la velocitat:** Es pot reduir la velocitat augmentant el parell disponible, ideal per a moviments que requereixen molta força i precisió. També es pot fer a la inversa.
- **Canviar de direcció:** Mitjançant engranatges cònics o de vis sens fi, cosa que permet que el moviment es transmeti entre eixos no paral·lels.
- **Sincronitzar:** Garantint que el moviment d'un component estigui perfectament lligat al d'un altre sense lliscaments.

[2]Si algú ha passat mai pel pont de la plaça cívica segurament haurà experimentat en algun moment aquest fenomen

Hi ha moltes formes d'acoblar dos engranatges o més, a la figura 2.11 en podem veure algunes.

2.2.3 Fabricació d'estructures i components

Un cop definits els fonaments mecànics i els mecanismes de transmissió, cal abordar com es materialitzaran aquests dissenys.

Hi ha diverses formes de fer estructures per prototips petits, però les principals són:

Fabricació Additiva La fabricació additiva, coneguda popularment com a impressió 3D, és un procés de producció que crea objectes tridimensionals a partir d'un model digital i mitjançant la superposició successiva de capes de material.

Dins d'aquest grup, la tecnologia més estesa és la FDM (*Fused Deposition Modeling*), on un filament termoplàstic és fos i dipositat per un capçal mòbil.

Parlant dels materials, els més utilitzats són els polímers termoplàstics com el PLA (Àcid Polilàctic), l'ABS (Acrilonitril Butadiè Estirè) i el PETG. També existeixen filaments tècnics carregats amb fibra de carboni o metalls per a aplicacions que requereixin més resistència.

Alguns dels avantatges d'aquest mètode és que permet una llibertat geomètrica gairebé total, facilitant la creació de peces buides o amb estructures internes complexes. A més de ser un mètode molt econòmic.

L'únic inconvenient que presenta la Fabricació Additiva és que les peces no tenen el millor acabat, patint anisotropia (propietats mecàniques diferents segons l'orientació de les capes) i amb acabats superficials força rugosos.

Fabricació Substractiva A diferència de l'additiva, la fabricació substractiva parteix d'un bloc sòlid de material (anomenat "tova") i va eliminant les parts sobrants mitjançant eines de tall fins a obtenir la geometria desitjada. Aquest procés es realitza habitualment mitjançant màquines de Control Numèric per Ordinador (CNC), com fresadores o torns.

Aquest mètode és compatible amb una gamma molt àmplia de materials, destacant els metalls (alumini, acer, llautó) i plàstics tècnics densos (com el Delrin o el niló mecanitzat).

El principal avantatge respecte als altres mètodes de fabricació és que ofereix una precisió dimensional excel·lent i toleràncies molt estretes, essencials per a acoblaments mecànics d'alta qualitat. L'acabat superficial és superior i les peces són isotròpiques, mantenint la seva resistència en totes les direccions.

Els inconvenients més grans d'aquest mètode són:

- Es genera una gran quantitat de residus en forma de ferritja.
- La complexitat de les peces està limitada per l'accés de l'eina de tall
- El cost és significativament més elevat per a prototips únics a causa del temps de preparació de la màquina i el cost de les eines.

2.3 Motors i controladors

Per tal de moure mecànicament l'antena, no ens deslliurem de fer servir motors (elèctrics), així que en aquest apartat donarem una base teòrica sobre aquests que ens permetrà ser capaços de seleccionar-ne el millor mes endavant.

2.3.1 Motors

Els motors elèctrics són màquines elèctriques rotatòries que transformen l'energia elèctrica en energia mecànica. El seu funcionament es basa en la llei de Lorentz, on un conductor pel qual circula un corrent elèctric, immers en un camp magnètic, experimenta una força mecànica ($\vec{E} = q \cdot (\vec{E} + \vec{v} \times \vec{B})$) [**?**].

Hi ha moltes menes de motors elèctrics diferents, aquí tenim les més habituals:

2.3.1.1 Motors de corrent continu (*DC motors*)

Són els més senzills en quant a construcció. Consten d'un estator (part fixa amb imants) i un rotor (part mòbil amb bobinats).

A la figura 2.12 s'observa clarament el seu funcionament: Quan el corrent elèctric circula per l'espira de l'induït (armadura) situada dins del camp magnètic creat pels imants permanents, es genera un parell de forces oposades que produeixen el gir. El component clau és el **commutador** i les **escombretes** (*brushes*), que inverteixen la polaritat del corrent a mesura que el rotor gira per mantenir el moviment en una direcció constant.

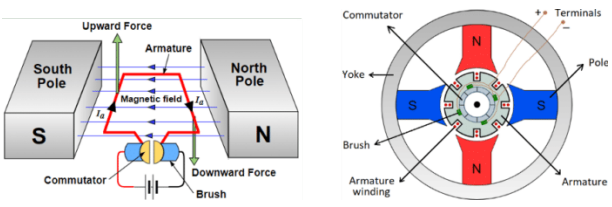

Figura 2.12: Diagrama d'un motor de corrent continu

Els principals avantatges d'aquest tipus de motors és que són força facils de fer servir i controlar la velocitat, no obstant això, també pequen d'un dels majors inconvenients, ja que no permeten un control de posició precís per si sols i tenen un desgast mecànic elevat a causa de les escombretes.

2.3.1.2 Motors pas a pas (*Stepper motors*)

El motor pas a pas és un dispositiu electromecànic que converteix impulsos elèctrics en moviments angulars discrets. A diferència dels motors convencionals, la rotació es divideix en un nombre fix de passos iguals.

Aquests motors també consten d'un estator i un rotor, només que en aquest cas l'estator està format per diversos bobinats independents mentre que és el rotor qui fa la funció d'imant permanent o un nucli de ferro dentat (veure imatge 2.13). El seu funcionament és senzill però enginyós...

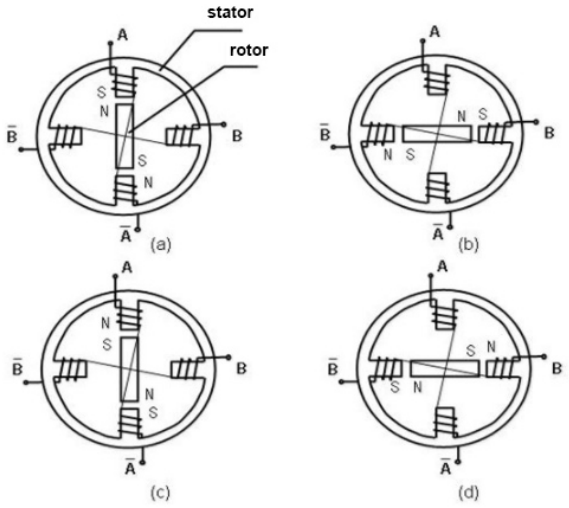

Figura 2.13: Diagrama del funcionament d'un motor pas a pas

Aquesta forma de fer les coses ens permet un control de posició extremadament precís en bucle obert, ja que és possible determinar la ubicació exacta del rotor simplement comptant els polsos enviats al controlador.

Tanmateix, aquesta tecnologia requereix una gestió acurada del corrent, ja que consumeixen energia de forma constant. Un dels seus riscos principals és la pèrdua de sincronisme o "pèrdua de passos"si el sistema es veu sotmès a càrregues superiors a la seva capacitat nominal o a acceleracions excessives, però no hauria de ser un problema si es fa una gestió correcte.

2.3.1.3 Servo-motors

Un servomotor s'ha d'entendre com un sistema de control de moviment integrat més que com un motor aïllat. Aquest conjunt aglutina un motor de corrent continu, un tren d'engranatges reductor i un circuit de control amb retroalimentació interna (vegeu la figura 2.14).

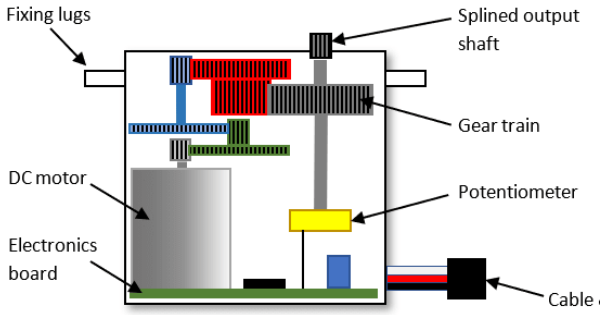

Figura 2.14: Diagrama d'un Servomotor

El seu element diferencial és el sensor de posició (sovint un potenciòmetre unit a l'eix de sortida) que informa constantment al controlador sobre l'angle real del motor. El circuit intern compara aquesta informació amb el senyal de referència extern i corregeix qualsevol desviació de manera automàtica.

2.3.2 Controladors

2.4 Fonaments d'informàtica

Aquest projecte tot i ser de telecomunicacions, fa un ús extensiu d'eines informàtiques, ja sigui per gestionar dades en temps real, fer càlculs complexos de mecànica orbital i oferir una interfície d'usuari intuïtiva. En aquesta secció es descriuen els pilars informàtics sobre els quals s'ha construït el programari de control i la interfície del prototip.

2.4.1 Llenguatges de programació

Un llenguatge de programació és un sistema estructurat de comunicació, format per un conjunt de símbols i regles sintàctiques i semàntiques, que permet a un programador definir de manera precisa les instruccions que una unitat de processament ha d'executar.

A diferència dels llenguatges naturals, els llenguatges de programació estan dissenyats per ser unívocs, eliminant qualsevol ambigüitat en la interpretació de les ordres.

En l'àmbit de l'enginyeria, aquests llenguatges actuen com el pont d'abstracció necessari

per traduir algorismes matemàtics i lògics en senyals elèctrics i canvis d'estat dins d'un sistema digital.

2.4.1.1 Nivells d'abstracció

L'evolució de la informàtica ha permès classificar els llenguatges segons la seva proximitat al maquinari físic o a la capacitat cognitiva humana:

- **Llenguatges de baix nivell:** Són aquells que estan totalment vinculats a l'arquitectura del processador. El màxim exponent és el *Llenguatge Màquina* (codi binari) i l' *Assemblador*. Proporcionen un control absolut sobre el maquinari i la memòria, però la seva complexitat és elevada i no són portables entre diferents arquitectures.

- **Llenguatges de mig nivell:** Sovint s'utilitza aquest terme per referir-se a llenguatges com C. Permeten efectuar operacions d'alt nivell (estructures de control, funcions) però mantenen la capacitat de gestionar directament punters de memòria i registres del sistema, essent l'estàndard per a sistemes encastats.

- **Llenguatges d'alt nivell:** Són llenguatges que prioritzen la facilitat d'ús i la llegibilitat. Utilitzen construccions properes al llenguatge humà (anglès) i abstrauen a l'usuari de la gestió interna de la memòria. Exemples clau són Python, Java o C#.

2.4.1.2 Processament del codi: Compilació i Interpretació

Perquè un processador pugui executar un programa escrit en un llenguatge d'alt o mig nivell, aquest s'ha de transformar en codi màquina. Existeixen dos mètodes principals per realitzar aquesta tasca:

1. **Compilació:** Un programa anomenat compilador llegeix tot el codi font i el tradueix en un fitxer executable binari abans de l'execució. Això garanteix una velocitat màxima d'execució, ja que el processador rep les ordres directament. Llenguatges com C i C++ segueixen aquest model.

2. **Interpretació:** El codi font no es tradueix prèviament, sinó que un programa intermediari (l'intèrpret) llegeix i executa les instruccions línia per línia en temps d'execució. Això facilita la depuració i la portabilitat, però a costa d'un major consum de recursos i menor velocitat. Python és el representant més significatiu d'aquest grup.

2.4.1.3 Paradigmes de programació

Un paradigma representa la filosofia o l'estil de programació utilitzat per resoldre problemes:

- **Programació procedimental:** Es basa en crides successives a procediments o funcions. L'estat del programa es modifica mitjançant una seqüència d'ordres (comú en C).
- **Programació Orientada a Objectes (POO):** Organitza el programari al voltant de "objectes"en lloc de funcions i lògica. Un objecte pot ser una representació d'un component real (per exemple, un motor o un satèl·lit) que té dades (atributs) i comportaments (mètodes). C++ i Python permeten aquest paradigma.

L'ús de diferents llenguatges i paradigmes dins d'un mateix projecte d'enginyeria permet optimitzar tant el control estricte del temps real en el maquinari com la gestió complexa de dades en la interfície d'usuari.

2.4.1.4 C lang

El llenguatge C és un llenguatge de programació compilat de propòsit general, àmpliament utilitzat en el desenvolupament de sistemes encastats (*embedded systems*), sistemes operatius i aplicacions de baix nivell on el control dels recursos del maquinari és crític.

Fou creat a principis dels anys 70, la seva naturalesa minimalista i la seva proximitat a l'arquitectura del processador el converteixen en l'eina estàndard per a la programació de microcontroladors, ja que permet una gestió directa de la memòria i els registres sense el consum de recursos que requereixen els llenguatges d'alt nivell.

Els punts forts d'aquest llenguatge respecte dels altres resideix principalment en la seva eficiència, portabilitat i velocitat. A més, en ser un llenguatge de mig nivell, ofereix la possibilitat de manipular adreces de memòria mitjançant punters, la qual cosa permet optimitzar l'ús de la memòria RAM en dispositius amb capacitats limitades.

Quan nosaltres volem desenvolupar un programa en C, el primer que hem de fer és escriure el codi font organitzat en fitxers amb extensions específiques. Habitualment, s'utilitzen fitxers de capçalera (*header files*) amb extensió `.h` per declarar prototips de funcions, estructures de dades i constants, mentre que els fitxers de codi font amb extensió `.c` contenen la implementació real de la lògica.

El programa comença sempre la seva execució a la funció denominada `main`. Aquesta funció pot rebre paràmetres des de l'entorn d'execució mitjançant els arguments `argc`, que indica el nombre d'arguments passats, i `argv`, que és un vector de cadenes de caràcters que conté els valors d'aquests arguments.

Dins d'aquest codi, s'utilitzen comandes del preprocessador per configurar l'entorn abans de la compilació. La directiva `#include` s'empra per importar biblioteques estàndard o fitxers de capçalera propis, mentre que la directiva `#define` permet crear macros o definir constants que el preprocessador substituirà per tot el document.

Abans d'executar qualsevol acció, però, cal declarar les variables especificant-ne el tipus, ja que C és un llenguatge de "tipat fort". En la següent taula es detallen els tipus de dades bàsics més utilitzats:

Tipus	**Descripció**	**Mida típica (bits)**
char	Caràcter o enter petit	8
int	Nombre enter	16 / 32
float	Nombre decimal de coma flotant	32
double	Nombre decimal de doble precisió	64
void	Absència de tipus o tipus genèric	-

Taula 2.2: Tipus de dades fonamentals en llenguatge C.

Pel que fa al control del flux, C utilitza bucles principals com `while`, que s'executa mentre es compleixi una condició, o `for`, ideal per a iteracions amb un nombre de passes conegut.

Finalment queden per explicar els punters. Un punter és una variable que, en lloc d'emmagatzemar un valor numèric o un caràcter de forma directa, conté l'adreça de memòria on es troba allotjada una altra dada. Mitjançant l'operador de referència (`&`) el programador pot obtenir l'adreça de memòria d'una variable, mentre que l'operador de desreferència (`*`) permet accedir o modificar el valor emmagatzemat en aquella posició específica.

Un cop tenim el nostre programa escrit en els fitxers corresponents, cal compilar-lo per tal de transformar el text en un fitxer binari executable pel processador.

El procés de compilació consta de diverses etapes:

1. Primer, el preprocessador gestiona les directives (com les mencions anteriors de `#include` i `#define`)

2. Després, el compilador analitza la sintaxi i tradueix el codi a llenguatge assemblador.

3. A continuació, l'assemblador genera el codi objecte.

4. Dinalment, l'enllaçador o *linker* combina el codi objecte amb les biblioteques del sistema per generar l'executable final.

Aquest procés es realitza normalment mitjançant eines com el compilador GCC (*GNU Compiler Collection*), on s'especifiquen els fitxers d'entrada i el nom del fitxer de sortida desitjat.

2.4.1.5 Python

Python és un llenguatge de programació d'alt nivell, interpretat i de propòsit general que s'ha consolidat com un dels estàndards *de facto* en l'àmbit de la computació científica, l'enginyeria i l'anàlisi de dades.

Creat a principis dels anys 90 per Guido van Rossum, la seva filosofia de disseny posa un èmfasi especial en la llegibilitat del codi i en una sintaxi clara que permet als desenvolupadors expressar conceptes complexos amb menys línies de codi de les que serien necessàries en llenguatges de més baix nivell com C o Java.

Aquesta capacitat d'abstracció el converteix en una eina ideal per a la realització de tasques ràpides i per a la gestió d'aquelles que no requereixen un control estricte del temps real de la CPU.

Els punts forts de Python resideixen en la seva versatilitat i en el seu vast ecosistema de biblioteques especialitzades, conegut com a filosofia "batteries included". A més, la seva gran comunitat d'usuaris garanteix una documentació extensa i una actualització constant de les eines de desenvolupament.

Quan volem desenvolupar un programa en Python, treballem generalment amb fitxers que tenen l'extensió `.py`. A diferència de C, Python no utilitza fitxers de capçalera ni requereix una funció `main` explícita per començar l'execució, tot i que sovint s'utilitza una estructura de control per definir el punt d'entrada del programa.

Un dels trets més distintius i obligatoris del llenguatge és l'ús de l'indentació (espais en blanc al principi de la línia) per delimitar els blocs de codi, substituint les claus ({}) d'altres llenguatges. Aquesta característica obliga a mantenir un codi net i visualment estructurat per definició.

Pel que fa a la gestió de dades, Python utilitza un sistema de tipat dinàmic i fort; això significa que no cal declarar el tipus d'una variable (com `int` o `float`) abans de fer-la servir, ja que l'intèrpret el dedueix en temps d'execució, però no permet operacions incompatibles entre tipus diferents sense una conversió prèvia.

El llenguatge ofereix estructures de dades integrades molt potents, com les llistes (arrays dinàmics), les tuples i els diccionaris (taules *hash*), que faciliten l'organització d'informació complexa. Per estendre les funcionalitats bàsiques, s'utilitza la comanda `import`, que permet carregar mòduls externs, mentre que la directiva `def` s'empra per crear funcions i estructures

lògiques.

L'execució d'un programa en Python difereix significativament del procés de compilació de C. En lloc de generar un binari executable per a una arquitectura específica, el codi font es passa a un intèrpret que el tradueix a un format intermedi anomenat *bytecode* (fitxers `.pyc`). Aquest *bytecode* és executat per la Màquina Virtual de Python (PVM), que llegeix i processa les instruccions en temps real.

Tot i que aquest mètode sol ser més lent que l'execució d'un binari compilat, ofereix una gran flexibilitat, ja que permet la depuració interactiva i facilita la portabilitat absoluta del codi entre diferents màquines, actuant com una capa d'abstracció entre el programador i el maquinari físic.

2.4.2 Disseny web

El disseny web és una disciplina que es basa en el desenvolupament d'aplicacions que s'executen dins d'un navegador. Hi ha moltes menes de pàgines web diferents, com ara portals informatius estàtics, plataformes de comerç electrònic, xarxes socials o aplicacions de control i monitoratge industrial. Però totes tenen una cosa en comú: l'arquitectura.

L'arquitectura fonamental de les aplicacions web es basa en el model client-servidor. En aquest esquema, el çlient"és el dispositiu de l'usuari final (com un ordinador o un telèfon mòbil) que executa el navegador web, mentre que el "servidor"és la màquina que allotja els fitxers de l'aplicació i gestiona la lògica de control.

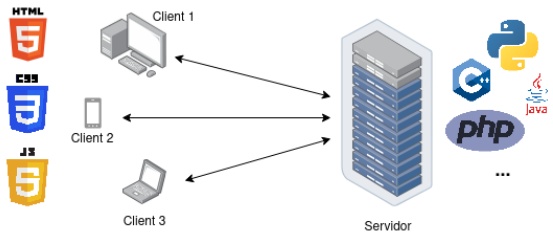

Figura 2.15: Representació de l'arquitectura client-servidor, amb els llenguatges més comuns de cadascún

A la part del client se la sol anomenar *frontend*, pel fet que constitueix la capa de presentació i la interfície visual amb la qual l'usuari interactua directament. És l'encarregada de renderitzar els elements gràfics i gestionar l'experiència d'usuari en el navegador.

Per contra, a la part del servidor se l'anomena *backend* pel motiu contrari: representa la lògica interna, el processament de dades i la comunicació amb el sistema operatiu o el maquinari, processos que s'executen en segon pla i que romanen ocults per a l'usuari final.

La comunicació entre ambdós (client i servidor) se sol realitzar mitjançant el protocol HTTP (*HyperText Transfer Protocol*) o la seva versió més segura, l'HTTPS (*HyperText Transfer Protocol Secure*), la qual afegeix una capa de xifratge per protegir la integritat de les dades. En aquest protocol, el client envia peticions i el servidor retorna els recursos necessaris, ja siguin documents HTML, fulls d'estil, imatges o dades en temps real.

L'intercanvi d'informació es canalitza principalment a través de dos mètodes o verbs HTTP: el GET i el POST.

El mètode GET s'utilitza primordialment per sol·licitar i rebre informació del servidor sense alterar-ne l'estat; en aquestes peticions, els paràmetres de cerca solen viatjar de manera visible dins de la mateixa adreça URL.

Per contra, el mètode POST s'empra quan el client vol enviar dades al servidor per ser processades, com ara l'enviament de credencials o ordres de moviment per als motors de l'antena. En aquest cas, la informació s'empaqueta dins del cos del missatge HTTP, cosa que permet gestionar volums de dades més grans i mantenir una comunicació més neta i segura des del punt de vista de la interfície.

L'estructura d'una pàgina web es defineix mitjançant l'HTML5 (*HyperText Markup Language*), un llenguatge de marques que organitza el contingut semànticament a través d'etiquetes. L'HTML s'encarrega d'establir els fonaments de la interfície, com ara els botons de control, les àrees de text i els contenidors per a les gràfiques. Complementàriament, el CSS3 (*Cascading Style Sheets*) s'utilitza per definir l'apartat visual i el disseny (*layout*). Mitjançant el CSS es controlen aspectes com el color, la tipografia, el posicionament dels elements i el disseny responsiu, que garanteix que la interfície s'adapti correctament a qualsevol mida de pantalla.

2.4.2.1 *FrontEnd*

Tornant al frontend, ja hem comentat que aquest representa la part de l'aplicació que s'executa directament en el navegador de l'usuari final.

Els llenguatges principals amb què es construeix tota l'estructura del costat del client són els següents:

- **HTML**: El llenguatge de marques encarregat de definir l'estructura bàsica i el contingut semàntic del document web.
- **CSS**: El llenguatge de fulls d'estil que s'ocupa de l'aspecte visual, el format i la disposició dels elements de la interfície.
- **Java Script (JS)**: El llenguatge de programació que permet dotar la pàgina de funcionalitats dinàmiques, lògica de control i interactivitat asíncrona.

A continuació farem una anàlisi ràpida de cadascun d'aquests llenguatges per comprendre millor el seu funcionament i sintaxi bàsica.

HTML (HyperText Markup Language) L'HTML no es considera un llenguatge de programació en el sentit estricte, sinó un llenguatge de marques que defineix l'esquelet de qualsevol aplicació web.

La seva funció principal és estructurar la informació mitjançant l'ús d'etiquetes (*tags*), les quals indiquen al navegador la jerarquia i la naturalesa dels elements que componen la pàgina, com ara encapçalaments, paràgrafs, taules o formularis de control.

```
<!DOCTYPE html>
<html>
<head>
    <title>Control Antena</title>
</head>
<body>
    <h1>Monitoratge de Sat l lits</h1>
    <div id="dades-antena">
        <p>Azimut: <span id="az">0.0</span>  </p>
        <p>Elevaci : <span id="el">0.0</span>  </p>
    </div>
    <button id="btn-inici">Iniciar Seguiment</button>
</body>
</html>
```

Listing 2.1: Exemple d'estructura bàsica en HTML5

CSS (Cascading Style Sheets) Si l'HTML és l'esquelet d'una pàgina web, el CSS n'és la pell, ja que defineix çom"es veu la pàgina, controlant aspectes com la tipografia, els colors, els marges i, especialment, la disposició de cada element en l'espai.

Mitjançant l'ús de selectors, el programador pot aplicar regles d'estil de forma global o específica, facilitant el manteniment del disseny i garantint una coherència visual en tota l'aplicació web.

```
body {
    background-color: #f0f0f0;
    font-family: Arial, sans-serif;
}

#dades-antena {
    border: 1px solid #333;
```

```
    padding: 20px;
    background: white;
    border-radius: 8px;
}

#btn-inici {
    background-color: #007bff;
    color: white;
    padding: 10px 20px;
    cursor: pointer;
}
```

Listing 2.2: Exemple de regles d'estil en CSS

JavaScript (JS) Ja sabem de l'existència de l'esquelet (HTML) i la pell (CSS), ara ja només queda per conèixer el múscul (i.e. el JavaScript). El JS és el motor de la interactivitat en el costat del client i l'únic llenguatge de programació que els navegadors web executen de forma nativa.

A diferència d'HTML i CSS, JS és un llenguatge de programació complet, amb estructures de control, variables i funcions. La seva potència resideix en la capacitat de manipular el DOM en temps real, cosa que permet que la pàgina web canviï de contingut sense haver de ser recarregada des del servidor.

En l'àmbit de la comunicació de dades, JavaScript gestiona la recepció d'informació complexa i la seva representació gràfica. Mitjançant l'ús d'objectes i mètodes, el programador pot crear lògiques de validació que assegurin que les coordenades enviades als motors estiguin dins dels límits físics de l'antena abans que la petició surti del navegador.

A més, la naturalesa asíncrona de JavaScript permet que el sistema continuï sent sensible a les ordres de l'usuari mentre s'espera la resposta d'una petició de dades, evitant bloquejos en la interfície durant l'execució del seguiment.

```
function actualitzarCoordenades(novaAz, novaEl) {
    // Validaci  de seguretat (l gica de negoci)
    if (novaAz >= 0 && novaAz <= 360) {
        // Manipulaci  del DOM en temps real
        document.getElementById('az').innerText = novaAz.toFixed(2);
        console.log("Azimut actualitzat correctament");
    } else {
        alert("Error: Coordenada d'azimut fora de rang");
    }
}
```

Listing 2.3: Exemple de lògica i manipulació del DOM amb JavaScript

Si combinem els anteriors codis ens queda una cosa com la que podem veure a la figura 2.16[3].

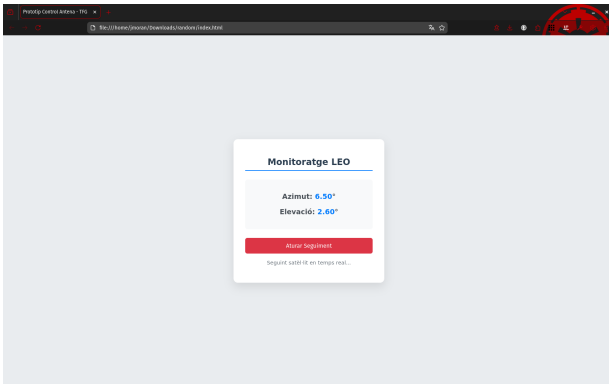

Figura 2.16: Pàgina web resultant dels codis que hem anat veient

2.4.2.2 *BackEnd*

Mentre que el *Frontend* s'encarrega de tot el que l'usuari pot veure i tocar, el *BackEnd* és el motor que s'executa en el servidor.

La seva funció principal és processar les dades enviades pel client, gestionar la lògica de negoci i interactuar amb bases de dades.

En aquest cas no hi ha un llenguatge estandarditzat, ja que el servidor és un entorn controlat on el programador té la llibertat d'escollir l'eina que millor s'adapti a les necessitats del projecte.

Un pot fer servir una gran varietat de llenguatges, com ara Python (molt utilitzat per a càlcul científic), Node.js (per a aplicacions de temps real), Java, Ruby o el clàssic C++. Cadascuna d'aquestes opcions ofereix diferents avantatges en termes de rendiment, escalabilitat o facilitat d'interconnexió amb el maquinari.

En aquest treball (com ja veurem més endavant) s'ha optat per fer servir el llenguatge PHP, així que l'explicarem ara amb una mica més de detall.

PHP (Hypertext Preprocessor) El PHP és un llenguatge de programació de codi obert especialment dissenyat per al desenvolupament web i que s'executa al costat del servidor.

[3]Hauríem de poder veure com els valors dels angles augmenta, però no podem representar-ho tan sols amb una imatge.

A diferència de JavaScript (en el seu ús convencional), el codi PHP no arriba mai al navegador de l'usuari; el servidor llegeix el codi, l'executa i només envia el resultat (normalment en forma d'HTML) al client.

Un dels trets fonamentals de PHP és la seva capacitat per integrar-se fàcilment amb servidors Apache o Nginx i la seva potència per interactuar amb fitxers i processos del sistema. Això el converteix en una eina extremadament útil per a sistemes de control, ja que permet rebre una petició HTTP des d'una interfície web i traduir-la en una ordre que el sistema operatiu pot entendre.

Pel que fa a la seva sintaxi, PHP és un llenguatge de tipat flexible on les variables sempre comencen amb el símbol del dòlar ($). Permet la programació tant procedimental com orientada a objectes, i destaca per la seva gran quantitat de funcions integrades per gestionar formularis, cadenes de text i protocols de xarxa.

```php
<?php
// Rebre coordenades per m tode POST
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $azimut = $_POST['azimut'];
    $elevacio = $_POST['elevacio'];

    // Validaci  b sica al servidor
    if ($azimut >= 0 && $azimut <= 360) {
        // Enviar comanda al port s rie (exemple conceptual)
        // shell_exec("echo 'AZ:$azimut' > /dev/ttyUSB0");
        echo json_encode(["status" => "success", "msg" => "Ordre enviada
            "]);
    } else {
        echo json_encode(["status" => "error", "msg" => "Rang invalid"])
            ;
    }
}
?>
```

Listing 2.4: Exemple de lògica de BackEnd amb PHP

2.4.2.3 Hosting

El terme *hosting* o allotjament web es refereix al servei que proveeix un sistema d'emmagatzematge i computació on resideixen els fitxers i la lògica d'una aplicació perquè aquesta sigui accessible a través d'una xarxa.

Per comprendre el seu funcionament, cal definir quatre conceptes fonamentals:

- **Servidor:** És la màquina física o virtualitzada que està permanentment encesa i connectada a la xarxa, esperant peticions dels clients per servir-los els recursos sol·licitats.
- **Adreça IP:** L'identificador numèric únic que permet localitzar el servidor dins d'una xarxa (ja sigui local o global) i que actua com la seva adreça postal digital.
- **Domini / Hostname:** El nom llegible (com `localhost` o `antena.local`) que s'associa a una adreça IP per facilitar-ne l'accés als humans, evitant haver de recordar seqüències numèriques.
- **Port:** El canal específic per on es transmet la informació; per defecte, el trànsit web HTTP utilitza el port 80, mentre que l'HTTPS utilitza el 443.

Sovint existeix la percepció errònia que un servidor és necessàriament una màquina de grans dimensions i gran potència ubicada en centres de dades massius de proveïdors com AWS (Amazon Web Services) o Google Cloud. Però no, de fet, qualsevol ordinador, independentment de la seva mida o capacitat de processament, pot actuar com a servidor. Això sí, sempre que estigui correctament configurat.

Per aconseguir crear aquest servidor personal i gestionar les peticions HTTP, existeixen un munt de tecnologies i programaris de servidor web, com ara:

- **Nginx:** Conegut per la seva alta eficiència i capacitat per gestionar moltes connexions simultànies.
- **Microsoft IIS:** El servidor web natiu per a entorns Windows Server.
- **Node.js:** Que permet utilitzar JavaScript al costat del servidor per gestionar fluxos de dades asíncrons.
- **LiteSpeed:** Un servidor d'alt rendiment sovint utilitzat en serveis de hosting comercial.

En aquest treball, però, ens hem decantat per l'ús d'**Apache HTTP Server**.

Servidor HTTP Apache Apache és un servidor web de codi obert, robust i altament modular que ha estat un estàndard en la indústria durant dècades. La seva funció principal és rebre les peticions HTTP dels clients (navegadors), processar-les i retornar el contingut corresponent, ja sigui un fitxer estàtic (HTML, imatge) o el resultat d'executar un *script* dinàmic (PHP).

El funcionament d'Apache es basa en un procés anomenat *daemon* (habitualment `httpd` o `apache2`), que s'executa en segon pla al sistema operatiu. Quan arriba una petició, Apache utilitza mòduls de processament per determinar quina acció cal prendre segons la configuració del servidor.

Un dels mòduls més importants per a aquest projecte és el `mod_php`, que permet que el servidor web entengui i executi codi PHP abans d'enviar la resposta al client.

Per fer servir Apache en un entorn de desenvolupament o control, cal seguir un flux de configuració i desplegament bàsic:

1. **Instal·lació i Directori Arrel:** Un cop instal·lat el programari, Apache defineix una carpeta específica del sistema com a "arrel" (típicament `/var/www/html` en sistemes Linux). Qualsevol fitxer col·locat en aquesta carpeta serà servit a la xarxa.

2. **Configuració de Fitxers:** El comportament del servidor es defineix en fitxers de configuració (com `apache2.conf` o `.htaccess`), on es poden establir permisos d'accés, redireccions o definir quins fitxers tenen prioritat (per exemple, carregar `index.php` per defecte).

3. **Gestió del Servei:** El servidor es gestiona mitjançant comandes de terminal que permeten iniciar, aturar o reiniciar el servei cada vegada que es realitza un canvi en la configuració.

4. **Accés Local:** Per accedir a l'aplicació des de la mateixa màquina, s'utilitza l'adreça de bucle de retorn (*loopback*) `127.0.0.1` o el nom `localhost`. Si es vol accedir des d'un altre dispositiu de la xarxa, s'ha d'utilitzar la IP privada del servidor.

Més endavant, a la fase de desenvolupament del programari del prototip, veurem com hem portat a la pràctica la configuració d'aquest servidor.

Capítol 3

Estat de l'art

L'auge del sector espacial i, especialment, la proliferació de les constel·lacions de satèl·lits en òrbita baixa (LEO), ha generat una gran diversitat d'enfocaments tecnològics per a la recepció de dades i el seguiment d'aquests dispositius.

Aquest treball es veu molt influenciat per dos projectes, un provinent del món científic i acadèmic per títol: *An Agile, Portable Antenna System for LEO Megaconstellation-Based PNT* [Qin24] ; i un segon provinent del món *maker* on es desenvolupa una base rotatòria per una càmera de fotos[L.20].

En aquest apartat només es tractarà el primer dels dos, ja que en disposem un paper i molta informació, mentre que pel segon cas no hi ha massa informació fora d'uns vídeos, i un README més centrat en el muntatge de la base rotatòria que no pas en explicar el treball.

3.1 *An Agile, Portable Antenna System for LEO*

L'estudi proposa una antena articulada mecànicament dissenyada per donar suport a l'anàlisi de l'ocupació de canals i a sistemes PNT. Notem que l'objectiu del projecte és pràcticament coincident amb el d'aquest informe.

Com a solució, els autors plantegen un mecanisme de seguiment d'alt rendiment capaç de reposicionar-se en menys d'un segon.

Hardware i Disseny Mecànic: El prototip utilitza una muntura *pan-tilt* Interbotix WidowX Dual XM430, equipada amb servomotors Dynamixel XM430 de corrent continu. Aquests motors proporcionen un parell de 4.10 Nm i una velocitat sense càrrega de 276 deg/s, prestacions suficients per moure amb precisió una antena de banya (*feedhorn*) estàndard de NSI-MI.

L'antena presenta un guany nominal de 24.4 dBi i opera en la banda de 10.0 a 15.0GHz, un rang òptim per a la recepció de senyals de la constel·lació Starlink, que és el cas d'estudi del paper.

A la figura 3.1 podem veure la seva solució.

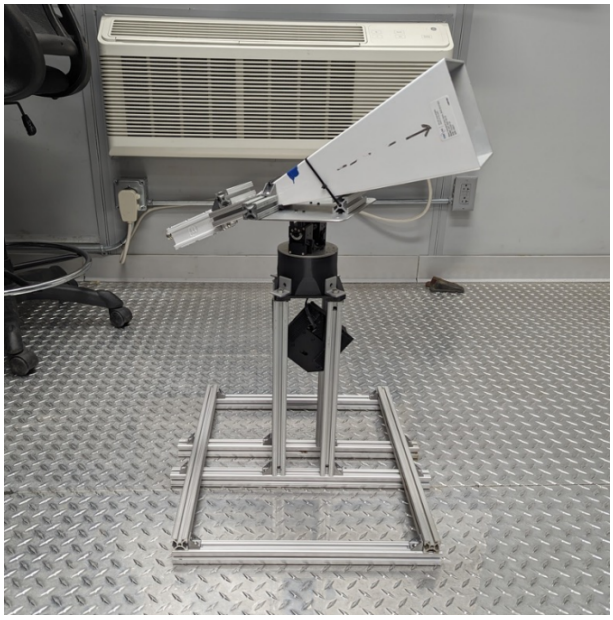

Figura 3.1: Estructura de l'antena rotatòria del paper de referència [Qin24]

Arquitectura de Software: El control del sistema s'estructura en dos mòduls principals que operen de forma asíncrona:

1. **Gestor de trajectòries:** S'encarrega de la descàrrega diària dels elements de dues línies (TLE) des de CelesTrak i de la propagació de les òrbites mitjançant el model SGP4.

2. **Controlador de motors:** Implementat sobre ROS (*Robot Operating System*), tradueix les coordenades d'azimut i elevació en ordres de moviment coordinat per als servomotors.

3.1.0.1 Resultats experimentals

Les proves de camp confirmen que el sistema pot adquirir mesures seqüencials de la constel·lació Starlink amb èxit. Els resultats demostren la capacitat del dispositiu per detectar fins a tres satèl·lits transmetent simultàniament cap a una mateixa cel·la de servei.

3.1.0.2 Conclusions i relació amb el present projecte

Aquest prototip valida la viabilitat d'utilitzar sistemes mecànics de baix cost per a la recerca avançada amb satèl·lits LEO. A més, estableix un precedent tecnològic sobre el qual s'ha basat el disseny de la nostra antena rotatòria, especialment pel que fa a l'ús de models SGP4 i l'estratègia de control mecànic.

Capítol 4

Desenvolupament del programari i interfícies

Aquest és el primer dels tres capítols dedicats al desenvolupament pràctic del projecte, en concret es treballa tot el relacionat amb el programari que tingui a veure amb el control lògic del sistema i amb les interaccions amb l'usuari. S'exclouen d'aquest apartat el programari de baix nivell (i.e. *firmware*) i els scripts de captura de dades, els quals tractaran en capítols posteriors.

S'ha optat per començar l'explicació per les interfícies gràfiques, les quals serveixen com el nexe d'unió entre les diferents funcionalitats i facilitarà la comprensió del flux de dades i operacions posteriors.

4.1 Interfície web

Amb l'objectiu de fer la interacció amb el sistema rotatori tan intuïtiva com sigui possible (evitant comandes llargues i farragoses), s'ha desenvolupat una interfície web que centralitza el control del prototip, cosa que permet que usuaris no familiaritzats amb el codi puguin governar l'antena amb seguretat.

4.1.1 La tecnologia

La decisió d'implementar una solució basada en tecnologies web, en lloc d'una aplicació d'escriptori nativa, es fonamenta en els següents factors:

- **Accessibilitat:** Una interfície web permet controlar el sistema des de qualsevol dispositiu connectat a la xarxa (ordinador, tauleta o telèfon mòbil). Sense necessitat d'anar

físicament a l'ordinador (que actuaria de servidor) o de connectar-se via SSH o altres protocols (menys amè).

- **Integració amb el SO:** Els servidors web moderns permeten una comunicació fluida amb el sistema operatiu, cosa que facilita l'execució de *scripts*, la lectura de fitxers i altres activitats en temps real.

- **Escalabilitat i Visualització:** Les tecnologies web ofereixen llibreries gràfiques potents que permetran, en futures iteracions, integrar mapes en temps real o gràfiques del seguiment dels satèl·lits de forma senzilla.

- **Facilitat de desenvolupament:** El món del disseny web és molt extens i està molt assentat en la societat actual, això fa que existeixin una gran varietat de recursos i eines d'on agafar informació i aprendre.

Dins de l'ecosistema de desenvolupament web, s'ha escollit el llenguatge PHP. Tot i que en l'actualitat existeixen alternatives com Node.js o Python (Flask/Django), el PHP s'ha seleccionat per la gran familiaritat de l'estudiant amb aquesta tecnologia.[1]

4.1.2 L'estructura i l'arquitectura

La pàgina web es divideix en tres seccions diferents:

- La **pàgina principal**: la qual servirà de resum del projecte i des d'on s'accedeix a les altres pàgines.

- La pàgina de **planificació de les captures**: des d'on es poden seleccionar satèl·lits per fer captures.

- La pàgina del **controlador lliure**: des de la que es podrà moure l'antena d'una forma més "manual".

A continuació s'explica en detall el propòsit i funcionament de cadascuna d'aquestes pàgines, però per tal de fer el treball el més transparent possible s'ha afegit el codi de les diferents pàgines a l'annex.

[1] Si a l'hora d'implementar el sistema definitiu es considera que cal fer servir una altra tecnologia es podrà fer sense una gran dificultat, doncs la interfície en si (*frontend*) no és massa complexa.

4.1.2.1 Pàgina principal

Aquesta pàgina actua com el node central i la seva funció és doble: d'una banda, contextualitzar el projecte i, de l'altra, servir de passarel·la cap a les eines de control. De les darreres hi ha dues:

- **Planificació de les captures:** On es podran programar captures als satèl·lits desitjats.
- **Controlar l'antena de forma lliure:** On es podrà moure l'antena a gust ja sigui per provar el funcionament, calibrar manualment o mostrar exemples de funcionament.

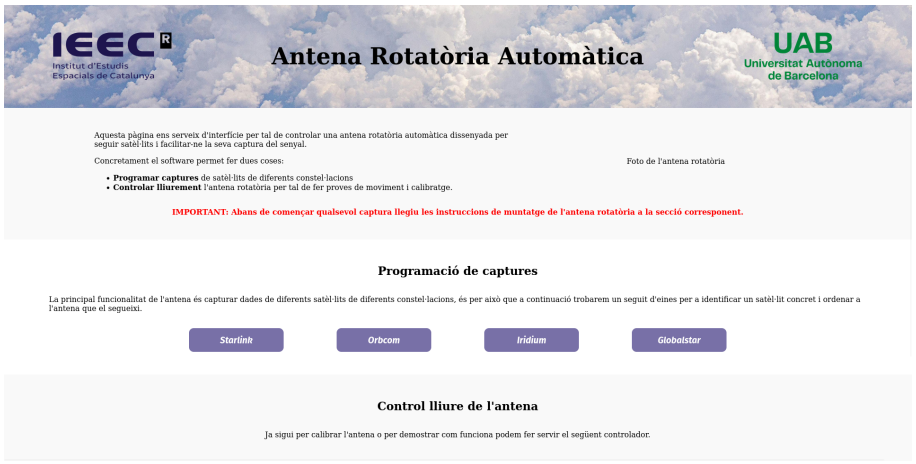

Figura 4.1: Pàgina principal

Pel que fa al desenvolupament, aquesta pàgina no ha posat masses inconvenients, ja que tots el *tags* són contenidors (i.e. ***divs***), amb un mínim d'estil per tal de centrar o canviar de color; exceptuant alguns ***buttons***, els quals senzillament adrecen a una altra pàgina fent servir o POST, si volem anar a la pàgina de planificació de les captures la qual requereix el nom de la constel·lació; o GET, en el cas del controlador lliure.

4.1.2.2 Pàgina de planificació de les captures

Aquest és el mòdul més complex i el que ha resultat més complicat de desenvolupar, ja que ha de permetre als usuaris seleccionar els satèl·lits dels quals es vulgui planificar les captures.

Per tal d'accedir a la pàgina l'usuari haurà de triar obligatòriament de quina constel·lació pretén fer captures, sempre es podrà optar a canviar de constel·lació a la part superior de la pàgina (vegeu la figura 4.2).

Un cop dins, es podrà veure un llistat de satèl·lits amb els següents paràmetres per satèl·lit:

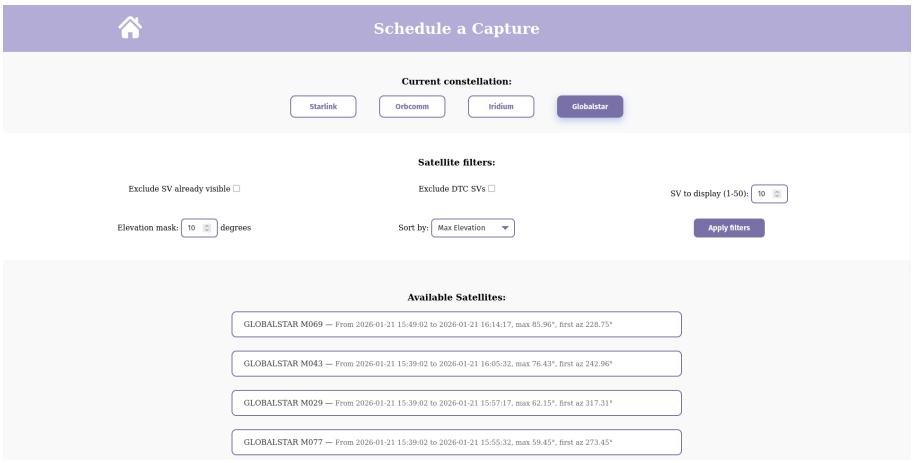

Figura 4.2: Pàgina de planificació de captures

- Nom de la constel·lació.
- Nom o número d'identificació del satèl·lit.
- Instant de temps d'entrada del satèl·lit en el camp de visió.
- Instant de temps de sortida del satèl·lit en el camp de visió.
- Elevació màxima del satèl·lit.
- Azimut en el moment de l'entrada del satèl·lit.

Tots els satèl·lits de la llista estaran disponibles en algun moment durant els pròxims 60 minuts, en altres paraules, tots aquells satèl·lits que no entrin dins del rang de visió durant aquest període queden exclosos de la llista. A l'apartat on es tracta el codi que genera la llista es veu més en detall el perquè d'això.

L'usuari podrà triar un d'aquests satèl·lits de la llista i prémer el botó de çomençar captura"que hi ha al final de la pàgina per tal de començar la captura d'aquest. Per tal d'evitar l'execució de diferents captures a la vegada es bloquejarà la pantalla i l'usuari s'haurà d'esperar (vegeu la figura 4.3).

Addicionalment, si a l'usuari no li interessa cap dels satèl·lits de la llista, s'ha creat una secció des de la qual es poden filtrar diferents satèl·lits (podem veure aquesta secció a la figura 4.3). Les formes de filtratge són les següents:

- Excloure satèl·lits visibles
- Indicar mínima elevació

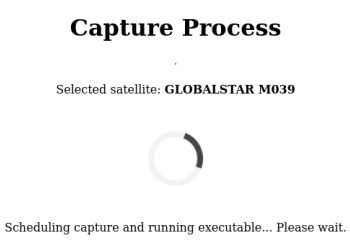

Figura 4.3: Procés de bloqueig de pantalla per tal de realitzar captures sense coincidències

- Excloure els satèl·lits DTC (*Direct to Cell*) [2].
- Indicar el nombre de satèl·lits a mostrar a la llista.

A més es podrà ordenar el llistat segons:

- Temps d'entrada: on els que entren abans en òrbita es veuran primers al llistat.
- Elevació: on els de més elevació passen pel davant dels altres.
- Llargada del període de visibilitat: on els que es poden veure durant més estona romandran a la part superior de la llista.

Finalment, també es pot editar la llargada de la llista, tot i que es recomana fer servir els filtres de forma correcta i intentar no fer la llista massa llarga per evitar equivocacions.

Pel que fa a la programació aquesta secció ha suposat ser més complexa que les altres pestanyes, ja que el codi no només serveix com a visor per l'usuari, sinó que també actua com a intermediari entre les entrades de l'usuari i els diferents algorismes en llenguatge C encarregats sigui de propagar les òrbites o processar les captures.

En primer lloc, el sistema recull els paràmetres de filtratge introduïts per l'usuari i els sotmet a un estricte procés de validació i sanejament mitjançant funcions com `filter_var` i `escapeshellarg`. Aquest pas és crític per garantir la seguretat del sistema, evitant la injecció de comandes malintencionades en invocar l'executable binari des del servidor web.

Un cop validats els filtres, el codi PHP interactua amb el sistema operatiu per executar els programes, enviant-li els arguments necessaris. La sortida dels programes serà en format de text pla, el qual serà capturat pel PHP i processat mitjançant expressions regulars (Regex). Aquesta tècnica permet transformar dades no estructurades en objectes de dades amb atributs clarament definits (nom, AOS, LOS, elevació i azimut).

Finalment, el disseny inclou una capa de lògica molt senzilla (JavaScript) que ubicarà els continguts d'aquests elements estructurats dins dels *tags* que pertoqui.

[2] Això és important només en el cas dels satèl·lits Starlink.

4.1.2.3 Pàgina del controlador lliure

Aquest apartat s'ha concebut com una eina per tal de dur a terme proves i calibratges del hardware sense haver de dependre del pas real d'un satèl·lit. En concret, les accions que es poden realitzar des d'aquest controlador són les següents:

- **Moviment manual:** Control precís dels eixos d'elevació i azimut. Podent modificar el nombre de graus a saltar mitjançant un *slider*. Totes les accions manuals es guarden en un historial en format JSON.
- **Calibratge:** Permet moure la base rotatòria de forma que l'antena miri a 0^{o} d'azimut i 90^{o} d'elevació.
- **Seqüència de test:** Seguit de moviments que s'executa de forma automàtica per comprovar que la base arriba sense problemes als límits físics de rotació i elevació.
- **Gestió de l'historial:** Dues funcions que permeten desfer o el darrer o tots els moviments introduïts de forma manual.
- **Forçar posició de repòs:** Comanda que força l'azimut a 0 i l'elevació a 90. Això servirà per fer calibratges manuals tenint en compte aspectes de l'entorn com la posició del nord o la possible inclinació de la superfície on es troba la base.

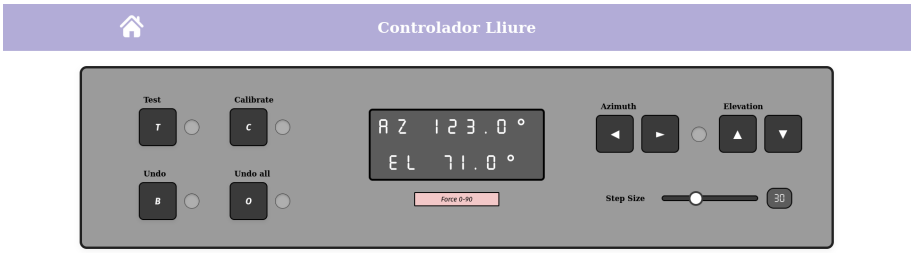

Figura 4.4: Pàgina del controlador lliure

A nivell de programació, el controlador lliure funciona de forma similar a la pàgina que gestiona les captures, és a dir, actuant com a intermediari entre el visor de l'usuari i un programa de baix nivell. A més, en aquest cas, també es bloquegen les entrades de l'usuari mentre s'estigui executant l'anterior entrada.

Al visor en si, podem veure un seguit de *buttons* identificats amb l'acció que realitzaren per via de textos. Quan qualsevol d'aquests *buttons* es premen se segueixen els següents passos d'execució:

1. **Activem la lògica d'execució asíncrona:** El programa detecta el *click* i, mitjançant una variable `isExecuting`, s'evita la superposició d'execució comandes. Addicionalment, s'invocarà la funció `disableControls()`, la qual, visualment, indica a l'usuari que la botonera es troba inhabilitada.

2. **Codifiquem i enviem de la comanda:** Es realitza una petició POST cap a un fitxer de control de la web, el qual, segons el botó premut, envia un caràcter identificatiu (T, C, L, R, U o D) perquè el programa sàpiga què fer.

3. **Esperem i activem el LED:** Per millorar l'experiència de l'usuari, el programari activa un L·LED" virtual a la pantalla canviant el seu color a verd durant el transcurs de l'operació.

4. **Tornem la botonera a l'estat original i la desbloquegem:** Un cop l'API retorna una resposta satisfactòria, la funció `updatePositionDisplay` actualitza els valors d'azimut i elevació a la pantalla del controlador, reflectint la posició real de l'antena després del moviment. A més, també es canviarà el color del L·LED" per simular que l'apaguem i s'actualitzarà la variable `isExecuting`.

Fer les coses d'aquesta forma tan ordenada permetrà garantir que els moviments de la base rotatòria es fan correctament, en ordre i sense cap mena de superposició de comandes.

4.1.3 Estil

L'estil d'aquesta pàgina es limita a centrar els continguts dels diferents contenidors i canviar colors del fons d'aquests.

El que si és important remarcar és el fet que s'han inclòs els logotips tant de la universitat (Universitat Autònoma de Barcelona) com de l'IEEC (Institut d'Estudis Espacials de Catalunya) entitat que ha promogut i finançat el projecte. Per tots dos logotips s'han seguit les instruccions indicades per les companyies sobre l'ús de la seva imatge corporativa.

Podem veure els estils de la pàgina web a l'annex .

4.1.4 Hosting

Un dels avantatges més significatius de fer la interfície gràfica una pàgina web és el fet que la podem hostejar en un servidor i accedir des de qualsevol lloc del món amb internet i un navegador. Ara bé, com que aquest projecte es tracta només d'un prototip, s'ha optat per hostejar el servidor temporalment de forma local a l'ordinador de l'estudiant, tot i que en fases més avançades del projecte es preveu la seva migració a un sistema encastat, com una Raspberry Pi, per garantir la seva autonomia.

Per tal d'hostejar el servidor s'ha fet servir Apache, un servidor HTTP de codi obert, robust i àmpliament utilitzat en la indústria. La tria d'Apache respon a la seva excel·lent integració amb el llenguatge PHP i la facilitat per gestionar permisos d'execució de fitxers binaris externs.

Els passos seguits per configurar el servidor en aquest entorn són els següents:

1. **Estructura del repositori:** Per mantenir l'ordre del projecte, s'ha creat una estructura de carpetes al directori arrel del servidor on se separen clarament les vistes (*Web-Page_Views*), els estils CSS, els scripts de client (JS) i, sobretot, els executables en C (*Executables*).

2. **Gestió de permisos d'execució:** Un dels punts crítics ha estat configurar els permisos del sistema de fitxers perquè l'usuari d'Apache tingui privilegis d'execució sobre els binaris compilats en C.

3. **Instal·lació i entorn:** s'ha actualitzat l'Apache a l'ordinador de l'estudiant i s'hi ha configurat un servidor de forma que actuï com a *host* virtual (i.e. *localhost*) al port 8080 (vegeu el fitxer de configuració d'Apache 4.1).

```
<VirtualHost *:8080>
    ServerAdmin webmaster@localhost
    DocumentRoot "/home/.../GitHub/TFG_web_code"

    <Directory "/home/.../GitHub/TFG_web_code">
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>
```

Listing 4.1: Configuració del VirtualHost d'Apache per al projecte.

Aquests passos s'hauran de repetir de cara a fer un servidor pròpiament dit, canviat el *host* virtual a un de real. Segurament també s'haurà d'afegir alguna capa de seguretat.

4.2 Identificació de satèl·lits

En aquest apartat es tractarà el programa que genera el llistat de satèl·lits disponibles a l'apartat "*pàgina de planificació de les captures*"de la pàgina web.

4.2.1 Arquitectura

A la secció anterior d'aquest informe (i.e. Interfície web) es deixa clar que aquest programa es crida en dos moments diferents:

- Quan l'usuari **accedeix a la pàgina** o **canvia de constel·lació** [3]
- Quan l'usuari **aplica filtres** al llistat

És important saber diferenciar entre aquests dos casos, doncs les comandes que s'executaran seran diferents. Pel primer cas la comanda es veu de la forma: "*./main G*", mentre que pel segon cas es veurà de la forma: "*./main G 1 1 30 1 15*". Els paràmetres que podem veure en ambdues comandes són les següents:

- Constel·lació
- Excloure satèl·lits visibles: escriure 1 els exclou i 0 els inclou.
- Excloure satèl·lits DTC: escriure 1 els exclou i 0 els inclou.
- Elevació mínima: amb un mínim de 0^{o} i un màxim de 90^{o}.
- Ordenar per: (1) elevació, de gran a petit; (2) temps d'entrada, de més d'hora a més tard; i (3) per llargada de temps de visió, de més a menys.
- Nombre de satèl·lits a imprimir per pantalla.

Un cop llançada la comanda, sigui quin sigui el motiu d'execució, el primer que farà és descobrir quina és la constel·lació que es vol analitzar (i.e. el primer paràmetre, igual en ambdós casos). Aquesta constel·lació podrà ser:

- Starlink: ja sigui escrivint "Starlink", "S"o "s"
- Orbcomm: ja sigui escrivint "Orbcomm", "O"o "o"
- Iridium: ja sigui escrivint Ïridium", Ï"o ï"
- Globalstar: ja sigui escrivint "Globalstar", "G"o "g"

En cas d'introduir una constel·lació incorrecta o senzillament introduir una comanda errònia, també es comprovarà en aquesta primera fase.

[3] Tot i semblar dos casos diferents en realitat són casos idèntics

Un cop descoberta la constel·lació es descarreguen les dades dels fitxers TLE corresponents per via d'un çurl"al servidor del NORAD on trobem aquests fitxers actualitzats. Aquests fitxers ens venen en forma textual i caldrà *parsejar*-los per poder tractar amb les dades, per a això s'ha creat una funció que tradueix les dades del format obtingut a l'útil.

A continuació el que s'ha fet és iterar per cada satèl·lit i, combinant les dades del TLE d'aquest amb la posició en format ECEF i la data actual (UCT) en format de data juliana, s'ha executat l'algorisme SGP4 per tal de propagar l'òrbita del satèl·lit per la següent hora [4].

Aquest algorisme SGP4 s'ha aprofitat a partir d'una llibreria per C.

Per cada instant de la propagació (1 segon) es transforma la posició ECEF del satèl·lit en format Azimut/Elevació. Aquest format permetrà mirar si el satèl·lit està visible des de la nostra posició o no i, en cas de ser-ho, anotarem l'azimut d'entrada i l'elevació màxima a la qual arribarà.

En acabar de fer el pas d'un satèl·lit, anotarem les següents dades del pas del satèl·lit en un *struct*.

- Nom del satèl·lit
- Temps d'entrada
- Temps de sortida
- Elevació màxima
- Azimut inicial

Com que s'obtenen molts *struct*s de tots els satèl·lits, s'ha decidit ordenar-los en un vector des del qual, un cop acabada la propagació dels satèl·lits, es filtren aquells que s'escaigui (i si s'escau) i s'ordenen fent servir un algorisme d'ordenació (*Bubble sort*) pel paràmetre que s'hagi indicat per l'usuari.

Aquesta llista serà impresa per pantalla i ja serà feina del servidor web guardar les dades en un fitxer (.txt) des del qual es puguin llegir i tractar des d'un altre lloc.

A continuació es desa un diagrama d'activitats simplificat de tot aquest funcionament del programa d'identificació de satèl·lits (vegeu la figura 4.5). De totes maneres, qui vulgui fer una anàlisi més a fons del codi el podrà trobar a l'annex.

[4] Al marc teòric s'explica el funcionament d'aquest algorisme.

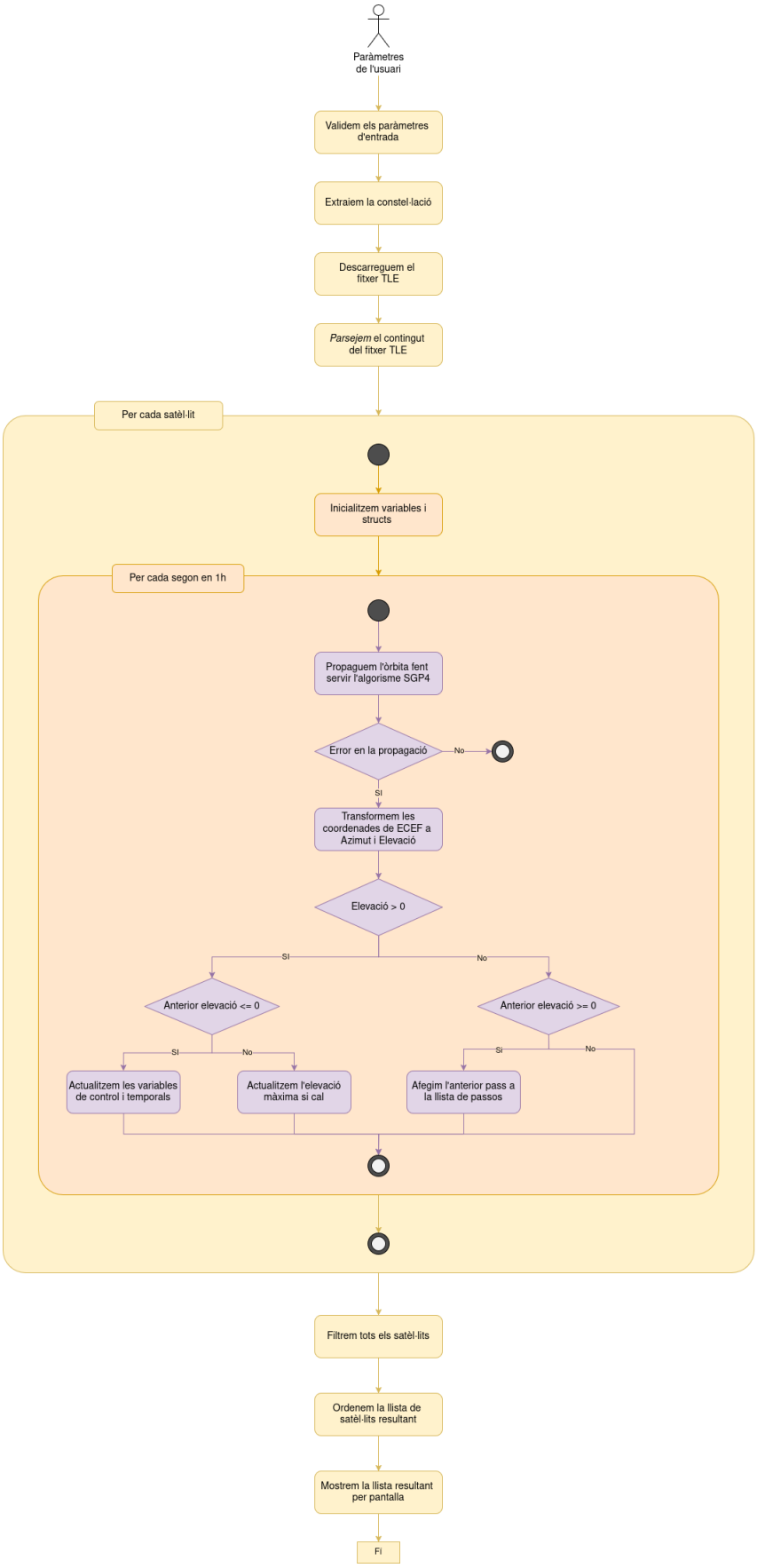

Figura 4.5: Diagrama d'activitats del codi d'identificació de satèl·lits[5]

4.2.2 Validació

Per tal de validar el funcionament d'aquest programa s'ha fet servir eines en línia que ens permeten veure la posició dels satèl·lits en viu (com ara [Glo24]). L'eina en concret que s'ha fet servir permet seleccionar un satèl·lit i veure per on passarà les pròximes hores.

A partir de les dades visuals de l'eina en línia i de la llista del nostre programa (vegeu la figura) ara només cal comparar les dues. Interessarà fixar-se en els següents aspectes:

- **Elevació màxima:** S'ordena per altura la llista i es busquen els SV a l'eina, en cas que totes les òrbites passin per sobre nostre a força altura es valida aquest aspecte.

- **Azimut d'entrada:** Es fa el mateix que amb l'elevació i es mra la direcció general per on entren les línies al nostre camp de visió (SE, N, O, ...) si coincideixen fins a cert marge amb els graus predits es valida aquest aspecte.

- **Hora d'entrada:** Si s'ordena la llista per temps i es fa que es vegin els satèl·lits que actualment es troben sobre nostre es pot veure molt ràpidament si hem fet bé aquest apartat. En cas que algun SV es trobi una mica lluny es pot comparar també l'hora de sortida i determinar si just acaba d'entrar o no.

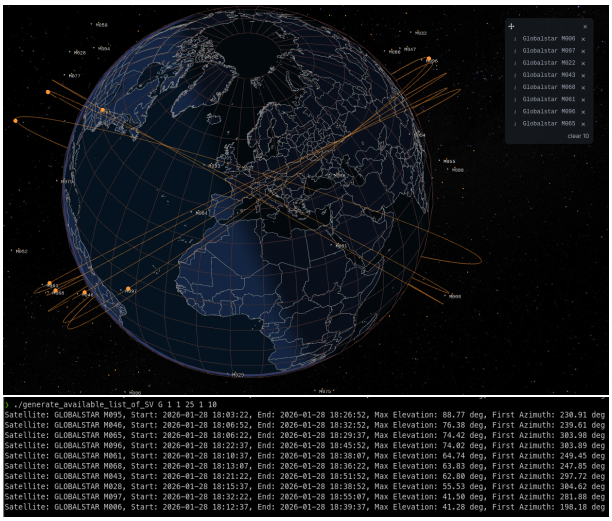

Figura 4.6: Comparació visual entre les geometries d'òrbita de Globalstar extretes de l'eina online [Glo24] i el llistat de satèl·lits de més altitud en la següent hora generat pel codi d'aquest projecte.

Si aquests tres factors s'aproximen al calculat, el dona el programa per bo.

4.3 Seguiment de satèl·lits

En aquesta secció es veu el programa encarregat de, donat un satèl·lit, generar el seguit de comandes que, posteriorment, enviarà a l'Arduino en temps real per tal de moure la base rotatòria.

4.3.1 Arquitectura

Aquest programa s'executarà quan l'usuari primi el botó de çomençar captura"al fons de la pàgina de planificació de captures. Aquesta mateixa acció enviarà dos paràmetres a l'executable: el nom del satèl·lit seleccionat i la constel·lació (exemple d'execució: *./process_antenna_movements "GLOBALSTAR M094"G*).

El primer que farà el programa amb aquestes dades és propagar l'òrbita del satèl·lit, d'una forma si similar a com ho fèiem amb el codi anterior (), però ara només amb un satèl·lit. Resumint:

1. **Descarreguem els fitxers TLE** de la constel·lació indicada.
2. **Extraiem les dades** del satèl·lit indicat del fitxer TLE.
3. **Propaguem l'òrbita** d'aquest satèl·lit fins que deixi d'estar dins del nostre rang de visió.

Per cada instant de propagació [6] farem dues coses:

1. **Convertirem les coordenades** ECEF a Azimut-Elevació i arrodonirem els valors dels darrers per tal de simplificar els següents passos.
2. **Generarem una comanda per l'Arduino** per a moure l'estructura rotatòria el total de graus de diferència respecte a l'anterior moviment.

Les comandes que nosaltres podem enviar a l'Arduino són les següents:

- **MAHXXX**: Acrònim per "Moure Azimut en sentit Horari"on XXX representa el nombre de graus a moure.
- **MAAXXX**: Acrònim per "Moure Azimut en sentit Anti-Horari"on XXX representa el nombre de graus a moure.
- **MEUYYY**: Acrònim per "Move Elevation Up"[7] on YYY representa el nombre de graus a moure.

[6] En el nostre cas un instant de propagació és 1 segon, però podria ser menys per una major precissió.

[7] Se que alguns acrònims venen del català i alguns de l'anglès, però sinó ho feia així les comandes quedaven molt similars les unes amb les altres.

- **MEDYYY**: Acrònim per "Move Elevation Down"on YYY representa el nombre de graus a moure.

La generació de comandes també té en compte coses com: el màxim nombre de graus a moure, si convé girar a la dreta o esquerra o si es tracta de la primera comanda o no. Però en l'àmbit conceptual només ens hem de quedar amb què: es generarà una comanda per cada instant de propagació on es realitzi un desplaçament de més d'un grau.

Les comandes es guardaran en un diccionari, que vincularà cada comanda amb l'instant de temps en el que s'ha de realitzar, per tal de tindre en compte l'evolució en temps real de l'òrbita i no executar tots els moviments un rere l'altre sense pausa.

Addicionalment, forçarem una comanda anomenada Ç"a l'instant en el que l'elevació del satèl·lit superi els 25 graus. Aquesta comanda no s'enviarà a l'Arduino, senzillament en ser detectada es donarà l'ordre d'executar el programa de captura del senyal, el qual s'iniciarà, executarà i acabarà independentment d'aquest.

Un cop generades les comandes ara cal executar-les, però, abans de res, calibrarem l'antena. Per calibrar entenem moure la base rotatòria a la posició 0-azimut i 90-elevació, idealment apuntant al nord, però, en ser un prototip, aquesta funcionalitat tan sols és un *placeholder*[8].

A l'hora d'executar les comandes senzillament crearem un bucle *while* etern que a cada iteració miri quina hora és i, en cas de ser superior al *timestamp* de la següent comanda, executarà la comanda i l'eliminarà de la llista de comandes. Dit això, les primeres comandes (i.e. les que ens mouran a l'origen de l'aparició del satèl·lit) s'executaran a l'instant 0, per tal d'aprofitar al màxim el temps mort abans de l'aparició del satèl·lit.

És imprescindible que, per executar les comandes, estigui connectat l'Arduino a un dels ports *serial* de l'ordinador.

Un cop acabades d'executar les comandes el programa desfarà el moviment realitzat per l'antena. Això es fa per tal de no entortolligar els cables amb el pas del temps.

4.3.2 Validació

Validar el correcte funcionament d'aquest programa és un procés costos, temporalment parlant. Per començar, cal encarar la base rotatòria al nord (o un nord fals) i assegurar-nos que tot estigui ben connectat; a continuació executarem el codi que ens proporciona el llistat de satèl·lits (recomanem ordenar per temps d'aparició i excloent els que ja estan en òrbita) i triarem el que ens faci més ràbia; finalment executarem la comanda del programa amb els paràmetres corresponents i comprovarem les següents coses:

[8] A les conclusions del projecte tornarem a tractar aquesta casuística.

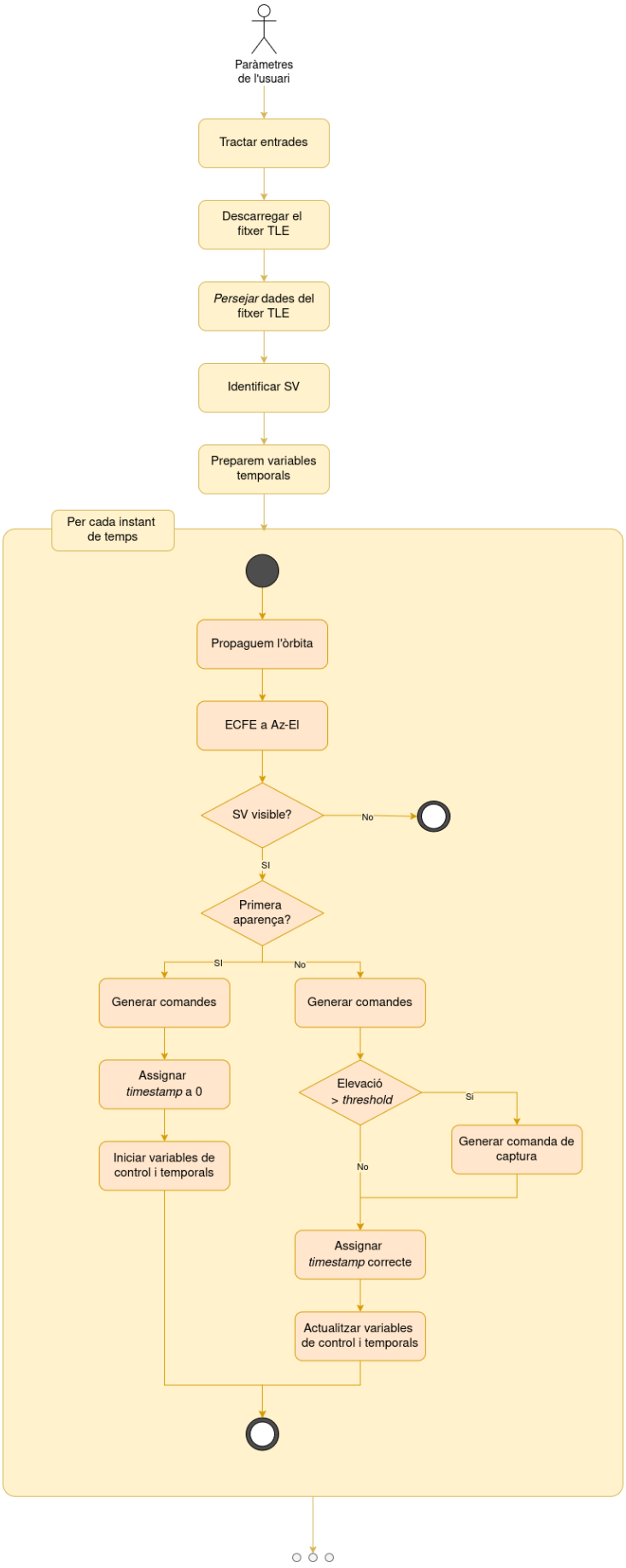

Figura 4.7: Diagrama d'activitats del codi del seguiment de satèl·lits 1

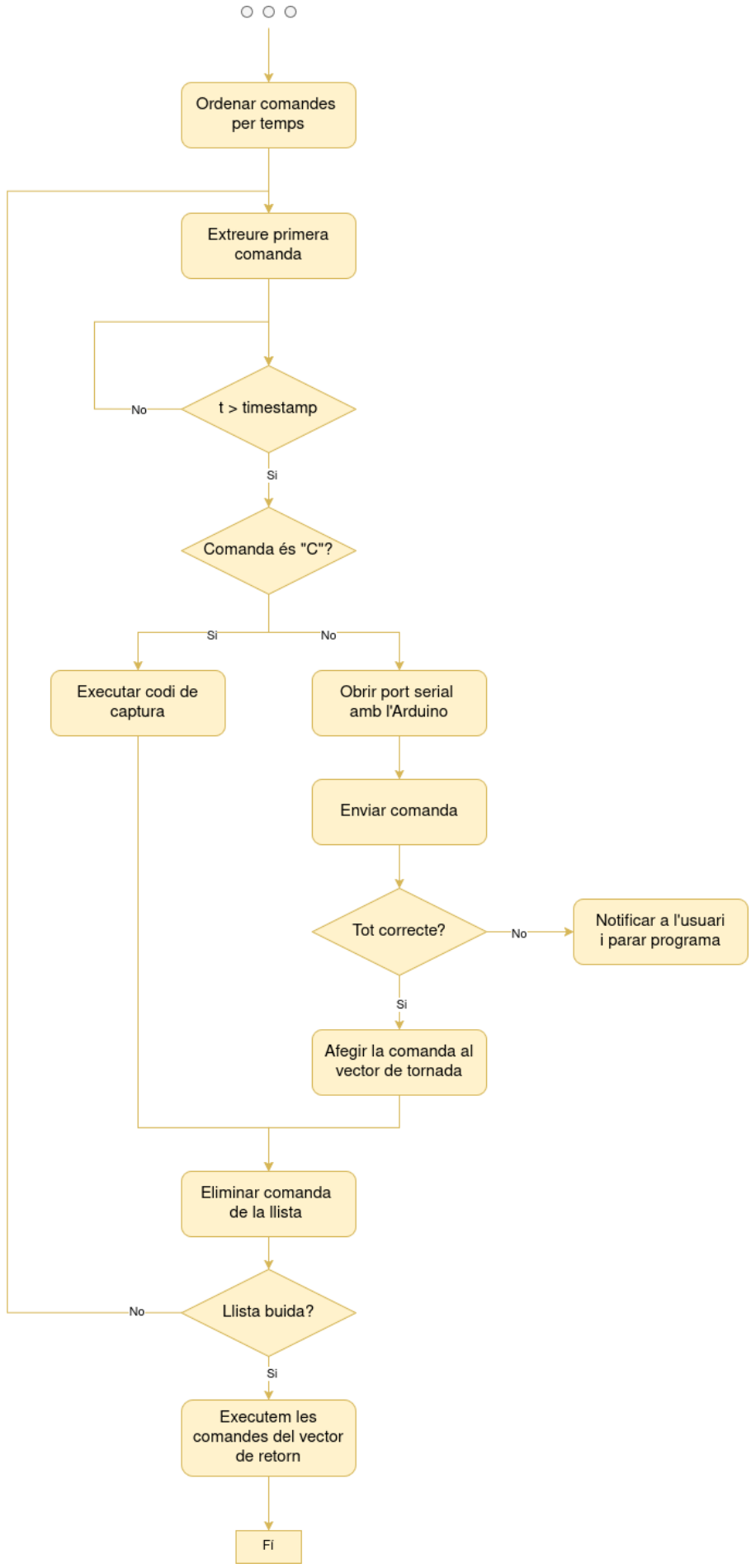

Figura 4.8: Diagrama d'activitats del codi del seguiment de satèl·lits 2

- **Els moviments que encaren l'antena amb el satèl·lit tenen sentit.** És a dir que no girin en sentit oposat, ni massa ni massa poc.
- **Els instants d'execució són correctes.** Si fem un seguiment de la direcció cap on apunta l'antena i la comparem amb la posició de l'òrbita real, hauríem de veure com aquests coincideixen.
- **Retorn de l'antena a la posició inicial correcte.** Si no hi ha hagut cap problema o error, en retornar l'antena a la posició original, no s'hauria de percebre cap desviament.

Durant la fase de proves del prototip es va realitzar un vídeo en format *timelapse* de la rotació de l'antena. Aquest el podeu trobar a l'enllaç: https://www.youtube.com/shorts/BQt4AMmJ8hA o (en cas d'estar llegint aquest document en físic) al QR de la figura 4.9.

Figura 4.9: QR al vídeo de l'antena rotant. Enllaç complet: https://www.youtube.com/shorts/BQt4AMmJ8hA

4.4 Moviment lliure

Aquest darrer programa és el més senzill de tots, ja que el seu propòsit és senzillament el de traduir comandes explícites de la pàgina web a l'Arduino i executar-les.

4.4.1 Arquitectura

L'únic moment en què l'usuari executarà aquest codi serà quan accioni botons a la pàgina de çontrolador lliure"de la pàgina web, aquests botons recordem que són:

- Test [T]: Mou la base rotatòria al límit.
- Calibrar [C]: Calibra l'antena al nord (deshabilitada al prototip).
- Enrere (*Back*) [B]: Desfà l'últim moviment realitzat.
- Origen [O]: Desfà tots els moviments.
- Forçar [F]: Força la posició actual de la base rotatòria a Azimut 0 i elevació 90.
- Pujar (Up) [U]: Mou la base rotatòria cap en amunt.
- Baixar (Down) [D]: Mou la base rotatòria cap en avall.
- Dreta [R][9]: Mou la base rotatòria en sentit horari.
- Esquerra [L]: Mou la base rotatòria en sentit antihorari.

El codi doncs acceptarà les comandes en la seva forma monosil·làbica (i.e. T, C, B, O, F, U, D, R, L) i un step com a segona paràmetre en el cas de les comandes de moviment individual (U, D, R, L).

Abans de continuar explicant s'ha de fer esmena al fet que, en conjunt amb aquest codi, s'ha creat un fitxer format JSON que ens permet fer un seguiment de la posició de l'antena independentment del moment en què es cridin les comandes. Si l'usuari mogués l'antena a la dreta un dia i el següent voldria desfer el moviment, només caldria llegir aquest fitxer i es podria retornar a l'origen. En total aquest fitxer JSON només té 3 entrades:

- *currentAz*: un enter que representa l'angle Azimut actual
- *currentEl*: un enter que representa l'angle d'elevació actual.
- *LastMovements*: un vector ordenat de les comandes realitzades (les comandes a l'estil Arduino).

L'existència d'aquest fitxer ens ajuda tant a evitar sobrepassar límits d'elevació com a evitar entortolligaments de cables a causa d'un elevat nombre de rotacions de la base.

Tornant al codi, un cop comprovada la validesa dels paràmetres i identificada la instrucció a executar, el codi llegeix el fitxer JSON i executa la rutina de tractament d'instrucció pertinent a cada cas, resumint:

[9]Som conscients de la incongruència i confusió que pot generar que les comandes s'anomenin dreta i esquerra en comptes d'horari i antihorari. Això és degut a una confusió inicial que s'ha tornat problemàtica de resoldre a temps d'edició d'aquest document.

Acció: Test [T] El primer que farà aquesta funció serà calibrar l'antena (vegeu l'apartat dedicat al calibratge), és a dir, mourà els eixos de la base a 0 graus d'Azimut i 90 d'elevació.

Un cop posicionat al centre, es realitzaran els següents moviments, en ordre:

1. Moure l'antena 360 graus en sentit antihorari.
2. Moure l'antena 90 graus cap en avall.
3. Moure l'antena 360 graus en sentit horari.
4. Moure l'antena 90 graus cap en amunt.
5. Esborrar els continguts del fitxer JSON i posar la posició actual (0-90).

L'objectiu d'aquesta funció és dur a terme una prova d'estrès a la base rotatòria per tal de veure si hi ha qualsevol cosa que impedeixi realitzar cap moviment.

Acció: Calibrar [C] Com ja hem dit en més d'una ocasió, aquesta acció hauria d'encarar l'antena al nord (Azimut 0 al nord) i apuntar l'antena al zenit de forma idònia (elevació 90). Per això s'haurien de fer servir sensors especialitzats que, per aquest prototip no farem servir.

De moment el que fa aquesta funció és, d'acord amb la posició actual de l'antena, retornar a 0^{o} d'Azimut i 90^{o} d'elevació i eliminar el contingut del fitxer JSON.

Acció: Enrere [B] Aquesta acció llegeix l'última comanda executada del fitxer JSON, n'extreu l'oposada (p.e: MAH010 es torna MAA010) i l'executa.

En comptes d'afegir la comanda inversa el que farem serà eliminar la comanda que hem contrarestat de la llista.

Acció: Origen [O] Aquesta acció llegeix el llistat complet de comandes i les desfà d'una en una de forma similar a la comanda "Enrere"(vegeu l'apartat corresponent a aquesta comanda).

Acció: Forçar [F] Aquesta acció esborra tot el contingut del fitxer JSON i escriu 0^{o} al paràmetre que indica l'azimut actual i 90^{o} al que representa l'elevació.

Aquesta funció és útil quan volem fer un calibratge manual de la base rotatòria.

Accions: Pujar [U], Baixar[D], Dreta [R] i Esquerra [L] Aquestes accions són potser les més fàcils d'entendre, de fet, totes elles segueixen la mateixa seqüència d'instruccions, en ordre:

1. Llegir el *step*.
2. Comprovar que el moviment és vàlid.
3. Generar comanda (format Arduino).
4. Executar la comanda
5. Escriure el moviment i actualitzar els camps del JSON

Aquests moviments individuals ens poden servir tant per fer un *fine tunning* del calibratge de l'antena, com per demostrar el seu funcionament o per realitzar accions manuals de prova.

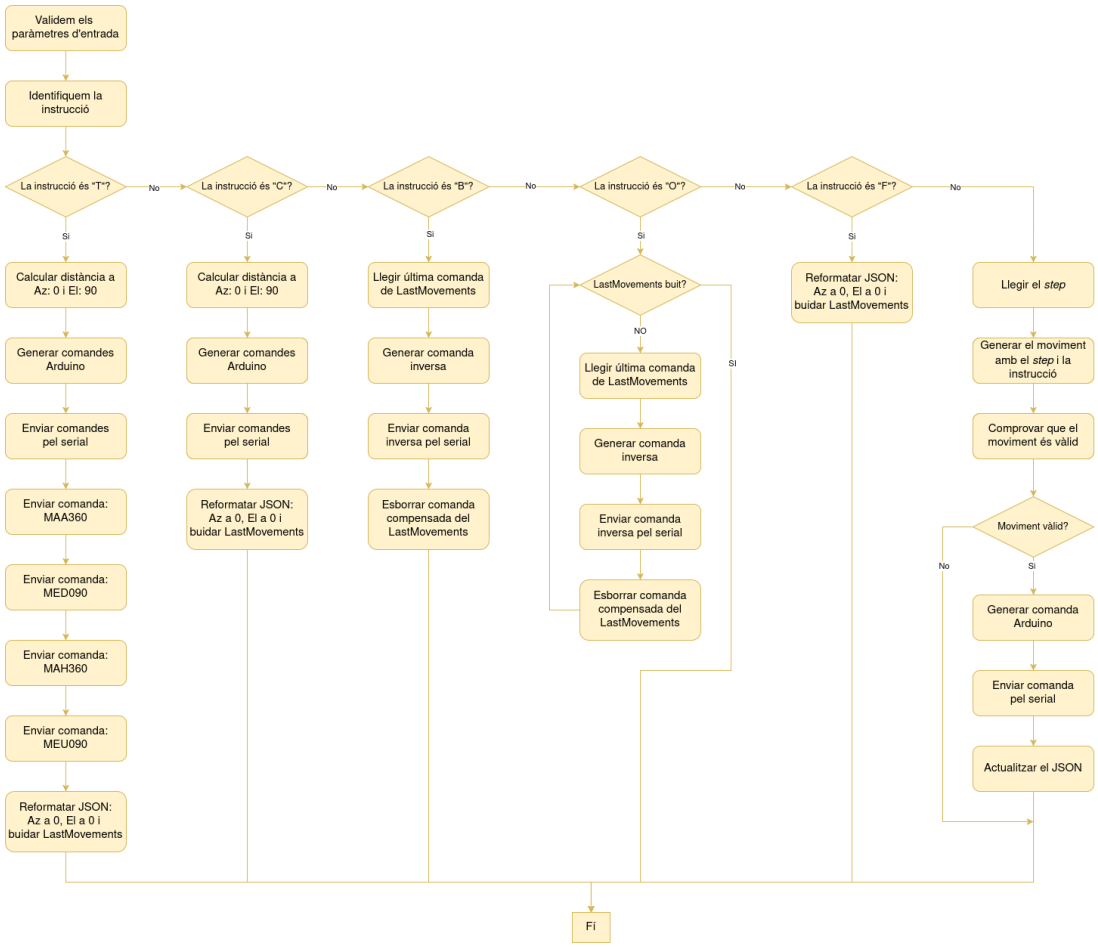

Figura 4.10: Diagrama d'activitats del codi del controlador lliure

4.4.2 Validació

La validació d'aquest programa es basa en anar comanda per comanda assegurant-se que: l'antena es mou en cas d'haver-se de moure i que el JSON s'actualitza correctament.

Tot i això, en haver de fer tantes validacions, és molt fàcil deixar-se de fer una comanda o, incús, confondre comandes entre elles i pensar-se que s'està validant el moviment horari en comptes de l'antihorari; així que cal anar amb compte.

Amb aquest últim programa finalitza l'explicació i el desenvolupament de la primera part del prototip (desenvolupament del programari i interfícies), tot i que no es pot donar per acabada a escala pràctica fins realitzar les altres parts i es pugui acabar de validar aquells punts que requereixin o de moure l'antena o de capturar senyal.

Capítol 5

Disseny i assemblatge de l'estructura

En aquest segon capítol dedicat al desenvolupament pràctic del projecte es pot veure tota la feina feta al voltant de l'estructura i *hardware* d'aquest. Es tractaran temes com el disseny mecànic, la selecció de components electrònics, l'arquitectura d'interconnexió i, finalment, la implementació del *firmware* que actua com a interfície de control entre les ordres d'alt nivell i els actuadors.

Cal puntualitzar que els elements dedicats exclusivament a la recepció del senyal (com ara l'SDR) queden fora de l'abast d'aquest capítol i seran analitzats detalladament a la secció corresponent.

Abans començar, recordem que l'estructura ha de ser capaç d'orientar mecànicament un element (i.e. l'antena) de forma que aquest pugui apuntar a qualsevol espai del cel visible (elevació ≥ 0).

5.1 Requisits i selecció de components electrònics

Per tal de dissenyar una estructura cal, abans de res, determinar quines són les càrregues que ha de suportar aquesta. Per aquest motiu, abans de continuar amb el modelatge del prototip, cal definir tots aquells components de mida i pesos significatius que conformen el sistema, començant per l'element principal: l'antena.

5.1.1 Antena

L'antena és l'element cèntric d'aquest projecte i, com a tal, determinarà els requisits de parell motor (*torque*), inèrcia i resistència estructural.

Tot i que existeixen diverses menes d'antenes (p.e. les Yagi-Uda, parabòliques, còniques o

phased arrays), s'ha optat per un disseny capaç d'adaptar-se diferents tipologies d'antenes. Amb l'objectiu de garantir l'escalabilitat del projecte i permetre futures línies d'investigació més enllà del prototip.

El sistema s'ha pensat doncs per al cas més restrictiu: **una antena parabòlica de 5kg i de dimensions (aproximades) de 350×350×50 mm**. Aquesta elecció respon al fet que les antenes parabòliques presenten una major resistència al vent i un moment d'inèrcia superior respecte a altres models.[1]

Ara bé, de cara al projecte, l'antena que farem servir és la "Gatling 2.4 MK II"de la companyia TrueRC. Aquesta antena en realitat són 4 antenes (i.e. un array de 4 antenes), té les propietats de la taula 5.1 i un diagrama de radiació com el de la figura 5.1.

Paràmetre	**Valor**
Guany (Gain)	15.5 dBic
Polarització	RHCP o LHCP
Ample de banda (Bandwidth)	2.3 – 2.7 GHz
SWR	$\leq 1.5:1$
Pes (Weight)	350 g
Dimensions	27 × 27 × 9 cm

Taula 5.1: Especificacions Tècniques de l'Antena Gatling 2.4 MK II

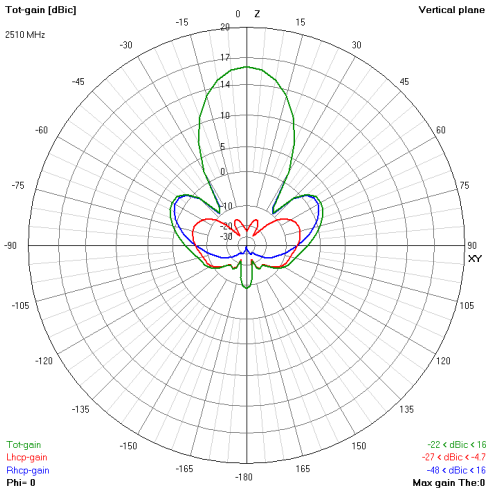

Figura 5.1: Diagrama de radiació de l'antena Gatling 2.4 MK II

Els principals motius pels quals hem triat aquesta antena són:

[1] Aquesta restricció no vol dir que el prototip hagi d'aguantar aquestes masses, sinó que, a l'hora de fer la tria de components i dissenyar l'estructura, es tindrà en compte.

- És una antena **provada per altres membres** del departament i que ha demostrat ser **capaç de resultats** positius altres cops.

- L'antena **ja està disponible** al grup de recerca promotor del TFG, cosa que minimitza el temps de cerca i compra.

- Les propietats com el **pes i les mides** fan que sigui una tria idònia de cara al prototip, ja que ens permetran fer proves sense preocupar-nos pels límits físics de l'estructura.

A la figura 5.2 podem veure l'aspecte d'aquesta antena.

Figura 5.2: Imatge de l'antena Gatling 2.4 MK II

5.1.2 Motors

La selecció dels motors és un factor determinant per assegurar tant la fluïdesa del moviment com la precisió angular necessària per al seguiment de satèl·lits LEO.

És per aquest motiu que, d'acord amb les tipologies analitzades en el marc teòric, s'ha optat per l'arquitectura de motors pas a pas (Stepper motors), concretament de la sèrie NEMA (vegeu la figura 5.3). Aquesta decisió es fonamenta en, com ja hem dit, la seva capacitat per mantenir un parell elevat i d'oferir una resolució angular molt precisa.

Ara, per determinar el model específic, cal calcular el parell motor nominal () necessari per vèncer les resistències del sistema sota les condicions de càrrega més severes, les quals es presenten quan l'eix d'elevació quan l'antena es troba en posició horitzontal (vegeu imatge 5.4).

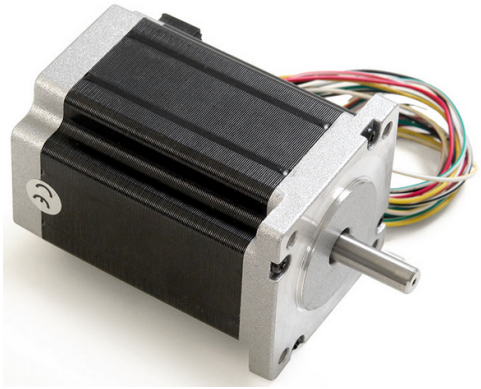

Figura 5.3: Imatge d'un motor pas a pas NEMA-23

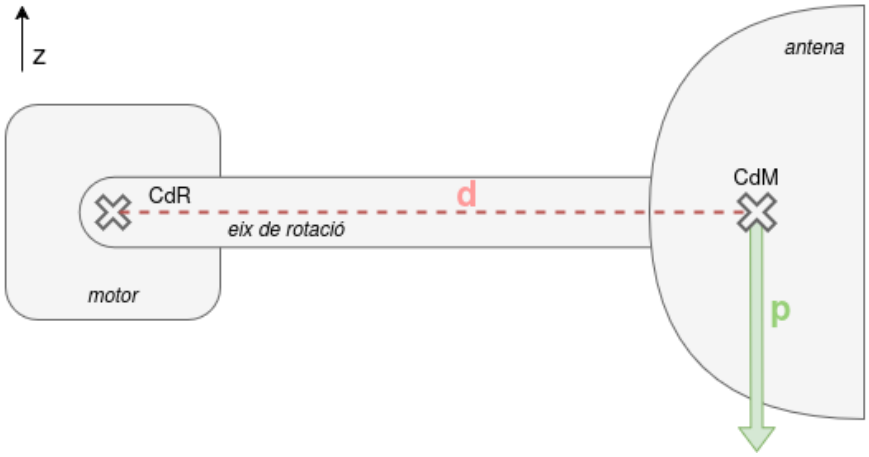

Figura 5.4: Esquema del cas més desfavorable. La fletxa negra indica la direcció al zenit (elevació 90^{o}), la d és la distància entre el centre de rotació i el centre de masses de l'antena i p el pes d'aquesta.

5.1.2.1 Anàlisi del parell estàtic (Γ_e)

El parell estàtic en el cas més desfavorable es calcula com:

$$\Gamma_e = m \cdot g \cdot d \cdot \sin(\Theta)$$

Considerant una massa $m = 5\text{kg}$, una gravetat $g \approx 9,81 m/s2$, un desplaçament del centre de masses $d = 0,2\text{m}$ i un angle crític (desplaçament respecte del zenit) $\Theta = 90$:

$$\Gamma_e = 5 \cdot 9,81 \cdot 0,2 \cdot 1 = 9.81 \text{ Nm} \approx 10 \text{ Nm}$$

5.1.2.2 Anàlisi del parell dinàmic (Γ_d)

A més del parell estàtic, a nosaltres també ens interessa moure l'antena, és per això que, per avaluar l'esforç durant el moviment, considerem el moment d'inèrcia de la càrrega (I). Aproximant l'antena a un disc de radi R=0,175 m (recordem que volem un diàmetre de 350mm)i aplicant el teorema de Steiner per al desplaçament d:

$$I = \frac{1}{2}mR^2 + md^2 = \frac{1}{2}(5)(0,175)^2 + 5(0,2)^2 \approx 0,27 \text{ kg} \cdot \text{m}^2$$

Per a una acceleració angular conservadora de $=1^{\text{o}}/\text{s}^2$ (convertit a radiants: 0,0175 rad/s^2):

$$\Gamma_d = I \cdot \alpha = 0,27 \cdot 0,0175 \approx 0,0033 \text{ Nm}$$

Es pot veure com, en aplicacions de seguiment satel·litari on les velocitats són baixes, el component estàtic (gravitatori) predomina clarament sobre el dinàmic.

5.1.2.3 Parell de disseny final

Sumant ambdós components i aplicant un factor de seguretat S=2 per absorbir pèrdues per fricció mecànica en els coixinets i la resistència aerodinàmica:

$$\Gamma_{total} = S \cdot (\Gamma_e + \Gamma_d) = 2 \cdot (10 + 0,0033) \approx 20 \text{ Nm} \tag{5.1}$$

Aquest valor de parell és molt elevat, sobretot pel tipus de motor que volem seleccionar (NEMA-17 o NEMA-32), els quals tenen uns parells que volten els 0.5Nm en el cas dels NEMA-17 o 2.4Nm en el cas dels NEMA-23.

No obstant això, com que la nostra aplicació no requereix realitzar moviments d'alta velocitat, ens podem permetre sacrificar aquesta per tal d'augmentar el moment fent servir una cadena d'engranatges reductors. Existeixen uns complements anomenats "*planetary gearboxes*"que permeten tot just això i que, a més, podem trobar models pensats per encaixar amb els motors NEMA esmenats.

A partir de tots aquests conceptes i valors hem decidit comprar el motor **NEMA-23**, (model 23HS32-4004S) el qual ofereix un parell nominal de 2,4 Nm (vegeu la taula 5.2 per veure les especificacions d'aquest motor). Malgrat que el disseny teòric per a una antena de 5 kg requeriria l'ús de reductores planetàries (*planetary gearboxes*) per assolir els 20 Nm de seguretat, s'ha determinat que per a la implementació del prototip actual (antena Gatling 2.4 MK II, amb una massa de 0,35 kg i un parell resistent de ≈0,6 Nm), el motor actuant de forma directa és suficient. Aquesta configuració permet simplificar la mecànica del prototip sense comprometre la viabilitat de l'estudi, mantenint la possibilitat d'afegir reductores en iteracions posteriors.

Especificació	**Valor**
Angle de pas (Step Angle)	1.8°
Parell estàtic (Holding Torque)	2.4 Nm (340 oz.in)
Corrent nominal/fase	4.0 A
Mida del marc (Frame Size)	57 x 57 mm
Longitud del cos	82 mm
Diàmetre de l'eix (Shaft Diameter)	8 mm
Longitud de l'eix	21 mm
Nombre de cables	4
Pes	1.2 kg

Taula 5.2: Especificacions tècniques importants del motor NEMA23 triat.

5.1.2.4 Controladors i gestió del pas (*Microstepping*)

L'arquitectura de motors pas a pas seleccionada es caracteritza per tindre una molt bona precisió a l'hora d'avançar i retrocedir graus. No obstant això, el control d'aquests motors presenta dos desafiaments tècnics principals:

- **Precisió del senyal:** A diferència d'altres tipus de motors, els *steppers* requereixen una commutació molt precisa dels senyals que dicten el moviment.
- **Tensions elevades:** Els motors NEMA-23 treballen a tensions força elevades (típicament entre 24V i 50V), mentre que els microcontroladors més habituals solen treballar a no més de 5V.

Per tal de governar aquests motors requerirem, doncs, una unitat de potència intermèdia (i.e. controlador) que actuï com a interfície entre els senyals lògics del microcontrolador i les bobines del motor. En el nostre cas hem decidit comprar el controlador **DM542T** de *StepperOnline* (vegeu la figura **??**).

Aquest model de controlador no només ens soluciona els problemes esmenats, sinó que, a més, té la capacitat de realitzar *microstepping* (vegeu la taula 5.3). Aquesta tècnica divideix cada pas físic del motor en subdivisions lògiques mitjançant la modulació del corrent en les bobines. Això permet augmentar la resolució del sistema, reduint les vibracions mecàniques i assegurant una trajectòria de seguiment més suau i precisa.

Addicionalment, el controlador DM542 ha estat dissenyat pensant per a donar abast a molts motors NEMA diferents, per tant, haurem d'indicar quin és el corrent que volem proporcionar

Figura 5.5: Controlador DM542 de *StepperOnline*

Pulse/rev	**SW5**	**SW6**	**SW7**	**SW8**
200	on	on	on	on
400	off	on	on	on
800	on	off	on	on
1600	off	off	on	on
3200	on	on	off	on
6400	off	on	off	on
12800	on	off	off	on
25600	off	off	off	on
1000	on	on	on	off
2000	off	on	on	off
4000	on	off	on	off
5000	off	off	on	off
8000	on	on	off	off
10000	off	on	off	off
20000	on	off	off	off
25000	off	off	off	off

Taula 5.3: Combinacions de *microstepping* del controlador DM542

a aquest.

Peak	**RMS**	**SW1**	**SW2**	**SW3**
1.00A	0.71A	on	on	on
1.46A	1.04A	off	on	on
1.91A	1.36A	on	off	on
2.37A	1.69A	off	off	on
2.84A	2.03A	on	on	off
3.31A	2.36A	off	on	off
3.76A	2.69A	on	off	off
4.50A	3.20A	off	off	off

Taula 5.4: Taula de corrents del controlador (Peak = RMS x 1.4)

Pel que fa al nostre cas concret, el controlador s'ha alimentat amb una tensió de 35V (el màxim permès per la nostra font) i l'hem configurat segons s'indica a la taula **??**.

	SW1	SW2	SW3	SW5	SW6	SW7	SW8
Tria	On	ON	OFF	On	On	Off	Off

Taula 5.5: Configuració del controlador del motor

En apartats posteriors veurem com connectar el motor al controlador i aquest a l'esquemàtic.

5.1.3 Microcontrolador

Per tal d'interpretar les comandes que envien els diferents programes d'alt nivell i traduir-les a trens de polsos pels controladors dels motors, cal una unitat de control.

A l'hora de fer aquesta tria no ens hem complicat i hem decidit que comprar un Arduino Uno (xip ATmega328P). El motiu de la tria sent que aquesta placa és un estàndard en el prototipatge electrònic, amb una tolerància elèctrica excel·lent i una facilitat d'interconnexió que minimitza els riscos de fallada en l'etapa de desenvolupament.

A la taula 5.6 es detallen les especificacions tècniques més rellevants de la unitat de control utilitzada:

Cal destacar que, tot i que el microcontrolador opera a 5V, és plenament compatible amb les entrades de control opto-acoblades del driver DM542T, garantint un aïllament galvànic entre l'etapa de control i la de potència.

Característica	**Especificació**
Microcontrolador	ATmega328P (arquitectura RISC de 8 bits)
Velocitat de rellotge	16 MHz
Memòria Flash	32 KB (dels quals 0,5 KB usats pel bootloader)
SRAM / EEPROM	2 KB / 1 KB
PINS E/S Digitals	14 (dels quals 6 proporcionen sortida PWM)
Entrades Analògiques	6 (ADC de 10 bits)
Tensió d'operació	5 V

Taula 5.6: Especificacions tècniques de l'Arduino Uno Rev3.

Component	Quantitat	Dimensions (mm)	Massa (kg)
NEMA 23	2	57 x 57 x 82	1.25
DM542	2	11.8 x 2.5 x 7.5	0.21
Gatling 2.4 MK II	1	270 x 270 x 90	0.35
Arduino	1	68 x 53 x 15	0.025

Taula 5.7: Taula resum dels components electrònics

5.2 Disseny de l'estructura i components mecànics

Un cop sabem quines són les característiques de disseny i tots els components electrònics seleccionats (vegeu la taula 5.7) ja podem començar a dissenyar l'estructura. Per a això el que hem fet ha sigut adaptar l'estructura d'una base rotatòria per una càmera de fotos (aplicada a l'estat de l'art d'aquest projecte).

Aquesta estructura ha estat pensada per a fabricar-se amb tecnologies d'impressió 3D (mètodes de Fabricació Additiva) i, per tant, no ens ve com una estructura completa, sinó per parts, cadascuna d'aquestes parts (models) s'ha d'imprimir per separat i ajuntar després amb les seves adjacents.

Aquesta estructura, però, presenta un problema fonamental que farà que hàgim de realitzar canvis i adaptacions. Aquest problema és la tria i el posicionament dels motors.

Resumint, els dissenyadors de l'estructura fan servir motors NEMA 17, els quals són considerablement més petits i lleugers que els nostres, això fa que, si fóssim a posar els nostres motors on els seus (vegeu figura 5.6) tindríem els següents problemes:

- **Inestabilitat:** en pesar tant i estar tan apartats del centre de l'estructura, els motors farien que aquesta s'inclinés cap a un costat, fent que l'estructura sencera corregués el perill de caure.
- **Problemes d'encaix:** en ser motors diferents no encaixen als forats dissenyats a les peces, cosa que fa que s'hagin de dissenyar nous forats i encaixos.
- **Col·lisions:** en ser més grans els angles de rotació de certes parts entrarien en contacte amb els motors.

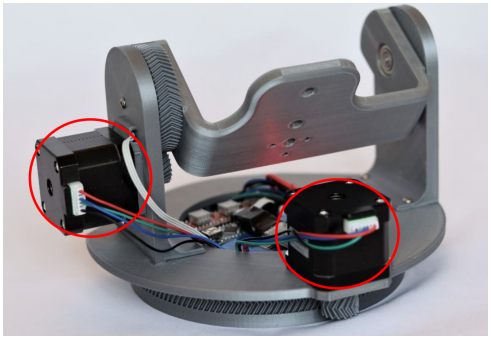

Figura 5.6: Estructura completa del projecte de la base rotatòria d'una càmera amb els motors identificats

Al llistat 5.8 podem veure el nom de les peces originals de l'estructura i a la imatge 5.7 el model final després del muntatge. A mesura que anem explicant els diferents canvis fets als models, ja veurem imatges de l'aspecte tant de les peces originals com de les modificacions. No obstant això, a l'annex del projecte podrem trobar totes les peces definitives fetes servir al prototip (incloses les que s'han conservat dels models originals).

Figura 5.7: Estructura completa del projecte de la base rotatòria d'una càmera.

Nom del model	Conservada	Modificada	Eliminada
17_Tooth_Herringbone_Gear		X	
21_Tooth_Herringbone_Gear.stl		X	
64_Tooth_Herringbone_Gear	X		
Printable_Bearing_Cage_Part_3			X
Printable_Bearing_Part_1			X
Printable_Bearing_Part_2			X
144_Tooth_Herringbone_Gear_Base_Mount		X	
Base_Mount_Bearing_Clip_Ring			X
Camera_Mounting_Bolt_Spacer	X		
Gear_Side_Bearing_Mount	X		
Gear_Side_Support		X	
Hall_Effect_Stepper_Mount			X
Idle_Side_Bearing_Mount	X		
Idle_Side_Support	X		
Pan_Mount		X	
Pan_Mount_Bearing_Clamp			X
Tilt_U-Mount	X		

Taula 5.8: Llistat dels diferents models del projecte de la base d'una càmera rotatòria. Les columnes Çonservada", "Modificadaï "Eliminadaïndiquen el seu rol en el nostre projecte.

A continuació explicarem totes les modificacions fetes als diferents models i els nous models dissenyats, començant per la part superior de l'antena i anant baixant.

5.2.1 Base del motor d'elevació

El primer problema que ens topem és amb el motor encarregat de moure l'antena en l'angle d'elevació, aquest com podem veure a la figura 5.6 està situat a l'exterior de l'estructura, provocant una gran inestabilitat.

Per solucionar això hem decidit moure el motor a l'interior de l'estructura (vegeu la figura 5.8). En fer aquest moviment ara tenim 2 nous problemes:

- El suport de l'antena (Tilt_U-Mount) es troba en mig.

- No hi ha res que suporti el motor a l'interior de l'estructura.

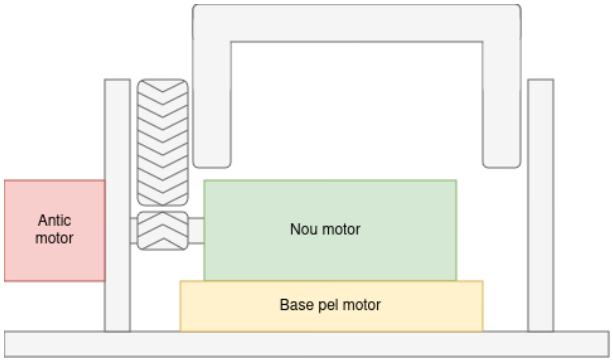

Figura 5.8: Representació gràfica de la nova ubicació del motor d'elevació i dels nous models que cal fer

El primer problema se soluciona fàcilment si, en comptes de fer que el suport tingui forma de U, fem que tingui forma de ∩. Això no només soluciona el problema sinó que també ens és més útil a l'hora d'encaixar diferents models d'antena, ja que hi ha més espai d'aquesta forma.

El segon problema ha sigut més complicat de resoldre, ja que, fins ara el motor anava acollat a la pota (Gear_side_support), però ara, en canviar de motor i de banda ja no es pot.

La nostra solució ha sigut, en comptes d'acollar el motor com l'altre a la mateixa pota, dissenyar un suport que vagi a la base dels suports (Pan_Mount) i que faci de base plana pel motor (vegeu el model 3D a la figura 5.9).

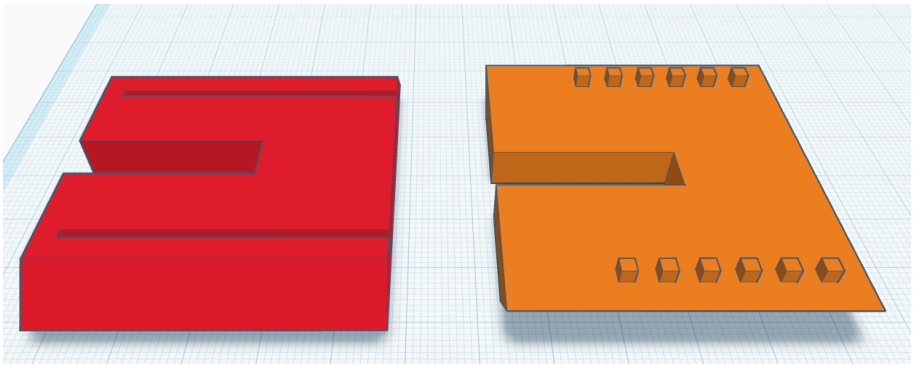

Figura 5.9: Model 3D de la base del motor

Podem veure a les figures 5.9 i **??** com aquest suport és força complex i té moltes entrades i sortints, els expliquem en ordre:

- Els **sortints quadrats** que podem veure a la part inferior són uns pins que encaixaran perfectament amb el model que va just a sota inferior (vegeu la figura **??**). Impedeixen que la base es bellugui.

- **L'entrada horitzontal gruixuda** que hi ha al centre permet que els cables del motor i d'altres components baixin a capes inferiors de la base rotatòria (a l'apartat del muntatge de l'estructura hi entrarem en més detall).
- Les **entrades allargades i primes** que podem trobar als laterals de la part superior són entrants on aniran arrepenjades unes parets planes (vegeu les figures 5.10) que serviran per evitar que el motor es mogui horitzontalment quan intenti moure's.

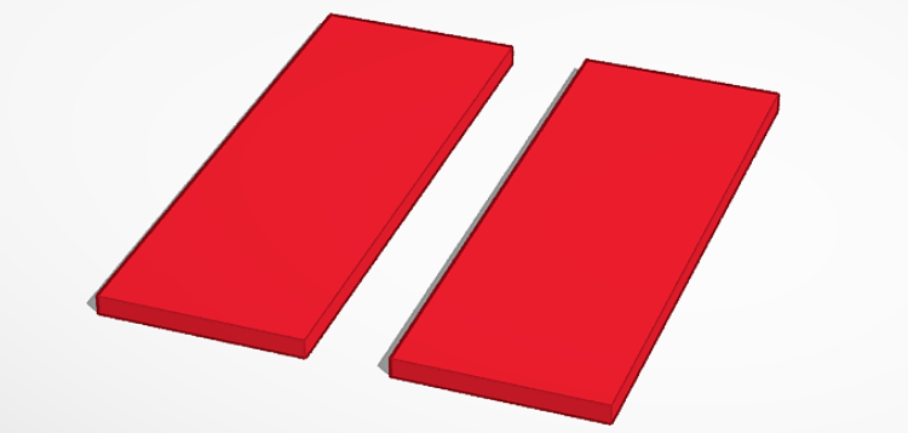

Figura 5.10: Model 3D de les parets del motor

Aquesta base del motor i les parets han resultat ser molt útils i pràctiques.

5.2.2 El motor Azimut i la base rotatòria

El motor que s'encarrega de moure l'elevació no és l'únic que causava problemes, el de l'Azimut també té un efecte desestabilitzant que podria fer caure l'antena en cas d'una ventada o davant d'una càrrega molt pesada. És per això que s'ha optat per treure el motor de la base rotatòria i reubicar-lo a terra (capa més baixa de l'estructura) (vegeu la figura 5.11).

Aquest canvi és necessari, però genera molts inconvenients:

- **Canvi de base rotatòria:** Fins ara aquest motor rotava junt amb la plataforma, però ara el motor no rota, però la mateixa plataforma s'ha de moure (en aquesta secció tractarem això).
- **Necessitat d'una base més alta:** En moure el motor avall caldrà elevar tota l'estructura (a la secció següent tractarem això).

Abans d'aplicar una solució al primer problema, cal analitzar l'element que permet el canvi entre la secció estàtica (la base fixada a terra) i la secció mòbil (la plataforma que sustenta

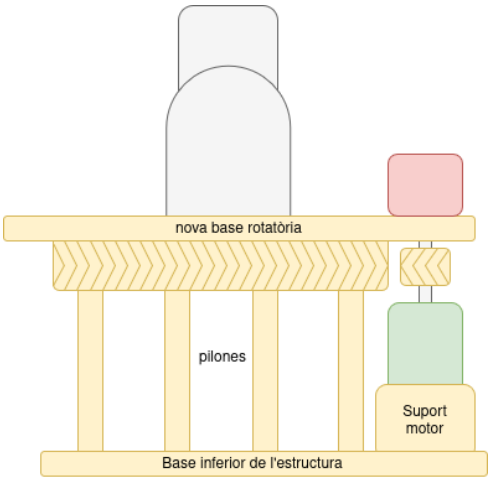

Figura 5.11: Representació gràfica de la nova ubicació del motor d'elevació i dels nous models que cal fer

l'antena). Aquest element s'anomena *Bearing* [2] i està conformat per dues anelles concèntriques amb una pista de rodament i una sèrie de boles d'acer a l'interior que minimitzen la fricció durant el gir. La seva funció principal és absorbir les càrregues axials derivades del pes de l'estructura i de l'antena, evitant que aquestes recaiguin directament sobre l'eix del motor i puguin arribar a danyar-lo[3].

Figura 5.12: Representació d'un *Bearing*

Resumint, el *bearing* permet que la plataforma superior "floti"sobre la base estàtica, desacoblant l'esforç estructural del moviment de rotació. A la taula **??** podem veure les especificacions del *bearing* que farem servir i a la figura 5.12 una representació d'un *bearing*. No analitzarem els *bearings* més enllà de les seves dimensions.

[2]En català correcte seria çoixinet de boles", però nosaltres farem servir la versió anglesa.

[3]Un exemple d'objecte de la vida quotidiana que fa servir un bearing és la joguina "*fidget spiner*"o les rodes d'un patinet o *skate*

Símbol	**Valor**	**Descripció**
d	127mm	Diàmetre de l'alesatge (Bore diameter)
D	146.05mm	Diàmetre exterior (Outside diameter)
B	12.7mm	Amplada (Width)
C_r	11200N	Capacitat de càrrega dinàmica bàsica, radial
C_{0r}	17800N	Capacitat de càrrega estàtica bàsica, radial
n_G	17101/min	Velocitat límit
$\approx$ m	310g	Pes
Dimensions		
r_{min}	0.38mm	Dimensió mínima del xamfrà
d_1	134mm	Diàmetre de l'espatlla de l'anell interior
D_1	139mm	Diàmetre de l'espatlla de l'anell exterior
d_1	130.8mm	Diàmetre del calibre de l'anell interior
D_1	140.8mm	Diàmetre del calibre de l'anell exterior
α	30°	Angle de contacte

Taula 5.9: Especificacions del *bearing* que farem servir

El problema és que en el disseny original, la part interior del *bearing* era la que rotava quan el motor s'impulsava contra la part exterior del *bearing* (i.e. la part estàtica). Ara, en fer que el motor es quedi estàtic, però que se segueixi impulsant contra la part exterior del *bearing*, aconseguim que sigui la part exterior la que roti i no la interior.

La solució ha sigut canviar els papers de la part exterior i la interior del *bearing*, fent que sigui la primera la que roti i la darrera la que es quedi estàtica.

Per a això el que hem fet ha sigut modificar la base rotatòria (Pan_Mount) per tal d'eliminar el que la subjectava a la part interior i ajuntar-la a l'element exterior (i.e. l'engranatge que impulsa el motor). A la figura 5.13 podem veure el resultat d'aquestes modificacions.

En aquest element (figures **??** i **??**) podem veure que, a part del que ja hem comentat del *bearing*, hi ha diferents forats a l'estructura. Bé aquests forats no són res més que socs on podrem encaixar: els suports laterals del suport de l'antena, la base del motor d'elevació (comentat a l'apartat anterior) i un forat central per on passar cables de la part rotatòria a l'estàtica.

Abans d'acabar l'apartat volem comentar el fet que aquest disseny (figures **??** i **??**) presenta 2 problemes:

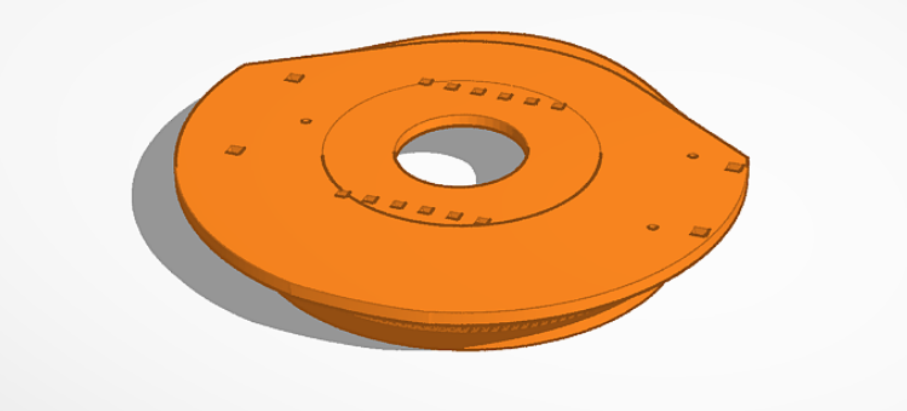

Figura 5.13: Model 3D de la base rotatòria

- **Problema d'encaix** entre el *bearing* i la part exterior: bàsicament el *bearing* no entrava i hem hagut de cisellar (amb un cisell i maça) l'interior de la peça impresa per tal de forçar l'encaix.
- **Problema de fregament superior:** tant la part interior com l'exterior del *bearring* tocaven la base rotatòria (fent ridícul l'ús del *bearing*) és per això que hem hagut de crear una volandera que separi els dos (vegeu la figura 5.14).

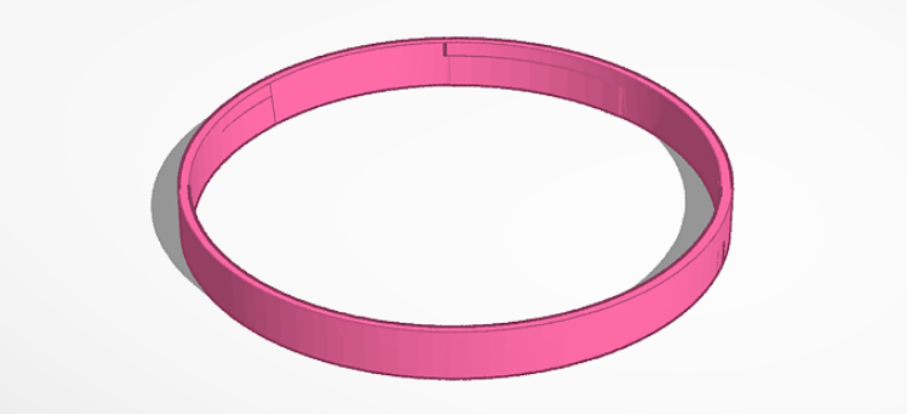

Figura 5.14: Model 3D de la volandera

5.2.3 La base estàtica

Finalment, per tal de solucionar el problema de l'altura afegida pel motor de l'Azimut, hem dissenyat una nova part interior del *bearing* (vegeu la figura 5.17) capaç d'encaixar-se amb 6 columnes còniques (vegeu la figura **??**).

També hem creat una base per tal d'evitar que la part inferior de les columnes balli i posi en risc l'estabilitat de l'estructura sencera (vegeu la figura 5.17). Aquesta mateixa base també

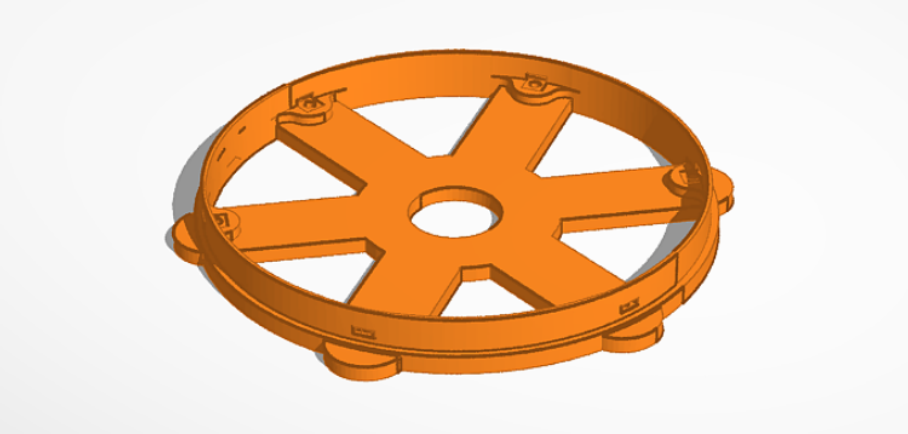

Figura 5.15: Model 3D de la base interior del *bearing*

Figura 5.16: Model 3D de la pilona

té dos sortints que donaran a la base del motor, un model que té forma de capsa i que evita que el motor d'Azimut es desenganxi de la resta de l'estructura (vegeu la fiugra **??**).

Podem veure com els braços que enganxen la base de l'estructura amb la del motor azimut tenen diferents forats, això ho hem fet a propòsit per veure quina era la distància idònia entre el motor i l'engranatge rotatori[4].

5.2.4 Procés de disseny

Pel que fa al procés de disseny d'aquestes peces, rara vegada s'ha dissenyat una peça i ha funcionat, sovint s'han necessitat dues o tres impressions de la mateixa peça, sigui perquè ens hem deixat una funcionalitat o una mesura no era del tot correcte.

[4]Fer aquest aspecte a prova i error suposa una menor dificultat que la mesura en si.

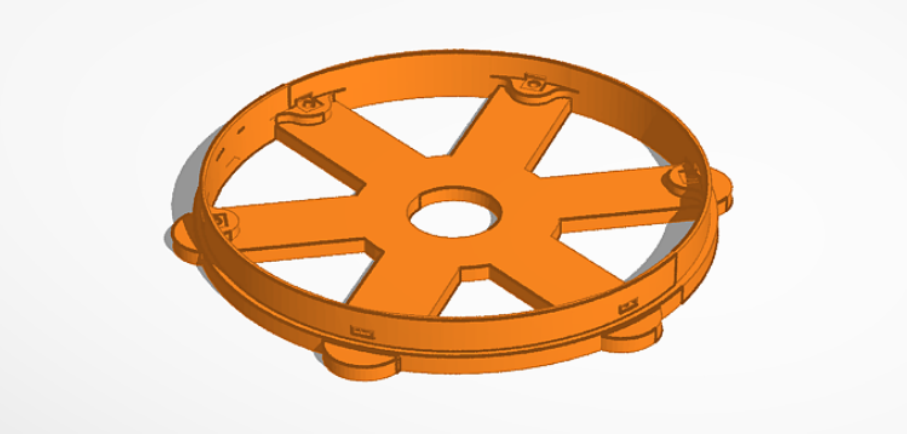

Figura 5.17: Model 3D de la base interior del *bearing*

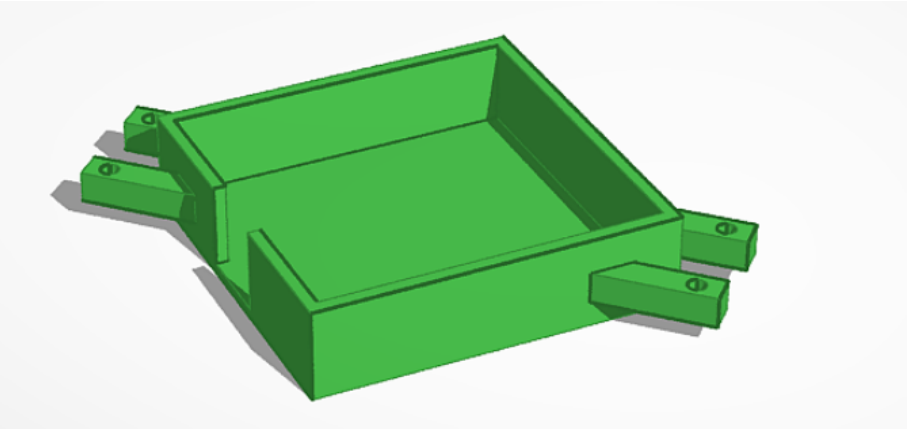

Figura 5.18: Model 3D de la pilona

Els dissenys dels models 3D s'han dut a terme amb un programa gratuït, obert i web anomenat "TinkerCAD". Aquest programa és força senzill i el seu ús està força limitat a aspectes acadèmics i d'aprenentatge o dissenys de models 3D no massa complexes, com en el nostre cas.

El procés d'impressió s'ha dut a terme a l'espai "Open Labs"de l'escola d'enginyeria, on disposen d'una gran varietat d'impressores 3D (i altres) i un equip tècnic sempre disposat a ajudar amb el que sigui.

A la secció següent tractarem el muntatge de l'estructura i dels models dissenyats en aquest apartat.

5.3 Muntatge de l'estructura

Un cop dissenyats tots els models de l'estructura ara cal ajuntar-los i consolidar l'estructura. A continuació deixem un llistat complet de totes els models, components electrònics i components mecànics que hem fet servir (vegeu la taula 5.10).

Quan tinguem tots els models impresos i el material comprat el procés de muntatge és el següent:

5.3.1 Muntatge dels models de la base rotatòria

Si comencem muntant la base des d'amunt cap avall el primer que hauríem de fer és cargolar el "64_Tooth_Herringbone_Gear"al "Tilt_U-Mount"fent servir diferents cargols M3 de 12mm [5]. A continuació cal encaixar els *bearings* petits als forats dedicats a ells als models "64_Tooth_Herringbone_Gearï "Tilt U-Mount".

Ara deixem aquesta part a part un moment i agafem els suports laterals (el Ïdle_Side_Supportï el "Gear_Side_Support") i els enganxem als forats dedicats a la "base rotatòria".

Ara per ajuntar aquestes dues parts farem servir els dos pinyols Ïdle_Side_Bearing_Mountï "Gear_Side_Bearing_Mount", els quals aniran entre la part interior dels *bearings* petits i els forats dedicats als suports corresponents. Un cop posats els pinyols caldrà, amb molta traça, ajuntar les dues parts.

Finalment, col·locarem la "Base del motor elevació"als forats corresponents a la "base rotatòria".

5.3.2 Muntatge dels models de la base estàtica

Comencem muntant la base estàtica caragolant les diferents pilones a la "Base pilones: superior", per això farem servir els caragols M16.

Un cop tinguem aquestes ben adherides i sense deixar espais entre la base i la pilona, caldrà situar la "Base pilones: inferior"a la base de les columnes. Aquesta es pot caragolar o no, si ho fem caldrà anar amb compte que la base potser es desestabilitza a causa dels caps dels caragols sortints; per contra, si no els caragolem, només haurem d'anar amb compte de no moure l'estructura amunt i avall i que se'ns caigui.

[5] Ens encantaria ensenyar imatges o un vídeo del procés de muntatge, però malauradament no s'ha pogut realitzar. A la pàgina web del projecte de la camara rotatòria es pot veure un vídeo del muntatge d'aquesta del qual podem extreure alguns passos.

Model / Component	Origen	Quantitat
17_Tooth_Herringbone_Gear	Adaptada	1
21_Tooth_Herringbone_Gear.stl	Adaptada	1
64_Tooth_Herringbone_Gear	Original	1
Base rotatòria	Adaptada	1
Gear_Side_Bearing_Mount		1
Gear_Side_Support	Adaptada	1
Idle_Side_Bearing_Mount	Original	1
Idle_Side_Support	Original	1
Tilt_U-Mount	Original	1
Pilona	Nova	6
Suport motor: elevació	Nova	1
Parets del motor: elevació	Nova	
Suport motor: Azimut	Nova	1
Base pilones: inferior	Nova	1
Base pilones: superior	Nova	1
Volandera base rotatòria	Nova	1
NEMA 23		2
DM542		2
Gatling 2.4 MK II		1
Arduino		1
Bearing gegant		1
Bearings pels suports verticals		2
Cargols M3 de 12mm		11
Cargols M3 de 16mm		12
Femelles M3		23
Cable coaxial F-M		1

Taula 5.10: Llistat complet del material necessari per muntar l'estructura

Pel que a la base del motor Azimut, ja la col·locarem més tard, quan hàgim de posar els motors.

5.3.3 Encaix de les parts: estàtica i rotatòria

Per encaixar les dues parts cal primer de res situar el *bearing* dins de la part rotatòria, tenint en compte que entre el *bearing* i la base aniria la volandera. A continuació només caldria situar la "Base pilones: superior"a la part interior del *bering*, aquesta part hauria d'encaixar de forma molt suau.

5.3.4 Col·locant components electrònics

Abans de connectar els components electrònics és convenient que fem un parell de coses amb els motors:

- Per començar, retallarem uns pocs centímetres dels cables dels motors. Això no només ho fem perquè aquests són molt llargs i sobren, sinó perquè aquests cables ens seran molt útils després per moure tensions i corrents elevades.
- A continuació trenarem els cables restants d'ambdós motors, per tal que no molestin en fases posteriors del muntatge.
- Finalment, encaixarem els pinyols "17_Tooth_Herringbone_Gearï "21_Tooth_Herringbone_Gear"a les puntes dels dos motors.

Ara ja podem col·locar els dos motors on pertoca. Per tal de col·locar el motor d'elevació cal seguir els passos:

1. Passar els cables trenats pels forats de la base del motor i de la base rotatòria, de forma que quedin entre les pilones.
2. Col·locar el motor amb traça al damunt de la seva base per tal que els engranatges encaixin bé.
3. Un cop el motor és a la seva posició s'hauran de posar les parets per tal d'evitar els moviments laterals un cop en marxa.

El motor encarregat del moviment Azimutal caldrà posar-lo de la forma:

1. Situar el motor dins de la seva base/caixa ("Suport motor: Azimut").
2. Encaixar el pinyó del motor a l'engranatge de la base rotatòria.
3. Caragolar la base del motor a la base de l'estructura allà on considerem fent servir els caragols M12.

Pel que fa al prototip els controladors i l'Arduino poden anar apart de l'antena sense necessitat d'estar acollats a res.

5.3.5 Connexions i esquemàtic dels components

Els passos a seguir per connectar els components són els següents:

1. **Pelem i connectem els cables dels motors** als controladors de la forma:

   - Cable negre connectat al port A+
   - Cable verd connectat al port A-
   - Cable vermell connectat al port B+
   - Cable blau connectat al port B-

   És molt important connectar els cables del color correcte al port correcte. Per a dur a terme aquesta tasca ens podem ajudar d'un tornavís pla petit i d'un pela-cables o unes estisores.

2. **Pelem i connectem els cables sobrants gruixuts**, que hem extret dels cables dels motors, de forma que quedin connectades les entrades de tensió positiva i negativa de cada controlador. També deixarem uns cables pelats sortints per connectar els cables de cocodril de la font.

3. **Connectem els cables "jumpers"des dels ports de l'Arduino** (ports del 5 al 10, ambdós inclosos[6]) **fins als ports dels controladors corresponents**. Cal anar amb compte de connectar-los als terminals negatius (PUL-, ENA- i DIR-), ja que els senyals de control funcionen amb polsos negatius.

4. Finalment, cal **connectar tots els terminals de control positius** (PUL+, ENA+ i DIR+) **entre ells i a la sortida de 5V de l'Arduino**.

5. **Activar o desactivar els interruptors del controlador** a les posicions indicades a la taula 5.11 (l'interruptor 4 es pot deixar a la baixa de moment) i situar l'interruptor de la part superior del controlador a 5V, indicant d'aquesta forma que els senyals de control són de 5V i no pas de 24V.

6. Finalment, **connectarem la nostra font d'alimentació** als cables sortints del controlador. Vigilant molt de connectar les tensions positives i negatives correctament.

[6]No és un error greu cometre una connexió de forma incorrece pel que a als pins de l'arduino, ja que després es pot arreglar amb el firmware de forma molt senizilla

	SW1	SW2	SW3	SW4	SW5	SW6	SW7	SW8
Tria	On	ON	OFF	OFF	On	On	Off	Off

Taula 5.11: Configuració completa del controlador del motor

A la figura 5.19 queden representades totes les connexions dels diferents components electrònics.

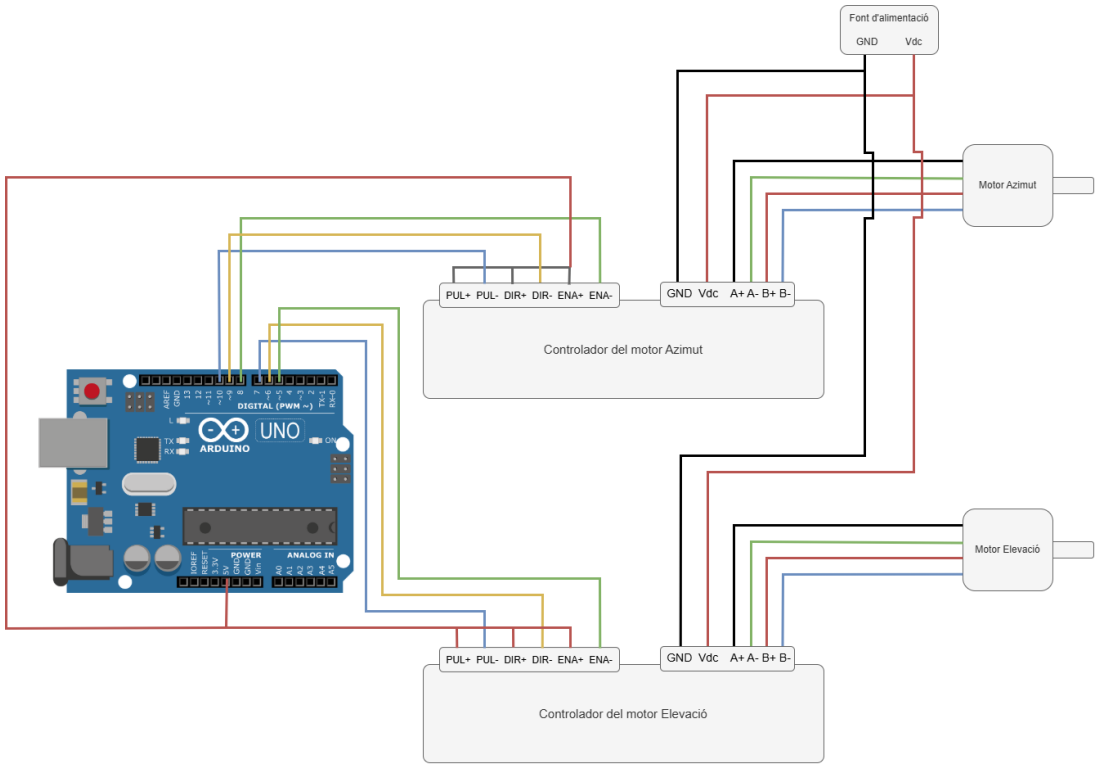

Figura 5.19: Esquemàtic dels diferents components electrònics.

5.4 Programari de control

En aquest apartat tractarem l'última peça del puzle necessària per fer que l'antena sigui capaç de rotar: el *firmware*. Aquest firmware haurà de ser capaç de, d'acord amb les comandes enviades pel *sotware* d'alt nivell, enviar els senyals elèctrics als components *hardware* per tal de fer rotar els motors de forma controlada.

5.4.1 Arquitectura

Aquest programa estarà en execució constant, però romandrà sense fer res fins que llegeixi una instrucció pel port serial. El fet que estiguem en un estat de "no fer res"no vol dir que no enviem senyals pels pins, a la taula **??** podem veure com estan configurats els pins en aquest estat de repòs.

Pin Arduino	**Controlador**	**Pin controlador**	**Estat inicial**
10	Azimut	PUL-	X
9	Azimut	DIR-	X
8	Azimut	ENA-	HIGH
7	Elevació	PUL-	X
6	Elevació	DIR-	X
5	Elevació	ENA-	HIGH

Taula 5.12: Pins de l'Arduino comparats amb els pins dels controladors i el seu estat inicial

Un cop l'usuari enviï una instrucció pel serial i l'Arduino farà el següent

1. Arreglarà la comanda per a tractar-la (eliminar terminacions, passar minúscules a majúscules i altres coses per l'estil).
2. Identificar la comanda (vegeu la taula 5.13).
3. Identificar els graus a girar.
4. Executar la funció pertinent al tipus d'angle que vulguem girar. Aquesta funció (que ara explicarem més fons) s'encarregà d'enviar els senyals pels pins.
5. Esborrar la comanda de la variable temporal i notificar a l'usuari de l'estat d'execució: *DONE*, *UNKNOWN* o *ERROR*.

Instrucció	Definició	Límits
MAHXXX	Moure Azimut en sentit Horari	000 ¡ XXX ¡ 360
MAAXXX	Moure Azimut en sentit Antihorari	000 ¡ XXX ¡ 360
MEUXXX	Moure Elevació cap en amunt (Up)	000¡ XXX ¡ 090
MEDXXX	Moure Elevació cap en avall (Down)	000¡ XXX ¡ 090

Taula 5.13: Llistat de comandes que pot rebre l'Arduino

Paràmetre	Valor Az.	Valor El.	Propòsit
PIN_{PUL}	10	7	Numero del pin PUL
PIN_{DIR}	9	6	Numero del pin DIR
Speed	600	600	Temps de cortesia
StepsPerDegree	152	85	Nombre de passes (steps) per avançar 1^{o}

Taula 5.14: Taula dels paràmetres diferents pels dos casos.

Aquestes funcions úniques de cada angle de rotació tenen un funcionament molt similar:

1. Identifiquem la direcció de gir.
2. Escrivim al pin que indica la direcció HIGH o LOW depenent del cas.
3. Calculem el nombre d'*steps* que s'ha de moure el motor per tal d'avançar el nombre de graus detectats. Cal tindre en compte la configuració del *microsteping* del controlador per això.
4. Per cada step que ha de fer el microcontrolador fem:
   (a) Escriure HIGH al pin que indica que hem de moure'ns un grau (PUL). Notem que el motor no girarà tota l'estona que tinguem aquest pin a l'alça, només girarà un step per pols, és a dir, per cada cop que enviem un HIGH. És per això que cal ressetejar després.
   (b) Esperem un temps de cortesia perquè el controlador tingui temps de llegir el senyal. Podem reduir aquest temps per tal d'augmentar la velocitat.
   (c) Escrivim LOW al pin PUL per tal de reiniciar el pols.
   (d) Ens esperem de nou un temps de cortesia

Les diferències entre les dues funcions són els valors dels diferents paràmetres que es fan servir. A la taula 5.14 podem veure aquests paràmetres, el que signifiquen i els valors per defecte en els dos casos.

A continuació (figures 5.20 i 5.21) podem veure el diagrama de flux d'aquest programa.

5.4.2 Validació

Per validar aquest programa no cal fer servir els codis d'alt nivell desenvolupats en apartats anteriors, podem senzillament fer servir la comunicació serial integrada a la mateixa IDE d'Arduino i llançar les comandes manualment.

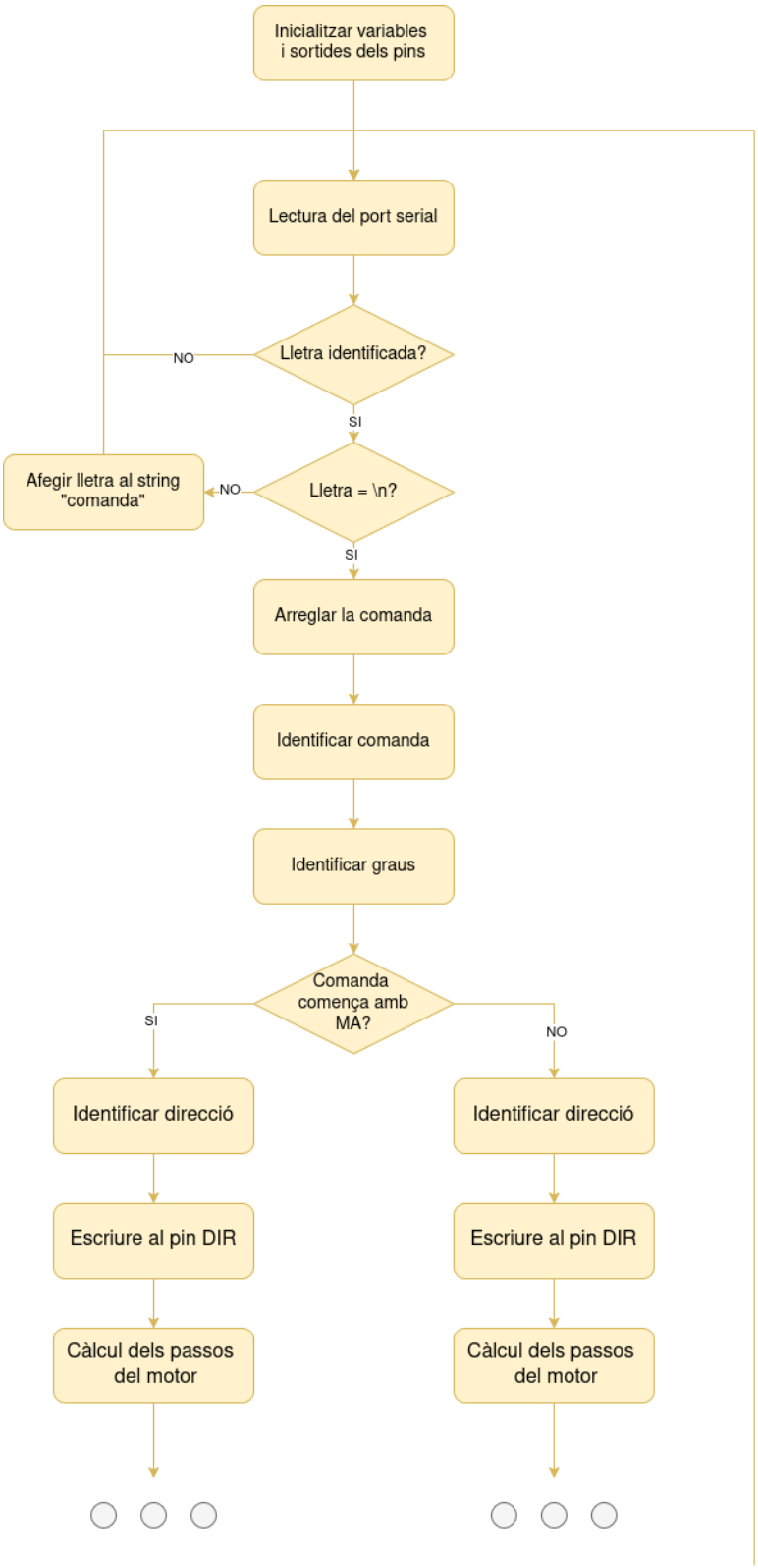

Figura 5.20: Diagrama d'activitats del *firmware* 1

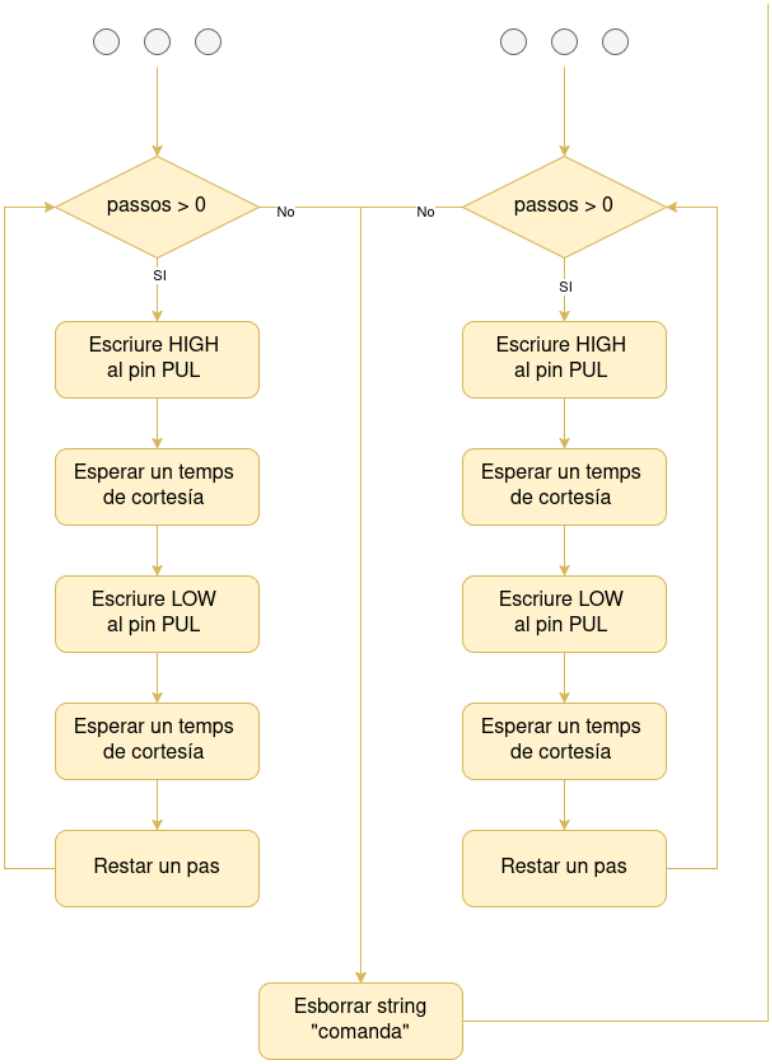

Figura 5.21: Diagrama d'activitats del *firmware* 2

Ens interessarà doncs validar (i hem validat) els següents aspectes:

- Detecció correcta del tipus de grau a girar (Azimut o Elevació).
- Detecció correcta de la direcció de gir.
- Detecció correcta de l'angle de gir.
- Els motors són capaços de girar en la direcció indicada.
- Els graus que avança el motor són els correctes. És a dir que 360 graus és una volta sencera (i exacta) en el cas de l'Azimut i que 90 graus en el cas de l'elevació enfoquen l'antena d'una direcció perpendicular respecte al zenit a una de paral·lela.

Al vídeo que es pot trobar a la figura 5.23 es pot apreciar el prototip de l'antena rotant i a la figura 5.22 una imatge del prototip muntat.

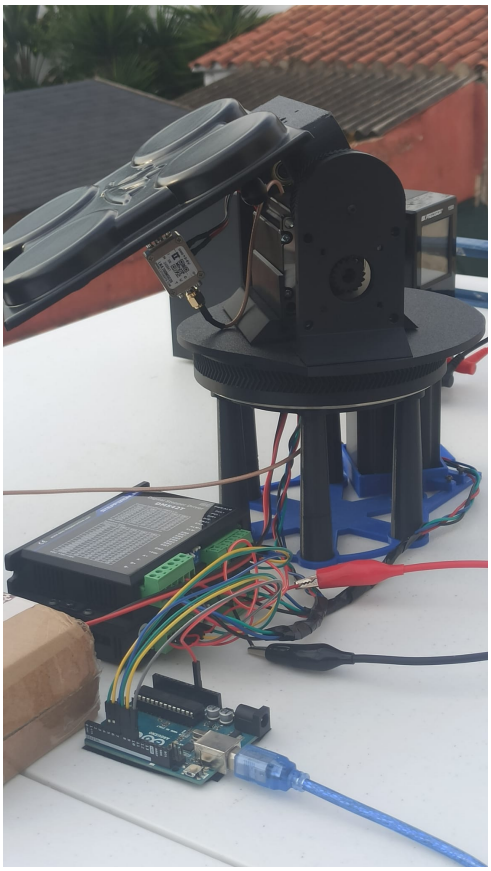

Figura 5.22: Prototip de l'estructura completa de l'antena muntada

Figura 5.23: QR al vídeo de l'antena rotant. Enllaç complet: https://www.youtube.com/shorts/BQt4AMmJ8hA

Capítol 6

Captura i processament del senyal

En aquesta darrera part del desenvolupament del projecte es parteix del fet que ja es disposa de tot el sistema rotatori i seguidor de satèl·lits muntat i funcional, tot i que sense validar.

Serà doncs el propòsit d'aquesta fase validar les següents hipòtesis:

- L'antena és capaç de fer un seguiment correcte dels satèl·lits.
- El sistema és capaç de capturar senyals més prometedors que si mantenim l'antena en una posició constant.

6.1 Procés de captura del senyal

Abans de validar els resultats cal explicar quina metodologia s'ha fet servir per a generar les captures.

Aquesta tasca no s'ha de prendre a la lleugera i ha suposat molts problemes tècnics i d'execució, és per això que també s'ha afegit un apartat tractant de les dificultats i entrebancs que s'han trobat en aquest procés de captures.

6.1.1 *Set-up* per fer captures

El "set-up"(o ...) que hem fet servir per a generar les captures consistia en situar tot el maquinari (base rotatòria, font d'alimentació i l'ordinador de l'estudiant) damunt d'una taula desplegable ubicada al balcó més alt i descobert de la casa de l'estudiant (vegeu la figura **??**.

Aquest set-up pot semblar una mica rudimentari, però presenta el gran avantatge de la mobilitat, ja que, per tal de garantir la integritat de l'antena i evitar deixar-la a la intempèrie,

Figura 6.1: Imatge del setup del procés de captura del senyal

cada cop que no s'estava generant captures es movia a l'interior de la casa. Això fa que, en cas de voler plegar l'estació, senzillament amb l'ajuda d'un tercer es mou la taula sense desmuntar el set-up.

A més tots els problemes d'inestabilitat que ens genera la taula són fàcilment evitables falcant pròpiament la taula o calibrant l'antena a una elevació diferent.

A l'hora d'ubicar tot el maquinari cal ser molt prudents de no generar ni curtcircuits ni impedir els moviments de l'antena per culpa d'un cable que s'ha desplaçat al lloc incorrecte.

El segon factor important a l'hora de muntar l'estació és ubicar el nord, ja que el prototip no disposa cap manera de fer-ho, per això senzillament hem agafat un parell de brúixoles, hem identificat el nord i hem fet que l'antena apunti en direcció aquest (Azimut: 0º és el Nord).

Addicionalment, també s'han fet unes marques en els eixos de coordenades: sud, est i oest; amb l'objecte que, visualment. sigui més fàcil saber la direcció on apunta el satèl·lit.

6.1.2 Programari de captura

Per fer les captures dels senyals s'ha fet servir una SDR, en concret el model BladeRF, aquesta placa es connecta per via d'un cable coaxial a l'antena i per USB a l'ordinador de l'estudiant.

Pel que fa al programari de captura, en un principi es va intentar fer una aproximació més científica, és a dir, quan el satèl·lit passes d'un angle indicat executàvem un script de *Python* (proporcionat per un dels companys del departament) que es connectava a la *BladeRF*, llegia el seu contingut i el guardava en un fitxer binari (.bin) per tal que, a posteriori, nosaltres poguéssim analitzar aquesta captura amb un altre script, aquest cop de Matlab.

Aquesta aproximació, tot i semblar més correcta, presenta els seus problemes:

- **Endarreriment de l'anàlisi dels resultats:** per tal de conèixer si un resultat és correcte o no cal esperar-se a tot el pas d'un satèl·lit (aproximadament 20 min) i aleshores executar el programa que analitza l'arxiu, el qual també triga el seu minut en processar el fitxer.

- **Mida dels fitxers:** Els fitxers de les captures generades per la *bladeRF* són molt pesats, perquè ens fem una idea, si volem capturar 10 minuts per XXX ample de banda i amb un sample-rate de XXX el fitxer pesaria XXX. Això fa que, sovint, haguéssim de fer captures molt curtes i poc representatives.

- **Dificultat afegida de programació:** Aquesta aproximació fa que hàgim de programar dos scripts més en dos llenguatges de programació diferents (Python i Matlab) que, tot i que els scripts fossin proporcionats per un company, han calgut retocs i modificacions que ens impedien fer una lectura correcta del senyal.

- **Error en paràmetres**: Com ja hem dit, si volem capturar una gran amplada de banda cal fer un fitxer de grans dimensions, això fa que sovint, per tal de reduir mides, es reduís l'amplada de banda. Això fa que si en un moment dictem una mala freqüència central (ha passat) senyal es vegi o retallat o fora del tros d'espectre que es pretén analitzar.

Aquests problemes, però, no són "mortals"que diguem, de fet es poden solucionar sigui: donant més memòria a l'ordinador, fent una programació correcta, tenint un coneixement més bo del senyal i estar en possessió d'una mica més de paciència i temps per fer l'anàlisi. Si aquests problemes se solucionessin doncs sí, aquesta seria la forma més correcta d'analitzar el senyal.

Existeix, però una opció més ràpida que es basa en descarregar-nos un programari de lectura en temps real de les dades de l'SDR (en el nostre cas hem fet servir CubicSDR, una aplicació de programari lliure disponible per Linux).

Si bé amb aquests programaris podem generar captures d'una forma molt més ràpida, evitant-nos els fitxers pesats, el temps d'espera i l'anàlisi posterior, també presenta un problema força important, cosa que ens impedeix fer una anàlisi profunda del senyal.

Aquest problema no és excloent tampoc, però limita les formes que tenim de comprovar si el senyal és veritablement d'un satèl·lit. Bàsicament en aquests programes només podem veure l'espectrograma del senyal en temps real durant menys d'un minut, recaient en haver de fer captures de pantalla per tal de demostrar el funcionament.

En el nostre cas, però, per culpa de certes dificultats i entrebancs (a l'apartat següent les tractarem) no ens hem pogut permetre el luxe de solucionar l'opció més lenta, però que ens oferia més dades i hem optat per la segona via d'anàlisi dels resultats, la més ràpida.

6.1.3 Dificultats i entrebancs

El procés de captura del senyal va començar aquest gener (gener del 2026), passades les vacances.

El motiu és un endarreriment en el desenvolupament de les dues primeres fases del projecte que ha fet que ens hàgim d'esperar a després de les vacances de Nadal per començar aquesta darrera fase.

Aquest endarreriment ens ha limitat molt el marge de maniobra a l'hora de fer les captures, sobretot si tenim en compte que encara quedaven pendents aspectes del treball com la redacció d'aquest informe.

A aquest poc marge de maniobra temporal se li ha d'afegir la mala sort amb la meteorologia, ja que, si fem memòria, recordarem que aquest gener ha sigut sorprenentment un mes molt plujós i amb moltes ventades fortes. Això ens ha limitat i condicionat moltíssim les estones de captures i ha sigut un dels principals motius que hem decidit canviar d'aproximació i fer servir programes de captura IRT en comptes de fer la parafernàlia dels fitxers .bin.

A més d'aquests inconvenients també se li ha d'afegir el fet que aquest treball el duu a terme un estudiant que, a part de tindre molts exàmens i entregues el gener, no té experiència amb la captura del senyal amb SDRs, tasca que presenta un cert nivell de dificultat inicial.

També hi ha hagut altres entrebancs, com a errors en la propagació dels satèl·lits, amb l'*step* dels motors i altres, però aquests problemes entren dins dels riscos esperats i no necessitem excusar-nos.

6.2 Validant el seguiment satel·litari

Abans de fer captures cal validar el correcte seguiment dels satèl·lits, això s'aconsegueix de la següent forma:

1. Muntem el set-up i orientem l'antena al nord.
2. Busquem un satèl·lit que hagi de passar en una estona fent ús del programari que hem creat.
3. Comprovem que aquest satèl·lit hagi de passar per sobre nostre a l'hora predita fent servir l'eina [Glo24].
4. Executem el programari que inicialitza els moviments de captura d'aquest satèl·lit.
5. Comprovem que l'antena faci un seguiment orbital similar al que podem trobar, de nou, a l'eina [Glo24]. Cal tindre en compte els angles dels moviments i el temps en què aquests

es fan.

Si aconseguim que l'antena executi el moviment de la captura per les següents menes de satèl·lits ens podem donar per satisfets.

- Un satèl·lit que hagi de passar en uns minuts: aquí validem:
  - Temps d'espera
  - Càlcul d'azimut d'entrada
  - Moviments de seguiment orbital (temps i posició)
- Un satèl·lit ja en el camp de visió: aquí validem:
  - Moviment d'enfocament al satèl·lit inicial (azimut i elevació)
  - Moviments de seguiment orbital (temps i posició) restants.

Tot i que no poder ensenyar el funcionament amb imatges, sí que podem dir que tots aquests casos s'han complert i que el moviment de l'antena s'ha validat satisfactòriament.

6.3 Validant la captura dels senyals

Per tal de dur a terme les captures cal seguir els següents passos:

1. **Muntem el set-up** correctament i orientem l'antena al nord.
2. **Obrim el programari de captura de senyals** de l'SDR en temps real i mirem que la nostra BladeRF estigui ben connectada i a la freqüència que toca.
3. **Busquem un satèl·lit** que hagi de passar en una estona fent ús del programari que hem creat.
4. Executem el programari que inicialitza els moviments de **seguiment d'aquest satèl·lit**.

Si hem fet això correctament i després de validar que els moviments de seguiment satel·litaris són correctes (evidentment) el que cal fer ara és mirar detingudament el progrés del senyal rebut al programa.

El que ens interessarà veure de les captures de senyal són les variacions de potència espectral en el domini del temps i la freqüència. Com s'observa figura 6.2 [1], el senyal de Globalstar es

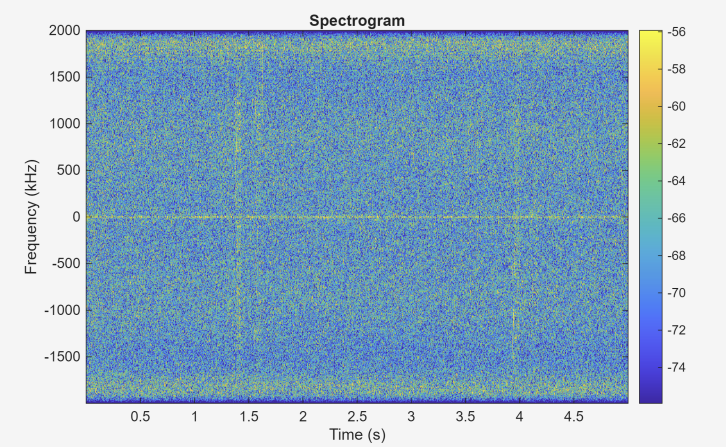

Figura 6.2: Espectrograma de referència d'una captura d'un satèl·lit de globalstar.

caracteritza per ocupar una banda ampla (en aquest cas, uns 4 MHz de visualització) i presenta una densitat de potència molt baixa, gairebé confosa amb el soroll de fons.

Les petites traces verticals més clares que apareixen puntualment a la imatge (instants 1.5s i 4s) indiquen ràfegues d'activitat o trànsit de dades del satèl·lit, confirmant que la captura s'està realitzant en la freqüència correcta malgrat la baixa relació senyal-soroll (SNR).

A continuació mostrarem un seguit d'imatges d'una de les captures realitzades amb el prototip d'antena rotatòria i on es demostra una correcta captura del senyal.

6.3.1 Elevació de 10 a 20 graus

En aquesta etapa inicial de la passada, amb una elevació compresa entre els 10 i els 20 graus, la captura mostra un entorn radioelèctric complex.

El primer que es pot observar a la figura 6.3 és que, en aquesta banda de freqüències (entorn dels 2483,5 - 2500 MHz), hi ha una presència notable d'interferències.

Això és degut a la proximitat de la banda ISM de 2,4 GHz, utilitzada per protocols com el Wi-Fi, el Bluetooth o, fins i tot, per les emissions residuals dels forns microones. Les emissions de Wi-Fi dels canals adjacents actuen com la principal interferència, essent especialment visibles en el marge esquerre de l'espectrograma, on el nivell de soroll base (noise floor) es veu clarament incrementat.

A més de les interferències de la banda ISM, cal considerar la dificultat inherent a la baixa elevació. Un angle baix implica no només haver de travessar una capa més gruixuda de l'atmosfera terrestre (augmentant l'atenuació i reduint el guany efectiu), sinó que, en apuntar més a

[1]Imatge proporcionada per un company del departament

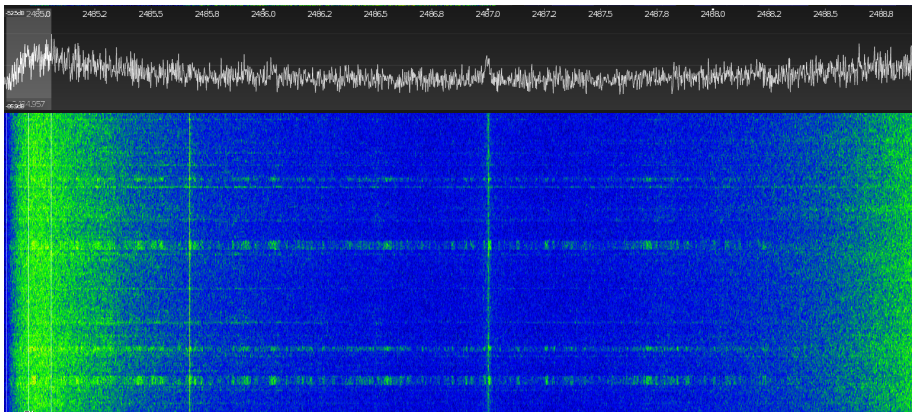

Figura 6.3: Captura del senyal per a una elevació de 10 a 20 graus

prop de l'horitzó, l'antena capta de forma més directa les interferències terrestres.

Malgrat la forta presència de soroll i el desavantatge geomètric, el prototip d'antena rotatòria és capaç de **detectar** el senyal amb èxit. S'aprecien **traces d'energia horitzontals** que trenquen la uniformitat del soroll, les quals corresponen als canals CDMA característics de la constel·lació Globalstar.

6.3.2 Elevació de 30 a 40 graus

A mesura que el satèl·lit guanya altura i se situa en una franja d'elevació d'entre 30 i 40 graus, s'observa una millora substancial en la qualitat de la recepció. Com es pot apreciar a la figura 6.4, l'increment de l'angle d'elevació té un efecte directe i positiu en la claredat de la captura.

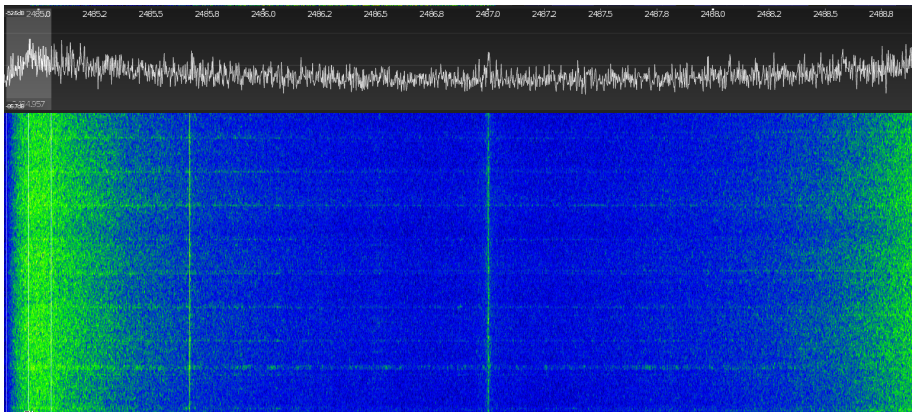

Figura 6.4: Captura del senyal per a una elevació de 30 a 40 graus

En aquesta fase, la distància relativa entre el satèl·lit i l'antena ha disminuït i el senyal ha de travessar una secció menor d'atmosfera, fet que redueix l'atenuació per trajectòria. Això es tradueix en una millora de la relació senyal-soroll: les traces horitzontals que representen els

canals de dades de Globalstar es tornen molt més definides i brillants (amb tons més verdosos i groguencs) respecte al blau del soroll de fons.

Un aspecte clau a destacar en comparar aquesta captura amb l'anterior és el següent:

- **Menor interferència terrestre**: En apuntar cap a una zona més alta del cel, l'antena es "desconnecta"parcialment del soroll ambiental provinent de l'horitzó. Tot i que la interferència del Wi-Fi al marge esquerre (2485 MHz) encara és present, el senyal del satèl·lit ja no lluita tan estretament contra el terra de soroll.

6.3.3 Elevació de 50 a 60 graus

En aquesta franja d'elevació, el satèl·lit continua el seu ascens cap al zenit, permetent una recepció cada cop més neta.

Com es pot observar a la figura 6.5, en arribar a aquest punt s'ha realitzat un lleuger desplaçament en la freqüència de sintonització del receptor SDR per tal de veure millor la captura. Aquest ajustament tècnic és clarament visible si ens fixem en la línia vertical de referència (un spur o interferència interna del dispositiu), la qual s'ha mogut de posició respecte a les captures anteriors.

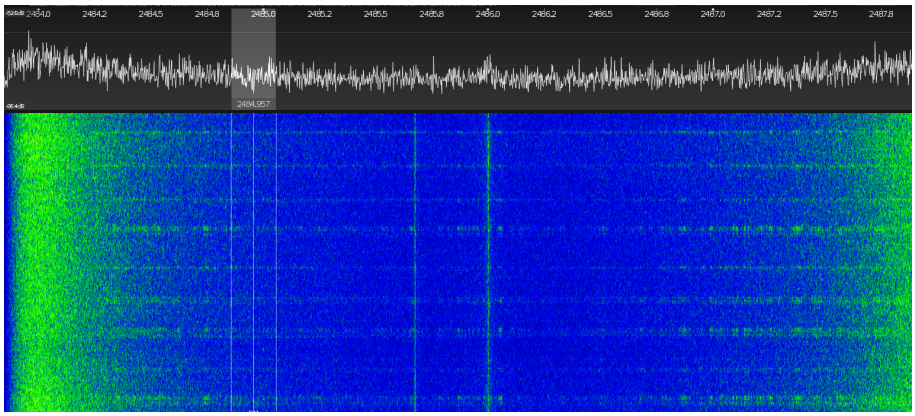

Figura 6.5: Captura del senyal per a una elevació de 50 a 60 graus

En haver augmentat l'angle d'elevació podem veure com hi ha:

- **Reducció de l'atenuació:** El senyal ha de travessar menys densitat atmosfèrica, millorant la claredat de les traces CDMA horitzontals.
- **Reducció d'interferències:** L'antena, en apuntar més amunt, comença a ignorar amb més eficàcia el soroll provinent de l'horitzó (com les xarxes Wi-Fi que es veien més intenses en angles baixos).

Tot i que encara no hem assolit el punt de màxima elevació de la passada, la definició actual de les ràfegues d'energia ja permet validar que el prototip d'antena rotatòria manté un seguiment precís, captant el senyal de banda ampla de forma robusta abans d'arribar a la culminació de l'òrbita.

6.3.4 Elevació de 70 a 80 graus

Arribant a la culminació de la passada, amb una elevació d'entre 70 i 80 graus, el senyal assoleix el seu punt àlgid de potència i claredat.

Com es pot observar a la figura 6.6, la recepció en aquest angle és la més nítida de tota la sèrie, ja que ens trobem en el moment de mínima distància relativa entre el satèl·lit i el prototip.

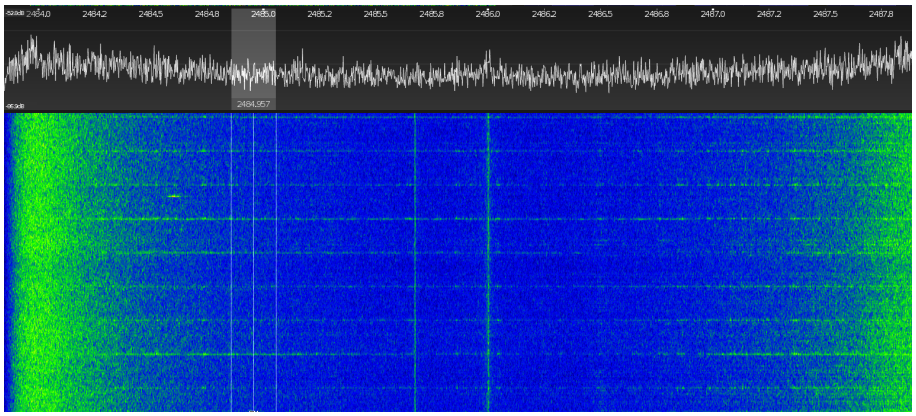

Figura 6.6: Captura del senyal per a una elevació d'uns 70-80 graus

En aquesta captura es fan evidents (de nou) els següents factors tècnics:

- **Mínima atenuació atmosfèrica**: El senyal recorre el camí més curt possible a través de l'atmosfera, el que resulta en unes traces horitzontals extremadament definides que sobresurten clarament del soroll de fons.

- **Interferències terrestres mínimes:** En apuntar pràcticament cap al zenit, l'antena ignora gairebé per complet el soroll provinent de l'horitzó i les xarxes Wi-Fi adjacents.

Aquesta imatge representa l'èxit del projecte, demostrant que el disseny de l'antena i el control de seguiment permeten capturar el senyal de la constel·lació Globalstar amb una qualitat òptima per al seu posterior processament.

6.3.5 Parada de la captura i conseqüències

Finalment, per si encara quedaven dubtes sobre si realment el que s'estava perseguint era un satèl·lit o una interferència d'origen terrestre, s'ha aturat el sistema de seguiment de l'antena un cop superat el punt de màxima elevació.

Després d'esperar poc més d'un minut, permetent així que el satèl·lit s'allunyi del camp de visió de l'antena, s'ha realitzat la captura que es mostra a la figura 6.7[2].

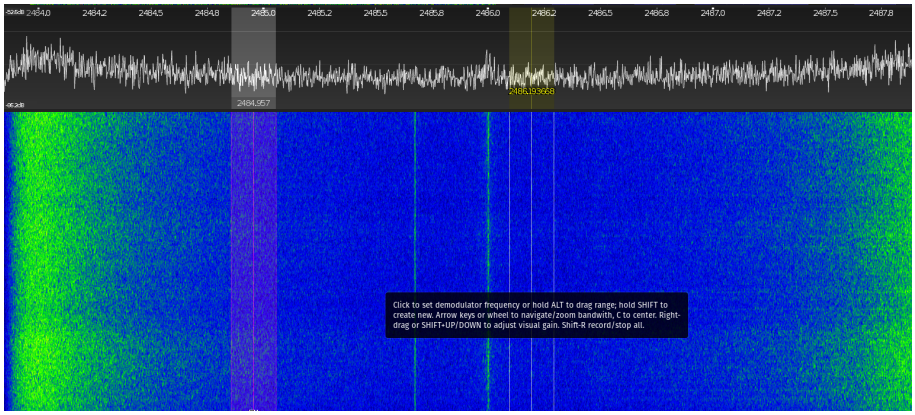

Figura 6.7: Captura del senyal amb l'antena desenfocada

Com es pot observar a la imatge, el canvi en l'espectrograma és radical. Mentre que la interferència del Wi-Fi al marge esquerre i els *spurs* verticals es mantenen immutables (confirmant que són fonts estàtiques o internes), les traces horitzontals de dades han desaparegut completament.

Aquesta absència d'energia en les freqüències de Globalstar demostra dues coses fonamentals per al projecte:

- **Origen del senyal**: El senyal captat anteriorment provenia efectivament d'un objecte en moviment ràpid (òrbita LEO) i no d'una emissió local.
- **Directivitat i seguiment**: L'antena del prototip té una directivitat prou alta com per deixar de percebre el senyal si l'apuntament no és precís, validant així la necessitat i l'èxit del mecanisme de rotació i seguiment desenvolupat.

[2]Lamentem que aparegui una imatge tan bruta, amb el bloc de text i les línies vermelles en mig, però es pot comprovar de totes formes que el senyal ja no apareix.

Capítol 7

Conclusions

Aquest projecte conclou molt satisfactòriament, havent assolit amb èxit els objectius plantejats inicialment, culminant en el desenvolupament d'un prototip funcional i robust per al seguiment de satèl·lits en òrbita terrestre baixa (LEO).

De manera específica, els èxits del projecte, es resumeixen en els següents punts (extrets dels objectius del treball):

- **Desenvolupament del programari de control i propagació:** S'ha implementat una eina integral que combina tant algorismes de propagació d'òrbites com eines de comunicació amb sistemes encastats amb una interfície visual intuïtiva que permet als usuaris no coneixedors del programa fer servir amb facilitat els recursos.
- **Disseny i construcció de l'estructura mecànica:** S'ha creat un sistema de posicionament rotatori (azimut i elevació) capaç d'orientar l'antena cap a qualsevol punt del cel visible. Aquesta estructura ha demostrat tenir la rigidesa i la precisió necessàries per seguir satèl·lits LEO en temps real.
- **Validació experimental mitjançant radiofreqüència:** La funcionalitat del conjunt (maquinari, programari i firmware) ha quedat demostrada a través de les captures del senyal d'un dels satèl·lits de la constel·lació Globalstar. L'anàlisi de les signatures espectrals i la desaparició del senyal en cessar el seguiment han servit com a prova irrefutable de la precisió de l'apuntament i de l'eficàcia del receptor SDR integrat.

En definitiva, els resultats obtinguts i documentats en els capítols anteriors confirmen que el prototip és una eina viable per a l'estudi de comunicacions satel·litàries i que té el potencial de facilitar molt la feina de captura del senyal als investigadors i equips que necessitin fer captures de senyals LEO de qualitat.

7.1 Aspectes de millora per a futures versions del prototip

Abans de tancar el projecte és convenient tractar tots aquells aspectes de millora de cara a la implementació final de l'antena rotatòria o per futures versions del prototip. Aquestes millores es poden resumir en:

- **Optimització de la mecànica:** L'estructura actual encara presenta uns quants aspectes de millora. De cara a un prototip final caldria fer el següent:
  - **Afegir un *planetary gearbox*:** de cara a augmentar la capacitat de càrrega del prototip.
  - **Millorar l'encaix dels engranatges 3D** o eliminar la necessitat per tal d'evitar problemes de *backlash* (joc mecànic) i el salt de passos.
- **Calibratge automàtic:** Convindria afegir sensors que ens permetin fer un calibratge no forçat i automàtic al nord.
- **Robustesa ambiental:** Per a una instal·lació permanent, caldria evolucionar cap a un disseny amb materials resistents a la degradació per raigs UV, a les pluges, càrregues de vent i altres fenòmens meteorològics.
- **Ampliar el programari i la interfície**: Noves versions de l'antena rotatòria podrien incloure:
  - Una eina de processament de captures
  - Una eina de programació de captures per tal de facilitar que molts investigadors puguin fer servir l'antena sense molestar-se.
  - Una eina de captura permanent que permeti a l'antena saltar d'un satèl·lit a un altre.

Aquestes millores demostren que, tot i que el prototip ha funcionat satisfactòriament i que el projecte ha sigut un èxit, això només és una primera part d'un projecte molt més gran.

Finalment, vull destacar que la realització d'aquest Treball Fi de Grau m'ha permès aprendre una gran varietat de conceptes tècnics del món dels satèl·lits com el funcionament d'aquests, els tipus, les diferents constel·lacions i la propagació d'òrbites. A m'ha permès desenvolupar un projecte de sistemes encastats complet, des de la primera interfície visual en format web fins a la darrera connexió electrònica dels motors, passant per tot tipus de programes en diferents llenguatges com C, python o Java Script.

7.2 Agraïments

Vull agrair primer de tot al meu tutor José A. López Salcedo per la seva dedicació, paciència i per haver-me ofert fer aquest projecte tant engrescador i del que he après tant.

També m'agradaria donar les gràcies a en Toni Reus i a en Marc Fernández pel seu ajut i mentoria a l'hora de desenvolupar l'estructura i realitzar les captures.

Capítol 8

Annex

8.1 Codis C del projecte

8.1.1 Generació de la llista de SV

```c
#include "Satellite_data/utils.h"
#include <ctype.h>
#include <strings.h>
#include "Satellite_data/download_and_parse_TLE_functions.h"
#include "Satellite_data/coordinates_translation_functions.h"

// Helper functions
void trim_whitespace(char *str) {
    // Trim leading whitespace
    char *start = str;
    while (*start && isspace((unsigned char)*start)) start++;
    if (start != str) memmove(str, start, strlen(start) + 1);

    // Trim trailing whitespace
    char *end = str + strlen(str) - 1;
    while (end >= str && isspace((unsigned char)*end)) *end-- = '\0';
}

void printTime_t(char* textBefore, time_t time, bool newline) {
    char time_str[50];
    struct tm *time_tm = gmtime(&time);  // Use gmtime() for UTC display
    time_tm->tm_hour += 1;
    mktime(time_tm); // Normalize the time structure
    strftime(time_str, sizeof(time_str), "%Y-%m-%d %H:%M:%S", time_tm);
    printf("%s %s%s", textBefore, time_str, newline ? "\n" : "");
}

```

```
28 // Define the observer's location (Sant Vicenc dels Horts, Catalonia, Spain)
29 GeodeticPosition observer = {
30     .latitude_deg = 41.393552,
31     .longitude_deg = 2.009787,
32     .altitude_km = 0.022 // 22 meters
33 };
34
35 int main(int argc, char *argv[]) {
36     const char* url_starlink = "https://celestrak.org/NORAD/elements/gp.php?
           GROUP=starlink&FORMAT=tle";
37     const char* url_orbcomm = "https://celestrak.org/NORAD/elements/gp.php?GROUP
           =orbcomm&FORMAT=tle";
38     const char* url_iridium = "https://celestrak.org/NORAD/elements/gp.php?GROUP
           =iridium&FORMAT=tle";
39     const char* url_globalstar = "https://celestrak.org/NORAD/elements/gp.php?
           GROUP=globalstar&FORMAT=tle";
40
41     int arg_offset = 0;
42     const char *selected_url = url_starlink;
43     char constellation = 'S';
44
45     if (argc > 1) {
46         if (isalpha((unsigned char)argv[1][0])) {
47             arg_offset = 1;
48             char c = toupper((unsigned char)argv[1][0]);
49             constellation = c;
50             if (c == 'S') selected_url = url_starlink;
51             else if (c == 'O') selected_url = url_orbcomm;
52             else if (c == 'I') selected_url = url_iridium;
53             else if (c == 'G') selected_url = url_globalstar;
54             else {
55                 if (strncasecmp(argv[1], "starlink", 8) == 0) selected_url =
                       url_starlink;
56                 else if (strncasecmp(argv[1], "orbcomm", 7) == 0) selected_url =
                        url_orbcomm;
57                 else if (strncasecmp(argv[1], "iridium", 7) == 0) selected_url =
                        url_iridium;
58                 else if (strncasecmp(argv[1], "globalstar", 10) == 0)
                       selected_url = url_globalstar;
59                 else {
60                     fprintf(stderr, "Unknown constellation '%s', defaulting to
                           Starlink.\n", argv[1]);
61                     selected_url = url_starlink;
62                     constellation = 'S';
63                 }
64             }
65         }
```

```
66      }
67
68      char* tle_data_string = download_tles(selected_url);
69      if (!tle_data_string) {
70          fprintf(stderr, "Failed to download TLE data. Exiting.\n");
71          return 1;
72      }
73
74      ownTLE* satellites = NULL;
75      int num_satellites = parse_tle_string(tle_data_string, &satellites);
76      free(tle_data_string);
77
78      if (num_satellites <= 0) {
79          printf("No satellites to process.\n");
80      } else {
81          double observer_ecef[3];
82          geodetic_to_ecef(&observer, &observer_ecef[0], &observer_ecef[1], &
                observer_ecef[2]);
83          time_t utc_time_unix = time(NULL);
84
85          Pass* passes = NULL;
86          int pass_count = 0;
87
88          for(int i = 0; i < num_satellites; ++i) {
89              char opsmode = 'i';
90              if (!sgp4init(opsmode, &satellites[i].tle.rec)) {
91                  fprintf(stderr, "SGP4 initialization failed.\n");
92                  free(satellites);
93                  return 1;
94              }
95
96              bool is_in_pass = false;
97              time_t pass_start_unix = 0;
98              double max_elevation_in_pass = 0.0;
99              double first_azimuth_in_pass = 0.0;
100             double current_jd = unix_to_julian(utc_time_unix);
101             double jd_epoch = satellites[i].tle.rec.jdsatepoch + satellites[i].
                    tle.rec.jdsatepochF;
102
103             for (double t_min = 0; t_min < 60; t_min += 0.25) {
104                 double tsince = (current_jd - jd_epoch) * 1440.0 + t_min;
105                 double current_julian_date = current_jd + (t_min / 1440.0);
106                 double sat_pos_eci[3], sat_vel_eci[3];
107
108                 int error_code = sgp4(&satellites[i].tle.rec, tsince,
                        sat_pos_eci, sat_vel_eci);
109                 if (error_code >= 0) {
```

```
110                     double sat_pos_ecef[3];
111                     eci_to_ecef(sat_pos_eci, current_julian_date, sat_pos_ecef);
112                     double azimuth, elevation, range;
113                     ecef_to_topo(observer_ecef, sat_pos_ecef, &observer, &
                            azimuth, &elevation, &range);
114
115                     if (elevation > 0.0) {
116                         if (!is_in_pass) {
117                             is_in_pass = true;
118                             pass_start_unix = utc_time_unix + (time_t)(t_min *
                                    60);
119                             first_azimuth_in_pass = azimuth;
120                             max_elevation_in_pass = elevation;
121                         } else if (elevation > max_elevation_in_pass) {
122                             max_elevation_in_pass = elevation;
123                         }
124                     } else if (is_in_pass) {
125                         is_in_pass = false;
126                         time_t pass_end_unix = utc_time_unix + (time_t)(t_min *
                                60);
127                         passes = realloc(passes, (pass_count + 1) * sizeof(Pass)
                                );
128                         strcpy(passes[pass_count].name, satellites[i].name);
129                         trim_whitespace(passes[pass_count].name);
130                         passes[pass_count].start_unix = pass_start_unix;
131                         passes[pass_count].end_unix = pass_end_unix;
132                         passes[pass_count].max_elevation = max_elevation_in_pass
                                ;
133                         passes[pass_count].first_azimuth = first_azimuth_in_pass
                                ;
134                         pass_count++;
135                     }
136                 }
137             }
138         }
139         // ... (Codi de filtratge i sortida omes per brevetat en l'exemple, pero
                inclos al projecte final)
140     }
141     free(satellites);
142     return 0;
143 }
```

Listing 8.1: Programa per a la generació de dades de visibilitat de satèl·lits (SGP4)

8.1.2 Gestió de la captura d'un satèl·lit concret

```c
#include "Satellite_data/utils.h"
#include "Satellite_data/download_and_parse_TLE_functions.h"
#include "Satellite_data/coordinates_translation_functions.h"
#include "Arduino_stuff/arduino_communications.h"
#include "AntennaMovementInstructions/command_handler_functions.h"
#include "AntennaMovementInstructions/JSON_related_functions.h"

// URLs per a les diferents constel·lacions
const char *url_starlink = "https://celestrak.org/NORAD/elements/gp.php?GROUP=starlink&FORMAT=tle";
const char *url_orbcomm = "https://celestrak.org/NORAD/elements/gp.php?GROUP=orbcomm&FORMAT=tle";

// Funcio principal de calcul d'orbita i generacio de comandes
int calculateOrbitAndGenerateCommands(char* satellite_name, char* constellation_name, char ***arduinoCommands_ptr){
    const char *url = NULL;
    // Seleccio de constel·lacio...

    // 1. Descarrega i parseig de TLE
    char* tle_data_string = download_tles(url);
    ownTLE* tles = NULL;
    int tle_count = parse_tle_string(tle_data_string, &tles);

    // 2. Propagacio SGP4 per als proxims 30 minuts
    for (double t_min = 0.0; t_min < 30.0; t_min += 0.0833) {
        time_t step_time_unix = current_time_unix + (time_t)(t_min * 60.0);
        double jd = unix_to_julian(step_time_unix);
        double tsince = (jd - jd_epoch) * 1440.0;

        sgp4(&selected_tle->tle.rec, tsince, sat_pos_eci, sat_vel_eci);
        eci_to_ecef(sat_pos_eci, jd, sat_pos_ecef);
        ecef_to_topo(observer_ecef, sat_pos_ecef, &observer, &azimuth_deg, &elevation_deg, &range_km);

        if (elevation_deg > 0.0) {
            // Generacio de comandes relatives (MAH, MAA, MEU, MED)
            // S'emmagatzemen en un array dinamic amb marca de temps
        }
    }
    return 0;
}

int main(int argc, char *argv[]) {
    // 1. Inicialitzacio i Calibratge
    handle_calibrate_action(json_path, curAz, curEl);

    // 2. Calcul d'execucio de passades
    calculateOrbitAndGenerateCommands(satellite_name, constellation_name, &arduinoCommands);

    // 3. Bucle de seguiment en temps real
    for (int idx = 0; arduinoCommands[idx] != NULL; idx += 2) {
        const char *cmd = arduinoCommands[idx];
        const char *ts = arduinoCommands[idx + 1];

        // Espera activa fins l'instant de temps de la comanda
        while (strcmp(now_str, ts) < 0) { usleep(100000); }

        if (strcmp(cmd, "C") == 0) {
            // Executa script de captura Python en un proc s fill (fork)
            if (fork() == 0) execlp("python3", "python3", script_path, "30", NULL);
        } else {
            // Envia moviment a l'Arduino i actualitza JSON
            send_command_and_update_json(cmd, json_path, currentAz, currentEl);
        }
    }

    // 4. Retorn a l'origen
    handle_origin_action(json2, json_path);
    return 0;
}
```

Listing 8.2: Nucli de control en temps real: Propagació orbital i gestió de moviments de l'antena

8.1.3 Gestió del moviment lliure de l'antena

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <unistd.h>
#include <fcntl.h>
#include <ctype.h>

#include "Arduino_stuff/arduino_communications.h"
#include "AntennaMovementInstructions/JSON_related_functions.h"
#include "AntennaMovementInstructions/utils.h"
#include "AntennaMovementInstructions/command_handler_functions.h"

#define AZ_MIN -360
#define AZ_MAX 360
#define EL_MIN 0
#define EL_MAX 90

/* Path al fitxer d'estat de l'antena */
#define JSON_PATH "./comm_files/antennaStatus.json"

int main(int argc, char **argv) {
    // 1: Tractament d'arguments
    if (argc < 2 || argc > 3) {
        fprintf(stderr, "Usage: %s <action (T|C|B|O|F|U|D|L|R)> [step]\n", argv
            [0]);
        return 2;
    }

    char action = (char)toupper((unsigned char)argv[1][0]);
    if (!(action == 'T' || action == 'C' || action == 'B' || action == 'O' ||
          action == 'F' || action == 'U' || action == 'D' || action == 'L' ||
              action == 'R')) {
        fprintf(stderr, "Unknown action '%c'\n", action);
        return 4;
    }

    int step = 1;
    if (argc == 3) {
        step = atoi(argv[2]);
        if (step < 1 || step > 360) {
            fprintf(stderr, "Invalid step value: %d (1-360)\n", step);
            return 3;
        }
    }
```

```
44
45      // 2: Lectura de l'estat actual (JSON)
46      char json_path[1024];
47      get_absolute_json_path(argv[0], json_path, sizeof(json_path));
48
49      char *json = read_file_alloc(json_path);
50      if (!json) {
51          fprintf(stderr, "Failed to read JSON file '%s'\n", json_path);
52          return 5;
53      }
54
55      int curAz = 0, curEl = 0;
56      parse_json_int(json, "currentAz", &curAz);
57      parse_json_int(json, "currentEl", &curEl);
58
59      // 3: Execuci  de l'acci  mitjan ant el command_handler
60      int result = 0;
61      if (action == 'O') {
62          result = handle_origin_action(json, json_path);
63      } else if (action == 'B') {
64          result = handle_back_action(json, json_path);
65      } else if (action == 'T') {
66          result = handle_test_action(json_path, curAz, curEl);
67      } else if (action == 'C') {
68          result = handle_calibrate_action(json_path, curAz, curEl);
69      } else if (action == 'F') {
70          result = handle_force_action(json_path, curAz, curEl);
71      } else if (action == 'L' || action == 'R' || action == 'U' || action == 'D')
            {
72          result = handle_movement_action(action, step, json, json_path, curAz,
              curEl);
73      }
74
75      free(json);
76      return result;
77  }
```

Listing 8.3: Controlador de l'antena: Interfície entre dades JSON i comandaments Arduino

8.1.4 Codis útils per a dades satel·litàries

```
1  // Header guard
2  #ifndef UTILS_H
3  #define UTILS_H
4
5  // Minimum required libraries
```

```c
#include <time.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <string.h>
#include <curl/curl.h>
#include <ctype.h>

// Link self created libraries
#include "../SGP4_library/libSGP4.h"

// Constants for coordinate transformations
#define PI 3.14159265358979323846
#define DEGREES_TO_RADIANS (PI / 180.0)
#define MINUTES_PER_DAY 1440.0
#define SECONDS_PER_DAY 86400.0

// WGS84 Ellipsoid Parameters
#define EARTH_RADIUS_WGS84 6378.137 // km (semi-major axis)
#define EARTH_FLATTENING_NORMAAIZED (1.0 / 298.257223563) // f
#define EARTH_ECCENTRICITY_SQUARED 0.00669437999014132 // e2 = 2f - f2

// Struct to hold the TLE data for a single satellite
typedef struct {
    char name[25];
    TLE tle; // Include the TLE struct from libSGP4.h
} ownTLE;

// Struct to hold observer's geodetic location
typedef struct {
    double latitude_deg;
    double longitude_deg;
    double altitude_km;
} GeodeticPosition;

// Struct to store a single visibility pass
typedef struct {
    char name[50];
    time_t start_unix;
    time_t end_unix;
    double max_elevation;
    double first_azimuth;
} Pass;

#endif // UTILS_H
```

Listing 8.4: Fitxer de capçalera utils.h: Definicions de constants i estructures de dades

```c
// Header guard
#ifndef DOWNLOAD_AND_PARSE_TLE_FUNCTIONS_H
#define DOWNLOAD_AND_PARSE_TLE_FUNCTIONS_H

// Function prototypes
char* download_tles(const char* url);
int parse_tle_string(const char* data, ownTLE** tles);

// Struct to store data received from the cURL callback
typedef struct {
    char *data;
    size_t size;
} MemoryStruct;

#endif // DOWNLOAD_AND_PARSE_TLE_FUNCTIONS_H
```

Listing 8.5: Fitxer de capçalera per a la descàrrega i el processament de dades TLE

```c
#include "utils.h"
#include "download_and_parse_TLE_functions.h"

// Callback function for cURL to write received data into memory
static size_t WriteMemoryCallback(void *contents, size_t size, size_t nmemb,
    void *userp) {
    size_t realsize = size * nmemb;
    MemoryStruct *mem = (MemoryStruct *)userp;

    char *ptr = realloc(mem->data, mem->size + realsize + 1);
    if (!ptr) {
        printf("not enough memory (realloc returned NULL)\n");
        return 0;
    }

    mem->data = ptr;
    memcpy(&(mem->data[mem->size]), contents, realsize);
    mem->size += realsize;
    mem->data[mem->size] = 0;

    return realsize;
}

// Function to download TLE data from a URL
char* download_tles(const char* url) {
    CURL *curl_handle;
    CURLcode res;
    MemoryStruct chunk;

```

```
29      chunk.data = malloc(1);  // Init with 1 byte, grown by realloc
30      chunk.size = 0;
31
32      curl_global_init(CURL_GLOBAL_ALL);
33      curl_handle = curl_easy_init();
34
35      if(curl_handle) {
36          curl_easy_setopt(curl_handle, CURLOPT_URL, url);
37          curl_easy_setopt(curl_handle, CURLOPT_WRITEFUNCTION, WriteMemoryCallback
                );
38          curl_easy_setopt(curl_handle, CURLOPT_WRITEDATA, (void *)&chunk);
39          curl_easy_setopt(curl_handle, CURLOPT_USERAGENT, "libcurl-agent/1.0");
40
41          res = curl_easy_perform(curl_handle);
42
43          if(res != CURLE_OK) {
44              fprintf(stderr, "curl_easy_perform() failed: %s\n",
                    curl_easy_strerror(res));
45              free(chunk.data);
46              chunk.data = NULL;
47          }
48          curl_easy_cleanup(curl_handle);
49      }
50      curl_global_cleanup();
51      return chunk.data;
52  }
53
54  // Function to parse the TLE data and populate an array of TLE structs
55  int parse_tle_string(const char* data, ownTLE** tles) {
56      if (!data) return 0;
57
58      int count = 0;
59      char* data_copy = strdup(data);
60      if (!data_copy) return 0;
61
62      char* context;
63      char* line = strtok_r(data_copy, "\n", &context);
64
65      while (line) {
66          char* name = line;
67          char* line1 = strtok_r(NULL, "\n", &context);
68          char* line2 = strtok_r(NULL, "\n", &context);
69
70          if (!name || !line1 || !line2) {
71              fprintf(stderr, "Incomplete TLE entry detected.\n");
72              break;
73          }
```

```
74
75          count++;
76          *tles = realloc(*tles, count * sizeof(ownTLE));
77          if (!*tles) {
78              fprintf(stderr, "Memory allocation failed for TLE array.\n");
79              free(data_copy);
80              return 0;
81          }
82
83          ownTLE* current_tle = &((*tles)[count - 1]);
84          strcpy(current_tle->name, name);
85          strcpy(current_tle->tle.line1, line1);
86          strcpy(current_tle->tle.line2, line2);
87
88          parseLines(&current_tle->tle, line1, line2);
89
90          line = strtok_r(NULL, "\n", &context);
91      }
92
93      free(data_copy);
94      return count;
95  }
```

Listing 8.6: Implementació de la descàrrega de dades amb libcurl i el parseig de TLE

```
1  #ifndef COORDINATES_TRANSLATION_FUNCTIONS_H
2  #define COORDINATES_TRANSLATION_FUNCTIONS_H
3
4  // Function prototypes for coordinate transformations
5
6  /**
7   * Converteix coordenades geodesiques (Lat, Lon, Alt) a ECEF (Earth-Centered,
        Earth-Fixed).
8   */
9  void geodetic_to_ecef(const GeodeticPosition* geo, double* ecef_x, double*
       ecef_y, double* ecef_z);
10
11 /**
12  * Transforma coordenades ECI (Inercials) a ECEF (Fixes a la Terra) usant el
        temps sideri.
13  */
14 void eci_to_ecef(const double* eci_pos, double julian_date, double* ecef_pos);
15
16 /**
17  * Calcula la posicio topocentrica (Azimut, Elevacio, Rang) des de l'observador
        al satel-lit.
18  */
19 void ecef_to_topo(const double* observer_ecef, const double* satellite_ecef,
```

```
    const GeodeticPosition* observer_geo, double* azimuth, double* elevation,
    double* range);

/**
 * Converteix el temps Unix (seconds since 1970) a Data Juliana.
 */
double unix_to_julian(time_t unix_time);

#endif // COORDINATES_TRANSLATION_FUNCTIONS_H
```

Listing 8.7: Fitxer de capçalera per a les funcions de transformació de coordenades (GEOD, ECEF, ECI, TOPO)

```
#include "utils.h"
#include "coordinates_translation_functions.h"

// Function to convert geodetic coordinates to ECEF (Earth-Centered, Earth-Fixed)
void geodetic_to_ecef(const GeodeticPosition* geo, double* ecef_x, double* ecef_y, double* ecef_z) {
    double lat_rad = geo->latitude_deg * DEGREES_TO_RADIANS;
    double lon_rad = geo->longitude_deg * DEGREES_TO_RADIANS;

    // Radius of curvature in prime vertical (N)
    double N = EARTH_RADIUS_WGS84 / sqrt(1.0 - EARTH_ECCENTRICITY_SQUARED * sin(lat_rad) * sin(lat_rad));

    *ecef_x = (N + geo->altitude_km) * cos(lat_rad) * cos(lon_rad);
    *ecef_y = (N + geo->altitude_km) * cos(lat_rad) * sin(lon_rad);
    *ecef_z = (N * (1.0 - EARTH_ECCENTRICITY_SQUARED) + geo->altitude_km) * sin(lat_rad);
}

// Function to convert ECI (Earth-Centered Inertial) to ECEF coordinates
void eci_to_ecef(const double* eci_pos, double julian_date, double* ecef_pos) {
    // Julian Date to Greenwich Sidereal Time (GST) - GMST calculation per USNO standards
    double jd = julian_date;
    double T_UT1 = (jd - 2451545.0) / 36525.0; // Julian centuries from J2000.0
    double gmst_seconds = 67310.54841 + (876600.0 * 3600.0 + 8640184.812866) * T_UT1
                          + 0.093104 * T_UT1 * T_UT1 - 6.2e-6 * T_UT1 * T_UT1 *
                              T_UT1;
    double gmst_degrees = fmod(gmst_seconds / 240.0, 360.0);
    if (gmst_degrees < 0) gmst_degrees += 360.0;
    double lon_rad = gmst_degrees * DEGREES_TO_RADIANS;

    // Rotation matrix for ECI to ECEF (rotation about z-axis by GMST)
```

```
29      double cos_lon = cos(lon_rad);
30      double sin_lon = sin(lon_rad);
31
32      ecef_pos[0] = eci_pos[0] * cos_lon + eci_pos[1] * sin_lon;
33      ecef_pos[1] = -eci_pos[0] * sin_lon + eci_pos[1] * cos_lon;
34      ecef_pos[2] = eci_pos[2];
35  }
36
37  // Function to convert ECEF to topocentric coordinates (azimuth, elevation,
        range)
38  void ecef_to_topo(const double* observer_ecef, const double* satellite_ecef,
        const GeodeticPosition* observer_geo, double* azimuth, double* elevation,
        double* range) {
39      double rx = satellite_ecef[0] - observer_ecef[0];
40      double ry = satellite_ecef[1] - observer_ecef[1];
41      double rz = satellite_ecef[2] - observer_ecef[2];
42
43      double lat_rad = observer_geo->latitude_deg * DEGREES_TO_RADIANS;
44      double lon_rad = observer_geo->longitude_deg * DEGREES_TO_RADIANS;
45
46      // Transformation to NEZ (North-East-Zenith) frame
47      double topoS[3];
48      topoS[0] = -sin(lat_rad) * cos(lon_rad) * rx - sin(lat_rad) * sin(lon_rad) *
            ry + cos(lat_rad) * rz;  // North
49      topoS[1] = -sin(lon_rad) * rx + cos(lon_rad) * ry;
                                                                  // East
50      topoS[2] = cos(lat_rad) * cos(lon_rad) * rx + cos(lat_rad) * sin(lon_rad) *
            ry + sin(lat_rad) * rz;   // Zenith
51
52      *range = sqrt(topoS[0]*topoS[0] + topoS[1]*topoS[1] + topoS[2]*topoS[2]);
53      *azimuth = atan2(topoS[1], topoS[0]);
54      *elevation = asin(topoS[2] / *range);
55
56      // Convert from radians to degrees
57      *azimuth = *azimuth / DEGREES_TO_RADIANS;
58      *elevation = *elevation / DEGREES_TO_RADIANS;
59
60      if (*azimuth < 0) *azimuth += 360.0;
61  }
62
63  // Helper function to convert a Unix timestamp to a Julian date
64  double unix_to_julian(time_t unix_time) {
65      return (double)unix_time / SECONDS_PER_DAY + 2440587.5;
66  }
```

Listing 8.8: Implementació de les transformacions de coordenades: Geodèsiques, ECI, ECEF i Topocèntriques

8.1.5 Codis útils per a les comandes lliures

```h
#ifndef UTILS_H
#define UTILS_H

#include <stddef.h>

/**
 * Inverteix una comanda de moviment per a operacions de "Undo".
 * MAA (Left) -> MAH (Right), MEU (Up) -> MED (Down), etc.
 */
void invert_command(const char *cmd, char *out, size_t out_len);

/**
 * Llegeix una linia del descriptor de fitxer (serie) amb un temps d'espera limitat.
 * Crucial per evitar que el programa s'aturi si l'Arduino no respon.
 * @return bytes llegits o -1 en cas de timeout/error.
 */
int read_line_with_timeout(int fd, char *buf, size_t buflen, int timeout_seconds);

/**
 * Funcio d'alt nivell que coordina l'enviament de la comanda a l'Arduino
 * i l'actualitzacio de l'estat local en el fitxer JSON.
 */
int send_command_and_update_json(const char *command, const char *json_path, int newAz, int newEl);

#endif // UTILS_H
```

Listing 8.9: Utilitats per a la manipulació de comandes i comunicació amb timeout

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <ctype.h>
#include <termios.h>

#include "utils.h"
#include "JSON_related_functions.h"
#include "../Arduino_stuff/arduino_communications.h"

/* Inverteix una comanda mantenint el format numeric (ex: MAH010 -> MAA010) */
void invert_command(const char *cmd, char *out, size_t out_len) {
    if (!cmd || !out || out_len < 8) return;

```

```
16      if (cmd[0] == 'C') { // Calibratge es mant  igual
17          snprintf(out, out_len, "C\n");
18          return;
19      }
20
21      const char *prefix = NULL;
22      if (strncmp(cmd, "MAA", 3) == 0)      prefix = "MAH";
23      else if (strncmp(cmd, "MAH", 3) == 0) prefix = "MAA";
24      else if (strncmp(cmd, "MEU", 3) == 0) prefix = "MED";
25      else if (strncmp(cmd, "MED", 3) == 0) prefix = "MEU";
26
27      if (prefix) {
28          const char *digits = cmd + 3;
29          snprintf(out, out_len, "%s%.3s\n", prefix, digits);
30      } else {
31          snprintf(out, out_len, "%s\n", cmd);
32      }
33  }
34
35  /* Envia una comanda a l'Arduino i actualitza el registre d'estat JSON */
36  int send_command_and_update_json(const char *command, const char *json_path, int
        newAz, int newEl) {
37      char port_path[256];
38      if (find_arduino_port(port_path, sizeof(port_path)) != 0) return -1;
39
40      int fd = open_and_configure_port(port_path, B9600);
41      if (fd < 0) return -1;
42
43      // Handshake inicial i enviament
44      char cmd_with_nl[64];
45      snprintf(cmd_with_nl, sizeof(cmd_with_nl), "%s\n", command);
46      write(fd, cmd_with_nl, strlen(cmd_with_nl));
47      tcdrain(fd); // Espera que es transmetin totes les dades
48
49      // Lectura de confirmacio "DONE" de l'Arduino
50      char reply[128];
51      read_line_with_timeout(fd, reply, sizeof(reply), 30);
52      close(fd);
53
54      // Actualitzacio del fitxer JSON d'estat
55      char *json = read_file_alloc(json_path);
56      char *inner = extract_last_movements_inner(json);
57
58      FILE *out = fopen(json_path, "w");
59      if (out) {
60          fprintf(out, "{\n  \"currentAz\": %d,\n  \"currentEl\": %d,\n", newAz,
            newEl);
```

```
61          fprintf(out, "  \"LastMovements\": [\n    \"%s\"%s%s\n  ]\n}",
62                  command, (inner && strlen(inner) > 0) ? ", " : "", inner ? inner
                        : "");
63          fclose(out);
64      }
65
66      free(json); free(inner);
67      return 0;
68  }
```

Listing 8.10: Implementacio de les utilitats de moviment: Inversio de comandes i sincronitzacio PC-Arduino-JSON

```
1   #ifndef JSON_RELATED_FUNCTIONS_H
2   #define JSON_RELATED_FUNCTIONS_H
3
4   #include <time.h>
5
6   /**
7    * @brief Calcula la ruta absoluta del fitxer JSON d'estat.
8    * * Utilitza la ruta de l'executable (argv[0]) per localitzar el fitxer
9    * de configuracio independentment del directori de treball.
10   */
11  void get_absolute_json_path(const char *argv0, char *json_path, size_t
        json_path_len);
12
13  /**
14   * @brief Llegeix un fitxer complet i l'emmagatzema en un buffer dinamic.
15   * @return Punter a la cadena (malloc'd) o NULL en cas d'error.
16   */
17  char *read_file_alloc(const char *path);
18
19  /**
20   * @brief Extreu un valor sencer d'un objecte JSON de manera simple.
21   * * Cerca la clau i parseja el valor numeric immediatament posterior als dos
         punts.
22   */
23  int parse_json_int(const char *buf, const char *key, int *out_value);
24
25  /**
26   * @brief Extreu el contingut de l'array "LastMovements".
27   * * Retorna una cadena amb els elements entre els claudators [ ].
28   */
29  char *extract_last_movements_inner(const char *buf);
30
31  /**
32   * @brief Utilitat per generar una marca de temps actual.
33   */
```

```c
void timestamp_now(char *out, size_t n);

/**
 * @brief Obt la posici actual (Azimut i Elevaci ) en una sola crida.
 */
int parse_json_position(const char *buf, int *out_az, int *out_el);

#endif // JSON_RELATED_FUNCTIONS_H
```

Listing 8.11: Interfície per a la manipulació del fitxer d'estat JSON

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <unistd.h>
#include <ctype.h>

#include "JSON_related_functions.h"

/* Calcula la ruta absoluta del JSON utilitzant el sistema de fitxers /proc de Linux */
void get_absolute_json_path(const char *argv0, char *json_path, size_t json_path_len) {
    char exe_dir[512] = {0};
    char proc_path[512] = {0};

    // readlink ens dona la ruta real del binari en execucio
    ssize_t len = readlink("/proc/self/exe", proc_path, sizeof(proc_path) - 1);

    if (len > 0) {
        proc_path[len] = '\0';
        char *last_slash = strrchr(proc_path, '/');
        if (last_slash) {
            size_t dir_len = (size_t)(last_slash - proc_path);
            strncpy(exe_dir, proc_path, dir_len);
            exe_dir[dir_len] = '\0';
        }
    } else {
        // Fallback en cas de sistemes no-POSIX
        strncpy(exe_dir, ".", 1);
    }

    snprintf(json_path, json_path_len, "%s/comm_files/antennaStatus.json", exe_dir);
}

/* Llegeix el fitxer complet a memoria per facilitar el parseig */
```

```c
char *read_file_alloc(const char *path) {
    FILE *f = fopen(path, "r");
    if (!f) return NULL;
    fseek(f, 0, SEEK_END);
    long sz = ftell(f);
    rewind(f);

    char *buf = malloc((size_t)sz + 1);
    if (buf) {
        size_t r = fread(buf, 1, (size_t)sz, f);
        buf[r] = '\0';
    }
    fclose(f);
    return buf;
}

/* Extractor de valors JSON: cerca la clau i converteix la cadena a sencer */
int parse_json_int(const char *buf, const char *key, int *out_value) {
    const char *p = strstr(buf, key);
    if (!p) return -1;
    p = strchr(p, ':');
    if (!p) return -1;

    char *endptr;
    long v = strtol(++p, &endptr, 10);
    if (p == endptr) return -1;

    *out_value = (int)v;
    return 0;
}

/* Extreu l'historial de moviments delimitat per claudators [ ] */
char *extract_last_movements_inner(const char *buf) {
    const char *p = strstr(buf, "LastMovements");
    if (!p) return NULL;
    p = strchr(p, '[');
    if (!p) return NULL;

    const char *q = strchr(++p, ']');
    if (!q) return NULL;

    size_t len = (size_t)(q - p);
    char *inner = malloc(len + 1);
    if (inner) {
        strncpy(inner, p, len);
        inner[len] = '\0';
    }
```

```
82      return inner;
83  }
```

Listing 8.12: Implementació de la persistència d'estat mitjançant el processament de fitxers JSON

```
1   #ifndef COMMAND_HANDLER_FUNCTIONS_H
2   #define COMMAND_HANDLER_FUNCTIONS_H
3
4   /**
5    * @brief Accio Origin: Desfa tots els moviments registrats al JSON fins a
         tornar al punt inicial.
6    */
7   int handle_origin_action(const char *json, const char *json_path);
8
9   /**
10   * @brief Accio Back: Desfa l'ultim moviment realitzat consultant l'historial.
11   */
12  int handle_back_action(const char *json, const char *json_path);
13
14  /**
15   * @brief Accio Test: Executa una sequencia de moviments predefinida per
         verificar els motors.
16   */
17  int handle_test_action(const char *json_path, int curAz, int curEl);
18
19  /**
20   * @brief Accio Calibrate: Mou l'antena a la posicio de referencia (Az=0, El=90)
         .
21   */
22  int handle_calibrate_action(const char *json_path, int curAz, int curEl);
23
24  /**
25   * @brief Accio Force: Actualitza l'estat intern a 0,90 sense realitzar moviment
         fisic.
26   */
27  int handle_force_action(const char *json_path, int curAz, int curEl);
28
29  /**
30   * @brief Accio Movement: Gestiona moviments simples (Esquerra, Dreta, Amunt,
         Avall) amb un pas determinat.
31   */
32  int handle_movement_action(char action, int step, const char *json, const char *
        json_path, int curAz, int curEl);
33
34  #endif // COMMAND_HANDLER_FUNCTIONS_H
```

Listing 8.13: Interfície del gestor de comandes d'alt nivell per al control de l'antena

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include "command_handler_functions.h"
#include "JSON_related_functions.h"
#include "../Arduino_stuff/arduino_communications.h"
#include "../AntennaMovementInstructions/utils.h"

// Funci  Origin: Desf  tots els moviments fins arribar al punt de rep s
int handle_origin_action(const char *json, const char *json_path) {
    char *inner = extract_last_movements_inner(json);
    if (!inner || strlen(inner) == 0) return 0;

    // Bucle de "rewind" basat en l'historial del JSON
    while (has_movements(inner)) {
        char *tmp_json = read_file_alloc(json_path);
        handle_back_action(tmp_json, json_path);
        free(tmp_json);
    }
    return 0;
}

// Funci  Calibrate: Sincronitza l'antena a la posici  Az=0, El=90
int handle_calibrate_action(const char *json_path, int curAz, int curEl) {
    // 1. Calcula la diferencia fins a la posicio de referencia
    int azimuth_correction = -curAz;
    int elevation_correction = 90 - curEl;

    // 2. Executa els moviments de correccio fisics
    if (azimuth_correction != 0) {
        char action = (azimuth_correction < 0) ? 'L' : 'R';
        handle_movement_action(action, abs(azimuth_correction), json, json_path, curAz, curEl);
    }
    // ... repetit per a elevacio ...
    return 0;
}

// Funci  Movement: La unitat b sica de moviment i actualitzaci  d'estat
int handle_movement_action(char action, int step, const char *json, const char *json_path, int curAz, int curEl) {
    // 1. Validaci  de l mits f sics (0-90 per elevaci , -360 a 360 azimut)
    int newAz = curAz, newEl = curEl;
    calculate_new_position(action, step, &newAz, &newEl);

    // 2. Enviament de la comanda serie a l'Arduino (ex: MAH010)
    char command[32];
    build_serial_command(action, step, command);
    send_to_arduino(command);

    // 3. Persist ncia: Actualitza el fitxer JSON amb la nova posicio i l'historial
    update_json_state(json_path, newAz, newEl, command);

    return 0;
}
```

Listing 8.14: Lògica de gestió de comandes: Calibratge, històrial i control de moviments

8.1.6 Codis útils per a la gestió del *hardware*

```c
// Header guard
#ifndef ARDUINO_COMMUNICATIONS_H
#define ARDUINO_COMMUNICATIONS_H

#include <termios.h>
#include <unistd.h>
```

```
7
8  /**
9   * @brief Obre i configura un port serie.
10  * * @param port_name El cam  al fitxer del dispositiu (ex: "/dev/ttyACM0").
11  * @param baud_rate La velocitat de transmissio (ex: B9600).
12  * @return int El descriptor de fitxer o -1 en cas d'error.
13  */
14 int open_and_configure_port(const char *port_name, speed_t baud_rate);
15
16 /**
17  * @brief Cerca un dispositiu serie Arduino connectat al sistema.
18  * * Escaneja els ports /dev/ttyACM* i /dev/ttyUSB* buscant identificadors de
        hardware.
19  * * @param out_path Buffer on s'escriura el cami detectat.
20  * @param out_len Mida del buffer out_path.
21  * @return int 0 si es troba, -1 si no es detecta cap Arduino.
22  */
23 int find_arduino_port(char *out_path, size_t out_len);
24
25 #endif // ARDUINO_COMMUNICATIONS_H
```

Listing 8.15: Fitxer de capçalera per a la comunicació sèrie amb Arduino

```
1  #include <termios.h>
2  #include <fcntl.h>
3  #include <sys/ioctl.h>
4  #include "arduino_communications.h"
5
6  /**
7   * Escaneja el sistema de fitxers /sys per trobar un dispositiu USB
8   * amb un Vendor ID (VID) compatible amb Arduino o xips serials comuns.
9   */
10 int find_arduino_port(char *out_path, size_t out_len) {
11     const char *prefixes[] = { "/dev/ttyACM", "/dev/ttyUSB" };
12     char dev[PATH_MAX], sysbase[PATH_MAX], buf[256];
13
14     for (size_t p = 0; p < 2; ++p) {
15         for (int i = 0; i < 16; ++i) {
16             snprintf(dev, sizeof(dev), "%s%d", prefixes[p], i);
17             if (access(dev, F_OK) != 0) continue;
18
19             snprintf(sysbase, sizeof(sysbase), "/sys/class/tty/%s/device",
                   basename(dev));
20             char candidate[PATH_MAX];
21             strcpy(candidate, sysbase);
22
23             for (int level = 0; level < 5; ++level) {
24                 // Comprovacio per fabricant o per Vendor ID (hexadecimal)
```

```
25                 if (read_first_line(probe, buf, sizeof(buf)) == 0) {
26                     int vid = (int)strtol(buf, NULL, 16);
27                     if (vid == 0x2341 || vid == 0x1A86 || vid == 0x0403) {
28                         strncpy(out_path, dev, out_len-1);
29                         return 0;
30                     }
31                 }
32                 // Pujar un nivell en l'arbre sysfs
33             }
34         }
35     }
36     return -1;
37 }
38 
39 /**
40  * Configura el port serie en mode "raw" per a una comunicacio neta.
41  * Utilitza la configuracio 8N1: 8 bits de dades, sense paritat, 1 bit de parada
        .
42  */
43 int open_and_configure_port(const char *port_name, speed_t baud_rate) {
44     int fd = open(port_name, O_RDWR | O_NOCTTY | O_NDELAY);
45     if (fd == -1) return -1;
46 
47     fcntl(fd, F_SETFL, 0); // Mode bloquejant
48 
49     struct termios tty;
50     tcgetattr(fd, &tty);
51 
52     cfsetospeed(&tty, baud_rate);
53     cfsetispeed(&tty, baud_rate);
54 
55     tty.c_cflag |= (CLOCAL | CREAD);    // Ignora control de modem
56     tty.c_cflag &= ~CSIZE;
57     tty.c_cflag |= CS8;                 // 8 bits de dades
58     tty.c_cflag &= ~PARENB;             // Sense paritat
59     tty.c_cflag &= ~CSTOPB;             // 1 bit de parada
60 
61     // Mode "Raw": desactiva processament de caracters especials (ICANON, ECHO,
        etc.)
62     tty.c_lflag &= ~(ICANON | ECHO | ECHOE | ISIG);
63     tty.c_oflag &= ~OPOST;
64 
65     tcsetattr(fd, TCSANOW, &tty);
66     return fd;
67 }
```

Listing 8.16: Implementació de la comunicació sèrie: Autodetecció de l'Arduino i configuració

de termios

8.1.7 *Firmware*: codi de l'arduino

```
#include <SoftwareSerial.h>

// Definici  de pins per als controladors de motors (Stepper Drivers)
#define PUL_AZIMUTH 10
#define DIR_AZIMUTH 9
#define ENA_AZIMUTH 8

#define PUL_ELEVATION 7
#define DIR_ELEVATION 6
#define ENA_ELEVATION 5

// Variables globals de control
String command = "";
bool completeCommandDetected = false;

int AzSpeed = 600;           // Microsegons entre polsos (velocitat)
int AzStepsPerDegree = 152;   // Relaci  de passos per grau (Azimut)
int ElStepsPerDegree = 85;    // Relaci  de passos per grau (Elevaci )

void moveAzimuth(bool dir, int deg){
  digitalWrite(DIR_AZIMUTH, dir ? LOW : HIGH);
  long totalSteps = (long) AzStepsPerDegree * deg;

  for (long i = 0; i < totalSteps; i++) {
    digitalWrite(PUL_AZIMUTH, HIGH);
    delayMicroseconds(AzSpeed);
    digitalWrite(PUL_AZIMUTH, LOW);
    delayMicroseconds(AzSpeed);
  }
}

void moveElevation(bool dir, int deg){
  digitalWrite(DIR_ELEVATION, dir ? LOW : HIGH);
  long totalSteps = (long) ElStepsPerDegree * deg;

  for (long i = 0; i < totalSteps; i++) {
    digitalWrite(PUL_ELEVATION, HIGH);
    delayMicroseconds(600);
    digitalWrite(PUL_ELEVATION, LOW);
    delayMicroseconds(600);
  }
}
```

```
43
44 void setup() {
45   Serial.begin(9600);
46   command.reserve(30);
47
48   pinMode(PUL_AZIMUTH, OUTPUT);
49   pinMode(DIR_AZIMUTH, OUTPUT);
50   pinMode(ENA_AZIMUTH, OUTPUT);
51   pinMode(PUL_ELEVATION, OUTPUT);
52   pinMode(DIR_ELEVATION, OUTPUT);
53   pinMode(ENA_ELEVATION, OUTPUT);
54
55   digitalWrite(ENA_AZIMUTH, HIGH);
56   digitalWrite(ENA_ELEVATION, HIGH);
57
58   Serial.print("Arduino ready!\n");
59 }
60
61 void loop() {
62   // 1. Lectura del port serie
63   while (Serial.available()) {
64     char inChar = (char)Serial.read();
65     if (inChar == '\n' || inChar == '\r') {
66       completeCommandDetected = true;
67     } else {
68       command += inChar;
69     }
70   }
71
72   // 2. Processament de la comanda
73   if(completeCommandDetected){
74     command.trim();
75     String commandStatus = "ERROR";
76
77     if(command.startsWith("M")){
78       if(command.indexOf("A") == 1){
79         // Comanda d'Azimut (MAH o MAA)
80         bool dir = (command.charAt(2) == 'A');
81         int deg = command.substring(3, 6).toInt();
82         moveAzimuth(dir, deg);
83       } else {
84         // Comanda d'Elevaci  (MEU o MED)
85         bool dir = (command.charAt(2) == 'D');
86         int deg = command.substring(3, 6).toInt();
87         moveElevation(dir, deg);
88       }
89       commandStatus = "DONE";
```

```
90      }
91
92      Serial.print(commandStatus + "\n");
93      command = "";
94      completeCommandDetected = false;
95    }
96  }
```

Listing 8.17: Firmware d'Arduino: Control de motors pas a pas i protocol de comunicació sèrie

8.2 Codis de la pàgina web

8.2.1 Vistes PHP

```
1  <!DOCTYPE html>
2  <html lang="ca">
3  <head>
4      <meta charset="UTF-8">
5      <title>Antena Rotatoria - LEO Satellites</title>
6      <link rel="stylesheet" href="WebPage_Style/basic_style.css">
7  </head>
8  <body>
9      <div id="title-bar">
10         <img src="WebPage_Style/images/IEEC_logo.png" alt="IEEC" class="logo">
11         <h1 id="project-title">Antena Rotatoria Automatica</h1>
12         <img src="WebPage_Style/images/UAB_logo.png" alt="UAB" class="logo">
13     </div>
14
15     <div id="introduction-section" class="section">
16         <div class="section-image-container-right">
17             <div class="text">
18                 <p>Interficie de control per a l'antena automatica de seguiment
                       de satellits.</p>
19                 <ul>
20                     <li><b>Programar captures:</b> Basat en dades TLE
                           actualitzades.</li>
21                     <li><b>Control lliure:</b> Moviment manual i calibratge.</li
                           >
22                 </ul>
23             </div>
24             <div class="image">
25                 <img src="WebPage_Style/images/antenna_photo.jpg" alt="Antena">
26             </div>
27         </div>
```

```
28          <p class="warning">IMPORTANT: Llegiu les instruccions de muntatge abans
                de procedir.</p>
29      </div>
30
31      <div id="schedule-section" class="section">
32          <h2>Programacio de captures</h2>
33          <div class="section-buttons-container">
34              <form action="WebPage_Views/schedule_captures.php" method="post"
                    class="button-box">
35                  <input type="hidden" name="constellation" value="S">
36                  <button type="submit" class="big-button">Starlink</button>
37              </form>
38              <form action="WebPage_Views/schedule_captures.php" method="post"
                    class="button-box">
39                  <input type="hidden" name="constellation" value="O">
40                  <button type="submit" class="big-button">Orbcom</button>
41              </form>
42              </div>
43      </div>
44
45      <div id="free-control-section" class="section">
46          <h2>Control lliure de l'antena</h2>
47          <div class="section-buttons-container">
48              <a href="WebPage_Views/free_control.php" class="link-as-button">
                    Controlador lliure</a>
49          </div>
50      </div>
51  </body>
52  </html>
```

Listing 8.18: Portal principal (index.php): Menú de navegació i gestió de captures

```
1   <?php
2   // Rutes als fitxers de comunicacio i executable C
3   $file_path = '../Executables/comm_files/Generate_available_list_of_SV.txt';
4   $executable_path = '../Executables/generate_available_list_of_SV';
5
6   // Gestio de la constel-lacio seleccionada
7   $constellation = $_REQUEST['constellation'] ?? 'S';
8
9   // Si es rep una peticio POST, processem els filtres
10  if ($_SERVER["REQUEST_METHOD"] == "POST") {
11      $excludeVisible = $_POST['excludeVisible'] ?? 0;
12      $elevationMask = $_POST['elevationMask'] ?? 10;
13      $sortBy = $_POST['sortBy'] ?? 1;
14      $SV_to_display = $_POST['SV-to-display'] ?? 10;
15
16      // Preparacio de la comanda per a l'executable C
17      $args = escapeshellarg($constellation) . " " . $excludeVisible . " 0 " . $elevationMask . " " . $sortBy . " " .
            $SV_to_display;
18      $command = escapeshellarg($executable_path) . " " . $args;
19
20      // Execucio del binari i redireccio de la sortida al fitxer de text
21      exec("{$command} > " . escapeshellarg($file_path) . " 2>&1");
22  }
23
24  // Lectura i parseig del fitxer generat pel programa en C
```

```
$sv_array = file($file_path, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
$sv_objects = [];
$regex = '/^Satellite:\s*(.+?),\s*Start:\s*([^,]+),\s*End:\s*([^,]+),\s*Max Elevation:\s*([0-9.+-]+)\s*deg/i';

foreach ($sv_array as $line) {
    if (preg_match($regex, $line, $m)) {
        $sv_objects[] = [
            'name' => trim($m[1]),
            'start' => trim($m[2]),
            'end' => trim($m[3]),
            'max_elevation' => floatval($m[4])
        ];
    }
}
?>

<div id="SV-list-section">
    <h3>Satellits disponibles:</h3>
    <form id="schedule-form" action="capture_process.php" method="post">
        <?php foreach ($sv_objects as $idx => $obj): ?>
            <input type="radio" name="satellite" value="<?= $obj['name'] ?>"
                   data-start="<?= $obj['start'] ?>" data-end="<?= $obj['end'] ?>">
            <label><?= $obj['name'] ?> (Max: <?= $obj['max_elevation'] ?> deg)</label><br>
        <?php endforeach; ?>
        <input type="submit" value="Programar Captura">
    </form>
</div>
```

Listing 8.19: Vista de programació de captures: Filtres dinàmics i execució de binaris de càlcul

```
<!DOCTYPE html>
<html lang="ca">
<head>
    <meta charset="utf-8">
    <title>Consola de Control - ARA</title>
    <link rel="stylesheet" href="../WebPage_Style/basic_style.css">
    <link rel="stylesheet" href="../WebPage_Style/free_control.css">
    <script src="../WebPage_Scripts/free_control.js" defer></script>
</head>
<body>
    <div id="title-section">
        <a href="../index.php" class="home-button">
            <img src="../WebPage_Style/images/home_icon.png" alt="Home">
        </a>
        <h1>Controlador Lliure</h1>
    </div>

    <div id="console">
        <div id="console-buttons-section">
            <div class="button-element">
                <div class="button-label">Calibrate</div>
                <button id="button-calibrate" class="button-button">C</button>
                <div id="led-calibrate" class="led"></div>
            </div>
            </div>

        <div id="console-display-section">
```

```
28              <div id="console-display">
29                  <div class="display-row">
30                      <span class="display-label">AZ</span>
31                      <span id="display-azimuth" class="display-value"> --.- deg</
                            span>
32                  </div>
33                  <div class="display-row">
34                      <span class="display-label">EL</span>
35                      <span id="display-elevation" class="display-value"> --.- deg
                            </span>
36                  </div>
37              </div>
38          </div>
39
40          <div id="console-arrows-section">
41              <div class="arrow-button-pair">
42                  <button id="arrow-left" class="arrow-button">&#9664;</button>
43                  <button id="arrow-right" class="arrow-button">&#9658;</button>
44              </div>
45              <input type="range" id="step-size" min="1" max="90" value="1">
46              <div id="step-size-value">1</div>
47          </div>
48      </div>
49  </body>
50  </html>
```

Listing 8.20: Interficie de control manual (free_control.php): Disseny de consola HMI

```
1   <?php
2   /**
3    * Aquest fitxer permet testejar la connexi  serie enviant ordres simples ('A'
         o 'B')
4    * a traves d'un executable binari escrit en C.
5    */
6   $exec_result = null;
7
8   if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['cmd'])) {
9       $cmd = $_POST['cmd'] === 'A' ? 'A' : 'B';
10      $exec_path = realpath(__DIR__ . '/../Executables/ArduinoTest');
11
12      if ($exec_path && is_executable($exec_path)) {
13          // Preparem els descriptors per capturar la sortida (stdout) i els
                errors (stderr)
14          $descriptorspec = [
15              0 => ['pipe', 'r'],
16              1 => ['pipe', 'w'],
17              2 => ['pipe', 'w']
18          ];
```

```
19
20          $proc = proc_open(escapeshellcmd($exec_path) . ' ' . escapeshellarg($cmd
                ), $descriptorspec, $pipes);
21
22          if (is_resource($proc)) {
23              $stdout = stream_get_contents($pipes[1]);
24              $stderr = stream_get_contents($pipes[2]);
25              $exit = proc_close($proc);
26              $exec_result = "Exit Code: $exit\nSTDOUT: $stdout\nSTDERR: $stderr";
27          }
28      } else {
29          $exec_result = "Error: Executable no trobat o sense permisos.";
30      }
31  }
32  ?>
33
34  <form method="post">
35      <button type="submit" name="cmd" value="A">Toggle LED On (A)</button>
36      <button type="submit" name="cmd" value="B">Toggle LED Off (B)</button>
37  </form>
38
39  <?php if ($exec_result): ?>
40      <pre><?= htmlspecialchars($exec_result) ?></pre>
41  <?php endif; ?>
```

Listing 8.21: Eina de test de comunicació (ArduinoTest.php): Verificació de la passarel·la PHP-C-Arduino

```
1   <?php
2   /**
3    * Aquest fitxer te un doble comportament:
4    * 1. Inicialment renderitza una pagina amb un 'spinner' de carrega.
5    * 2. Via AJAX, s'autocrida per executar el binari 'process_antenna_movements'.
6    */
7
8   $executable_path = '/../Executables/process_antenna_movements';
9
10  // Bloc d'execucio AJAX
11  if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['action']) && $_POST['action'] === 'run') {
12      header('Content-Type: application/json; charset=utf-8');
13
14      $sat = $_POST['satellite'] ?? 'Unknown';
15      $const = $_POST['constellation'] ?? 'S';
16      $exec_path = realpath(__DIR__ . $executable_path);
17
18      // Execucio segura amb proc_open
19      $descriptorspec = [
20          0 => ['pipe', 'r'], // stdin
21          1 => ['pipe', 'w'], // stdout
22          2 => ['pipe', 'w']  // stderr
23      ];
24
25      $cmd = escapeshellcmd($exec_path) . ' ' . escapeshellarg($sat) . ' ' . escapeshellarg($const);
26      $proc = proc_open($cmd, $descriptorspec, $pipes);
27
28      if (is_resource($proc)) {
29          fclose($pipes[0]);
30          $stdout = stream_get_contents($pipes[1]);
31          $stderr = stream_get_contents($pipes[2]);
```

```
32          $status = proc_close($proc);
33
34          echo json_encode([
35              'success' => true,
36              'exit_code' => $status,
37              'stdout' => $stdout,
38              'stderr' => $stderr
39          ]);
40      }
41      exit;
42  }
43  ?>
44
45  <script>
46      fetch(window.location.href, {
47          method: 'POST',
48          headers: {'Content-Type': 'application/x-www-form-urlencoded'},
49          body: new URLSearchParams({
50              action: 'run',
51              satellite: '<?= $satellite_safe ?>',
52              constellation: '<?= $const_letter ?>'
53          })
54      })
55      .then(resp => resp.json())
56      .then(data => {
57          document.getElementById('loader').style.display = 'none';
58          document.getElementById('result').textContent = data.stdout;
59      });
60  </script>
```

Listing 8.22: Processament de la captura (capture_process.php): Execució asíncrona mitjançant AJAX i proc_open

8.2.2 Api per gestionar peticions

```
1   <?php
2   /**
3    * API per al control de l'antena.
4    * Rep peticions POST amb l'accio (U, D, L, R, T, C, B, O, F) i el pas (step).
5    * Retorna l'estat actual en format JSON.
6    */
7
8   header('Content-Type: application/json');
9   $json_path = '../Executables/comm_files/antennaStatus.json';
10  $executable_path = '../Executables/moveAntenna';
11
12  // Handler per a obtenir l'estat actual (GET)
13  if ($_SERVER['REQUEST_METHOD'] === 'GET' && isset($_GET['get_status'])) {
14      $json_content = @file_get_contents($json_path);
15      $json_data = json_decode($json_content, true);
16      echo json_encode([
17          'success' => true,
18          'currentAz' => $json_data['currentAz'] ?? 0,
19          'currentEl' => $json_data['currentEl'] ?? 0
20      ]);
21      exit;
```

```php
}

// Validacio de l'accio i execucio del binari C
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $action = $_POST['action'] ?? null;
    $step = isset($_POST['step']) ? intval($_POST['step']) : 1;

    // Construccio de la comanda: ./moveAntenna <accio> [pas]
    $command = escapeshellarg($executable_path) . ' ' . escapeshellarg($action)
        . ' ' . escapeshellarg($step);

    exec($command . ' 2>&1', $output, $return_var);

    // Llegir l'estat actualitzat despres del moviment
    $updated_json = json_decode(@file_get_contents($json_path), true);

    echo json_encode([
        'success' => ($return_var === 0),
        'currentAz' => $updated_json['currentAz'] ?? 0,
        'currentEl' => $updated_json['currentEl'] ?? 0,
        'output' => implode("\n", $output)
    ]);
}
?>
```

Listing 8.23: Backend API (antenna_control_api.php): Gestió de peticions AJAX i pont cap a l'executable C

8.2.3 Estil

```css
/* Header principal amb imatge de fons del cel */
#title-bar {
  width: 100%;
  padding: 2em;
  background-color: #333;
  background-image: url('/WebPage_Style/images/sky_image.jpg');
  background-size: cover;
  background-position: center;
  display: flex;
  justify-content: space-around;
  align-items: center;
  color: black;
}

/* Cos de la pagina centrat */
body {
```

```
  display: flex;
  flex-direction: column;
  align-items: center;
  min-height: 100vh;
  margin: 0;
}

/* Seccions amb colors alterns per millorar la llegibilitat */
.section {
  width: 90%;
  padding: 2em 5%;
  display: flex;
  flex-direction: column;
  align-items: center;
}

#introduction-section, #free-control-section { background-color: #f9f9f9; }
#schedule-section, #hardware-mount-instructions-section { background-color: #
    ffffff; }

/* Contenidor de text i imatges lateral */
.section-image-container-right {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

/* Disseny dels botons d'accio principal */
.button-box button, .link-as-button {
  width: 200px;
  height: 50px;
  background-color: #7870a7;
  border-radius: 10px;
  color: white;
  font-weight: bold;
  text-decoration: none;
  cursor: pointer;
  border: none;
}
```

Listing 8.24: Estils base (basic_style.css): Disseny estructural i estetica de seccions

```
/* Importacio de la font digital per als valors de telemetria */
@font-face {
  font-family: 'DigitalFont';
  src: url('../WebPage_Style/images/seven-segment.ttf') format('truetype');
}

/* El contenidor principal simula un xassis metal-lic */
#console {
  width: 80%;
```

```
  background: #9b9b9b; /* Gris industrial */
  display: flex;
  flex-direction: row;
  border: 5px solid #202020;
  border-radius: 15px;
  box-shadow: 0 4px 12px rgba(0,0,0,0.3);
}

/* Estilitzacio del display de 7 segments */
#console-display {
  background-color: #5c5c5c;
  border: 4px solid black;
  padding: 0.5rem 1rem;
}

.display-label, .display-value {
  color: white;
  font-family: 'DigitalFont', monospace;
  font-size: 2.7rem;
  letter-spacing: 0.4em;
}

/* Disseny de botons fisics amb ombres per simular relleu */
.button-button, .arrow-button {
  width: 80px;
  height: 80px;
  background-color: #3a3a3a;
  color: white;
  border: 2px solid #000;
  border-radius: 10px;
  cursor: pointer;
}

/* LEDs d'estat (canviaran de color via JavaScript) */
.led {
  width: 30px;
  height: 30px;
  background: #afafaf; /* Apagat per defecte */
  border-radius: 50%;
  box-shadow: inset 0 2px 4px rgba(0,0,0,0.5);
}
```

Listing 8.25: Estils de la consola de control (free_control.css): Estetica de panell d'operacions

```
/* Seleccio de constel-lacio: Botons d'estat */
.constellation-button {
    min-width: 140px;
    border: 2px solid #7870a7;
    border-radius: 10px;
    background: #ffffff;
    color: #7870a7;
    transition: all 0.2s ease;
}

.constellation-button.selected {
    background: #7870a7;
    color: #ffffff;
}

/* Llista de Satel-lits: Transformacio de Radio Buttons a Targetes */
#schedule-form input[type="radio"] {
    display: none; /* Amaguem el cercle natiu */
}

#schedule-form label {
    display: block;
    width: 100%;
    padding: 1em 1.5em;
    border: 2px solid #7870a7;
    border-radius: 10px;
    cursor: pointer;
    transition: all 0.2s ease;
}
```

```
30
31 /* Efecte visual en seleccionar un satel-lit */
32 #schedule-form input[type="radio"]:checked + label {
33     background-color: #7870a7;
34     color: #ffffff;
35     transform: scale(1.02); /* Lleuger zoom per indicar seleccio */
36 }
37
38 /* Subtitols de dades orbitals (Elevacio, Azimut, Temps) */
39 #schedule-form label small {
40     color: #666666;
41     font-size: 0.9rem;
42 }
43
44 #schedule-form input[type="radio"]:checked + label small {
45     color: #e8e6f0;
46 }
```

Listing 8.26: Estils de llista de captures (schedule_captures.css): Targetes interactives i filtres

8.3 Models 3D pròpis

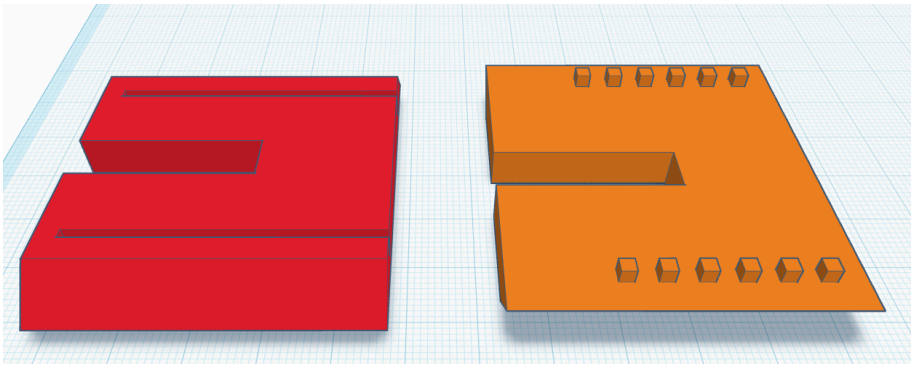

Figura 8.1: Model 3D de la base del motor

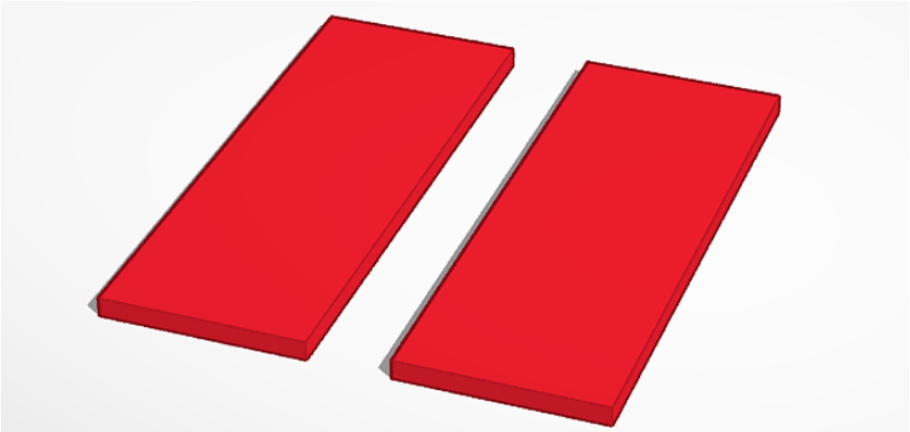

Figura 8.2: Model 3D de les parets del motor

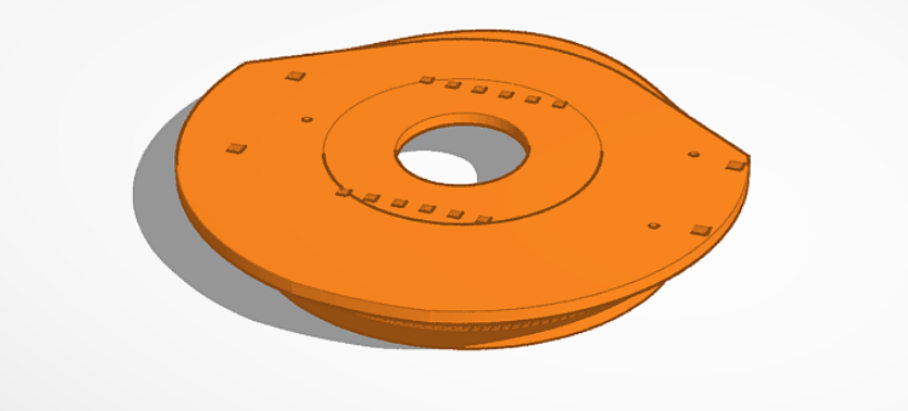

Figura 8.3: Model 3D de la base rotatòria

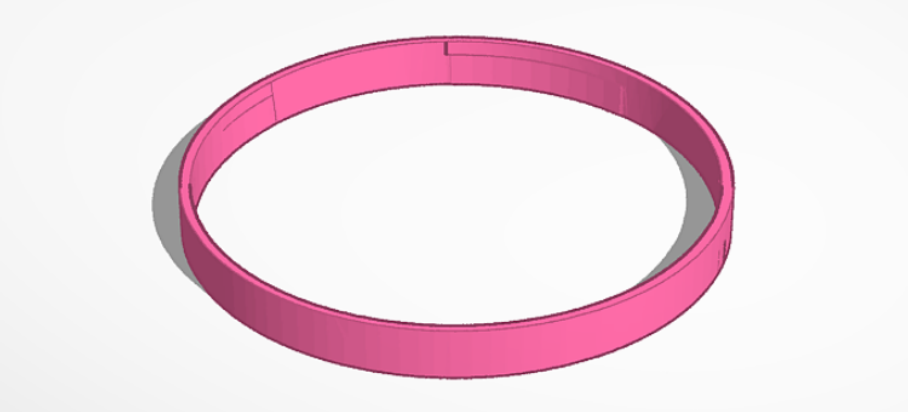

Figura 8.4: Model 3D de la volandera

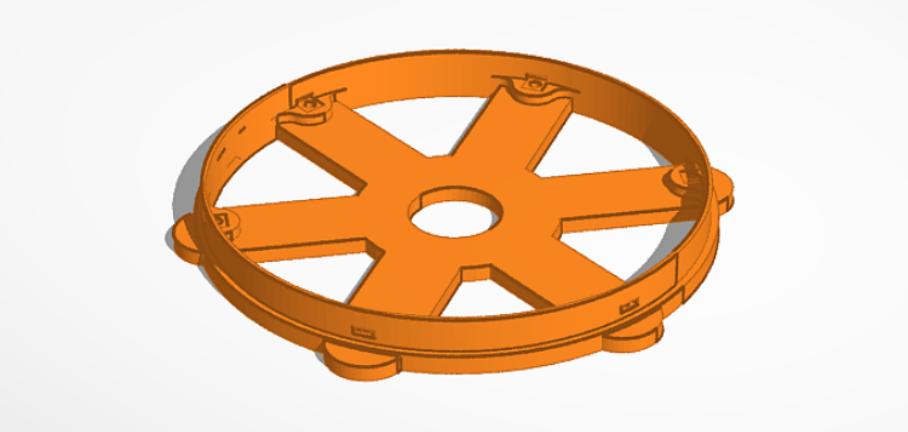

Figura 8.5: Model 3D de la base interior del *bearing*

Figura 8.6: Model 3D de la pilona

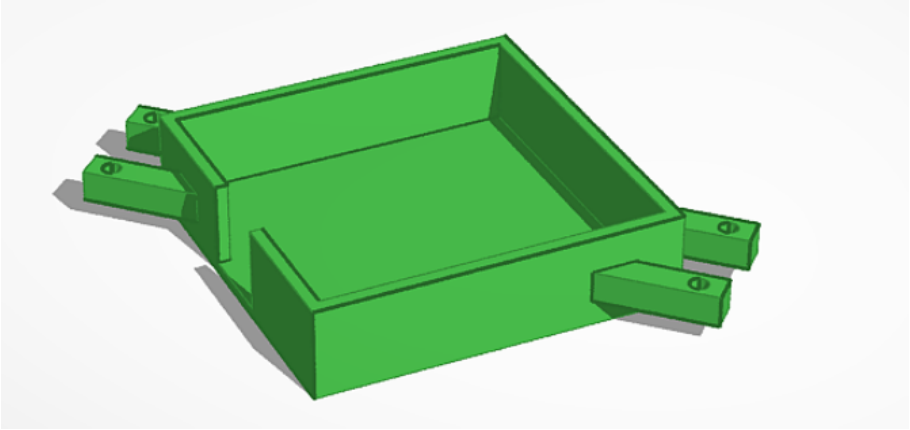

Figura 8.7: Model 3D de la pilona

Bibliografia

[Aer23] Aerospace Answers, "Difference between LEO, MEO, GEO, and HEO orbits (illustrated)", https://aerospaceanswers.com/difference-between-leo-meo-geo-and-heo-orbits-illustrated/, 2023, accedit el 22 de maig de 2024.

[Any23] Anywaves, "LEO, MEO, GEO: How satellite orbit directly shapes antenna design", https://anywaves.com/resources/blog/leo-meo-geo-how-satellite-orbit-directly-shapes-antenna-design/, 2023, accedit el 22 de maig de 2024.

[Bab18] Florin Babota, "Azimuth and altitude for northern latitudes (technical illustration)", ResearchGate, https://www.researchgate.net/figure/Azimuth-and-altitude-for-northern-latitudes$_fig2_327941333$, 2018, $accedit el 22 de maig de 2024$.

[Cha16] Valeria Chagina, Dmitriy Grishko, Vera Maiorova, "Satellite's trajectory propagation at near-circular orbits using tle files in the simplified sgp model", *Science and Education of the Bauman MSTU*, Vol. 16, 01 2016.

[eve24] everything RF, "What is antenna beamwidth?", https://www.everythingrf.com/community/what-is-antenna-beamwidth, 2024, accedit el 22 de maig de 2024.

[Glo24] Globalstar, Inc., "Globalstar official website", https://www.globalstar.com, 2024, accedit el 22 de maig de 2024.

[Koh22] Kriti Kohli, "What is Inertia: Meaning, Definition, Importance & FAQs", https://www.aakash.ac.in/blog/what-is-inertia-meaning-definition-importance-faqs/, 2022, accedit el 22 de maig de 2024.

[L.20] Sébastien L., "Antenna rotator - mount for 2 motors and 1 antenna", Thingiverse, https://www.thingiverse.com/thing:4316563, 2020, model 3D. Accedit el 22 de maig de 2024.

[Mon00] Oliver Montenbruck, Eberhard Gill, "Real-time estimation of SGP4 orbital elements from GPS navigation data", *International Symposium SpaceFlight Dynamics*, Biarritz, France, jun. 2000, mS00/28, DLR, German Space Operations Center.

[Qin24] Wenkai Qin, Zacharias M. Komodromos, Todd E. Humphreys, "An agile, portable antenna system for LEO megaconstellation-based PNT", *University of Texas at Austin, Department of Aerospace Engineering and Engineering Mechanics*, 2024, recerca sobre Posicionament, Navegació i Sincronització (PNT) mitjançant megaconstel·lacions.

[The03] The McGraw-Hill Companies, Inc., *McGraw-Hill Dictionary of Scientific & Technical Terms*, McGraw-Hill Education, New York, USA, $6^{\underline{\text{th}}}$ ed., 2003, copyright © 2003.

[Val06] David A. Vallado, Paul Crawford, "SGP4 orbit determination", *Center for Space Standards and Innovation / University of Dundee*, 2006, presentat originalment el 2006 com a actualització del model de propagació SGP4.

[Vic20] Marcos Vicente, "Starlink satellite constellation (illustration)", Wikimedia Commons, https://commons.wikimedia.org/w/index.php?curid=8966125, 2020, llicència CC BY-SA 4.0. Accedit el 22 de maig de 2024.

[Viq24] Viquipèdia contributors, "Mecànica analítica — Viquipèdia, l'enciclopèdia lliure", https://ca.wikipedia.org/wiki/Mec

[Wal70] J. G. Walker, "Circular orbit patterns providing continuous whole earth coverage", Technical Report RAE-TR-70211, Royal Aircraft Establishment Farnborough (United Kingdom), Farnborough, United Kingdom, nov. 1970, accession Number: AD0722776.

[Wik24] Wikipedia contributors, "Satellite constellation — Wikipedia, the free encyclopedia", https://en.wikipedia.org/wiki/Satellite$_c onstellation$, 2024, $accedit el 22 de maig de$2024.

[Yod24] James Yoder, "Satellite map - real-time visualization of Starlink and other satellite constellations", https://satellitemap.space/, 2024, accedit el 22 de maig de 2024.